



Diplomová práce

Grafické uživatelské rozhraní v režimu okamžitého vykreslování

Studijní program:

N0613A140028 Informační technologie

Autor práce:

Bc. Radek Mocek

Vedoucí práce:

Ing. Jiří Jeníček, Ph.D.

Ústav informačních technologií a elektroniky

Liberec 2026



Zadání diplomové práce

Grafické uživatelské rozhraní v režimu okamžitého vykreslování

Jméno a příjmení:

Bc. Radek Mocek

Osobní číslo:

M24000142

Studijní program:

N0613A140028 Informační technologie

Zadávající katedra:

Ústav informačních technologií a elektroniky

Akademický rok:

2025/2026

Zásady pro vypracování:

1. Proved'te rešerši GUI v režimu okamžitého vykreslování
2. Navrhněte ukázkovou aplikaci
3. Ve vybraných knihovnách implementujte a otestujte aplikaci
4. Zhodnoťte výsledky

<i>Rozsah grafických prací:</i>	Dle potřeby dokumentace
<i>Rozsah pracovní zprávy:</i>	40 – 50 stran
<i>Forma zpracování práce:</i>	tištěná/elektronická
<i>Jazyk práce:</i>	čeština

Seznam odborné literatury:

1. GREYSTONE, E.: Desktop Development in Practice: Building Cross-Platform GUI Applications, 2025, ISBN 979-8293215676
2. Ryder, D. E.: Mastering C++ Programming For GUI Development With Qt, 2024, ISBN 979-8340505392

<i>Vedoucí práce:</i>	Ing. Jiří Jeníček, Ph.D. Ústav informačních technologií a elektroniky
-----------------------	--

<i>Datum zadání práce:</i>	13. října 2025
<i>Předpokládaný termín odevzdání:</i>	11. května 2026

doc. Ing. Josef Černohorský, Ph.D.
děkan

L.S.

doc. Ing. Petr Červa, Ph.D.
garant studijního programu

Prohlášení

Prohlašuji, že svou diplomovou práci jsem vypracoval samostatně jako původní dílo s použitím uvedené literatury a na základě konzultací s vedoucím mé diplomové práce a konzultantem.

Jsem si vědom toho, že na mou diplomovou práci se plně vztahuje zákon č. 121/2000 Sb., o právu autorském, zejména § 60 – školní dílo.

Beru na vědomí, že Technická univerzita v Liberci nezasahuje do mých autorských práv užitím mé diplomové práce pro vnitřní potřebu Technické univerzity v Liberci.

Užiji-li diplomovou práci nebo poskytnu-li licenci k jejímu využití, jsem si vědom povinnosti informovat o této skutečnosti Technickou univerzitou v Liberci; v tomto případě má Technická univerzita v Liberci právo ode mne požadovat úhradu nákladů, které vynaložila na vytvoření díla, až do jejich skutečné výše.

Současně čestně prohlašuji, že text elektronické podoby práce vložený do IS/STAG se shoduje s textem tištěné podoby práce.

Beru na vědomí, že má diplomová práce bude zveřejněna Technickou univerzitou v Liberci v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů.

Jsem si vědom následků, které podle zákona o vysokých školách mohou vyplývat z porušení tohoto prohlášení.

Poděkování

Děkuji vedoucímu své práce Ing. Jiřímu Jeníčkovi, Ph.D.
za poskytnutou podporu.

Grafické uživatelské rozhraní v režimu okamžitého vykreslování

Abstrakt

Tato práce se zabývá paradigmatem označovaným jako IMGUI, což je specifický přístup k vývoji aplikací s grafickým uživatelským rozhraním. Cílem práce je prozkoumat, jak jsou existující knihovny s tímto přístupem vhodné k tvorbě aplikací pro koncové uživatele. Zároveň je tento přístup porovnán s „tradičním“ paradigmatem označovaným jako RMGUI. Ukázkovou aplikací této práce je editor diagramů, který je implementován v knihovnách *Dear ImGui*, *egui* a *Qt*. Během implementace vyšlo najevo, že z pohledu dostupných funkcí nejsou vybrané knihovny s přístupem IMGUI tak vyspělé, jako je knihovna *Qt* s přístupem RMGUI. Zároveň ale přístup IMGUI vede na kratší a přímočařejší kód, což se může projevit i v rychlosti vývoje. Výkon ukázkových aplikací je nakonec otestován ve dvou typech benchmarků. Aplikace v *Dear ImGui* a *egui* mají vyšší režijní náklady, ale oproti knihovně *Qt* se jejich spotřeba po přidání zátěže příliš nemění. Nečekaným výsledkem je nadměrné využití operační paměti knihovnou *egui*, které se objevuje s přibývajícím počtem znaků v textovém poli.

Klíčová slova: grafické uživatelské rozhraní, režim okamžitého vykreslování, *Dear ImGui*, *egui*, *Qt*, editor diagramů

Graphical user interface in immediate rendering mode

Abstract

This thesis examines the ImGui paradigm, a specific approach to developing applications with a graphical user interface. The aim of the thesis is to examine how suitable existing libraries with this approach are for creating user-facing applications. This approach is also compared with the “traditional” paradigm known as RMGUI. The sample application in this thesis is a diagram editor implemented using *Dear ImGui*, *egui*, and *Qt*. During implementation, it became apparent that, in terms of available features, the chosen ImGui libraries are not as powerful as the RMGUI-based Qt. On the other hand, the ImGui paradigm results in shorter and more straightforward code, which may also lead to faster development. The performance of the sample applications is tested in two types of benchmarks. Applications in Dear ImGui and egui have higher overhead, but compared to Qt, their resource usage does not significantly increase under heavier load. An unexpected finding is excessive memory usage by the egui library, which is noticeable as the number of characters in the text box increases.

Keywords: graphical user interface, immediate mode, Dear ImGui, egui, Qt, diagram editor

Obsah

Seznam zkratk	12
1 Úvod	13
2 Grafické uživatelské rozhraní v režimu okamžitého vykreslování	14
2.1 Demonstrace rozdílu mezi přístupy RMGUI a IMGUI	15
2.2 Důsledky přístupu IMGUI	16
2.3 Existující knihovny pro tvorbu GUI aplikací s přístupem IMGUI . . .	19
2.3.1 Knihovna Dear ImGui	20
2.3.2 Knihovna egui	21
2.3.3 Knihovna Nuklear	22
2.3.4 Knihovna microui	22
2.3.5 Knihovna raygui	23
2.3.6 Knihovna Gio	23
2.3.7 Knihovna dvui	24
2.4 Knihovna Qt	24
3 Návrh ukázkové aplikace	25
3.1 Princip popisu diagramu pomocí textu	26
3.1.1 Návrh schématu pro popis diagramu v jazyce TOML	28
4 Implementace ukázkové aplikace	31
4.1 Implementace v knihovně Dear ImGui	31
4.1.1 Tvorba uživatelského rozhraní v knihovně Dear ImGui	33
4.1.2 Víceřádkové textové pole v knihovně Dear ImGui	36
4.1.3 Zpracování vstupního textu v jazyce TOML	38
4.1.4 Vykreslení diagramu v knihovně Dear ImGui	41
4.1.5 Další funkce aplikace v knihovně Dear ImGui	43
4.2 Implementace v knihovně egui	49
4.2.1 Tvorba uživatelského rozhraní v knihovně egui	50
4.2.2 Víceřádkové textové pole v knihovně egui	52
4.2.3 Vykreslení diagramu v knihovně egui	53
4.2.4 Další funkce aplikace v knihovně egui	54
4.3 Implementace v knihovně Qt	55
4.3.1 Vykreslení diagramu v knihovně Qt	57
4.3.2 Další funkce aplikace v knihovně Qt	58

5	Porovnání přístupů a ukázkových aplikací	63
5.1	Porovnání přístupů IMGUI a RMGUI z pohledu funkcí a aplikačního kódu	63
5.2	Porovnání výkonu ukázkových aplikací	65
5.2.1	Porovnání výkonu při vykreslování diagramu	66
5.2.2	Porovnání výkonu při vykreslování ovládacích prvků	67
6	Závěr	69
	Seznam použité literatury	71
A	Příloha	73
B	Ukázkové kódy a výsledky porovnání výkonu ukázkových aplikací	73

Seznam obrázků

2.1	Zpoždění při aktualizaci okna s automatickou velikostí	17
2.2	Knihovny Dear ImGui a Qt v prostředích Windows 10 a KDE Plasma	18
2.3	Knihovny Nuklear, microui, raygui a Gio ve svém výchozím vzhledu .	24
3.1	Drátěný model navrhované ukázkové aplikace	26
3.2	Schéma pro popis uzlů diagramu v jazyce TOML	29
3.3	Schéma pro popis cest diagramu v jazyce TOML	30
4.1	Dear ImGui: oznámení chyby za běhu	33
4.2	Dear ImGui: konflikt identifikátorů pro ovládací prvky	34
4.3	Dear ImGui: problém horizontálního posouvání	35
4.4	Dear ImGui: kontextová nápověda	36
4.5	Dear ImGui: výpočet pozice zvýraznění chyby	37
4.6	Dear ImGui: víceřádkové textové pole	38
4.7	Princip určení přesných pozic uzlů v diagramu	42
4.8	Dear ImGui: panel nástrojů, výběr uzlu	44
4.9	Dear ImGui: změna barvy uzlu	44
4.10	Dear ImGui: změna tvaru uzlu	45
4.11	Dear ImGui: funkce „táhni a pusť“	47
4.12	Dear ImGui: nabídka s ikonami a nativní dialog	48
4.13	Dear ImGui: modální okno varující před neuloženými změnami	48
4.14	Dear ImGui: aplikace v tmavém režimu, okno s nastavením	49
4.15	egui: rozložení ovládacích prvků	51
4.16	egui: dvě okna se stejným názvem	52
4.17	egui: víceřádkové textové pole	53
4.18	Dear ImGui a egui: hlavní a sekundární panel nástrojů	54
4.19	egui: aplikace ve světlém a tmavém režimu, okno s nastavením	55
4.20	Qt: víceřádkové textové pole a problém s ikonami v tmavém režimu .	56
4.21	Qt: nevhodný indikátor neuložených změn v záhlaví okna	57
4.22	Qt: výběr barev v prostředích Windows 10 a Cinnamon	59
4.23	Princip změny úrovně přiblížení vykresleného diagramu	60
5.1	Dear ImGui a Qt: záložka <i>Zobrazení</i> v okně s nastavením	65
5.2	Benchmark vykreslení diagramu: ukáзка	66
5.3	egui: textové pole a využití operační paměti	67
5.4	Benchmark ovládacích prvků v Dear ImGui, egui a Qt	67

B.1	Benchmark vykreslení diagramu: celkové využití procesoru systémem	77
B.2	Benchmark vykreslení diagramu: využití operační paměti procesem	77
B.3	Benchmark vykreslení diagramu: doba trvání mezi dvěma měřeními	77
B.4	Benchmark vykreslení diagramu: doba trvání mezi dvěma měřeními bez textového pole	78
B.5	Benchmark vykreslení diagramu: stejný trend využití procesoru na dvou různých zařízeních	78
B.6	Benchmark ovládacích prvků	78

Seznam tabulek

2.1	Přehled knihoven pro tvorbu GUI aplikací s přístupem ImGui	19
-----	--	----

Seznam zdrojových kódů

3.1	Rozdíl mezi formáty TOML a JSON	27
4.1	Dear ImGui: rozbalovací seznam s tvarem uzlu	46
4.2	Dear ImGui: posuvník v pravé části panelu	46
4.3	egui a Dear ImGui: uzávěry a stavový automat	50
4.4	Qt: výňatek z konstruktoru hlavního okna	57
4.5	Qt: princip signálů	59
B.1	Kód demonstrující přístup ImGui v knihovně Dear ImGui	73
B.2	Kód demonstrující přístup RMGUI v knihovně Qt	74
B.3	Porovnání knihoven: celočíselné vstupní pole	75
B.4	Dear ImGui a Qt: záložka <i>Zobrazení</i> v okně s nastavením	76

Seznam zkratek

ANTLR	<i>ANother Tool for Language Recognition</i> , nástroj pro generaci překladačů
API	<i>application programming interface</i> , rozhraní pro programování aplikací
CPU	<i>central processing unit</i> , procesor
GCC	<i>GNU Compiler Collection</i> , sada překladačů pro jazyky C, C++ ad.
GPU	<i>graphics processing unit</i> , grafický procesor
GUI	<i>graphical user interface</i> , grafické uživatelské rozhraní
IMGUI	<i>immediate mode graphical user interface</i> , grafické uživatelské rozhraní v režimu okamžitého vykreslování
JSON	<i>JavaScript Object Notation</i> , jazyk pro popis struktury dat
MSVC	<i>Microsoft Visual C++</i> , překladač pro jazyky C a C++
MVVM	<i>model-view-viewmodel</i> , návrhový vzor
OS	<i>operační systém</i>
RAM	<i>random-access memory</i> , operační paměť
RFD	<i>Rusty File Dialogs</i> , knihovna v jazyce Rust pro zobrazení nativních dialogových oken
RMGUI	<i>retained mode graphical user interface</i> , grafické uživatelské rozhraní v režimu soběstačného vykreslování
SDL	<i>Simple DirectMedia Layer</i> , multiplatformní multimediální knihovna
SDK	<i>software development kit</i> , sada nástrojů pro vývoj softwaru
SVG	<i>Scalable Vector Graphics</i> , standard pro popis vektorové grafiky
TOML	<i>Tom's Obvious Minimal Language</i> , jazyk pro popis struktury dat
WPF	<i>Windows Presentation Foundation</i> , knihovna umožňující tvorbu GUI aplikací pro operační systém Windows
WYSIWYG	<i>what you see is what you get</i> , typ programu, kde má editovaný dokument stejnou podobu jako výsledný dokument
WYSIWYM	<i>what you see is what you mean</i> , typ programu, kde je výsledný dokument vygenerován z editovaného textového dokumentu
XML	<i>Extensible Markup Language</i> , jazyk pro popis struktury dat
YAML	<i>Yet Another Markup Language</i> , jazyk pro popis struktury dat

1 Úvod

Nedílnou součástí desktopových operačních systémů je aplikační software s grafickým uživatelským rozhraním. Koncoví uživatelé těchto aplikací mohou používat různá zařízení, která se liší mimo jiné v nainstalovaném operačním systému. Při vývoji GUI aplikací proto není od věci cílit na více používaných platformách zároveň. Vývoj multiplatformních GUI aplikací usnadňují knihovny jako *Qt*, *GTK*, nebo *wxWidgets*. Styl psaní aplikačního kódu v těchto knihovnách funguje na stejném principu, kdy jsou definovány ovládací prvky a k nim příslušné callback metody.

Existují však i alternativní přístupy k vývoji GUI aplikací. Jedním z nich je grafické uživatelské rozhraní v režimu okamžitého vykreslování. Na první pohled vede tento přístup ke kratšímu a lépe čitelnému aplikačnímu kódu, k tvorbě „běžných“ aplikací pro koncové uživatele se ale v praxi příliš nepoužívá. Existující knihovny se často používají pro přidání ladících nástrojů do již existující interaktivní aplikace. Nastává tak otázka, jak jsou tyto knihovny vhodné pro tvorbu běžných aplikací s grafickým uživatelským rozhraním.

V této práci je nejprve popsáno, co přesně „okamžitý režim“ v kontextu tvorby GUI aplikací znamená (kapitola 2). Následuje návrh ukázkové aplikace (kapitola 3), která je pak implementována ve třech různých knihovnách (kapitola 4). Kapitola 5 nakonec shrnuje poznatky získané během vývoje. Jednotlivé knihovny jsou porovnány z pohledu aplikačního kódu a dostupných funkcí. U výsledných aplikací je pak porovnán jejich výkon.

Aplikace s grafickým uživatelským rozhraním se skládají z oken a ovládacích prvků (*widgetů*). V této práci používám české názvy ovládacích prvků a pro ujasnění uvádím i názvy anglické. Velká část prvků totiž nemá jednoznačný překlad a existující překlady může být obtížné dohledat. Inspirací mi byl slovník vizuálních prvků komunity KDE [1].

2 Grafické uživatelské rozhraní v režimu okamžitého vykreslování

Grafické uživatelské rozhraní v režimu okamžitého vykreslování (*immediate mode graphical user interface*, zkráceně IMGUI) je specifický přístup k programování aplikačního softwaru s grafickým uživatelským rozhraním. Základní koncept tohoto přístupu veřejně představil Casey Muratori v roce 2005 a s podobnou myšlenkou pracoval i Thierry Excoffier v roce 2003 [2]. Definice přístupu IMGUI není příliš komplexní ani striktní, a proto se postupem času začaly objevovat různé úhly pohledu a interpretace toho, co by okamžitý režim v kontextu vývoje GUI aplikací měl a neměl znamenat. [3]

Autor přístupu IMGUI využil pro jeho pojmenování přívlastek „okamžitý“ z důvodu určité podobnosti k režimu okamžitého vykreslování v některých grafických API té doby [3]. Příkladem je Direct3D, grafické API od společnosti Microsoft, které mohlo dříve pracovat ve dvou režimech: soběstačný (*retained*) a okamžitý (*immediate*). Z pohledu abstrakce je soběstačný režim na vyšší úrovni, kdy si knihovna interně uchovává model 3D scény a od aplikačního kódu přijímá příkazy, které manipulují s objekty v této scéně. Pro vykreslení snímku pak knihovna transformuje informace o objektech ve scéně na sadu příkazů pro vykreslování. V okamžitém režimu se s modelem scény pracuje na straně aplikačního kódu a knihovna přijímá až samotné příkazy pro vykreslování. Tím sice narůstá počet úkonů, které musí být provedeny na straně aplikačního kódu, zároveň se ale zvětšuje flexibilita a prostor pro optimalizace. [4, 5]

Podobně lze nahlížet na programování GUI aplikací. Běžně používané knihovny, které tvorbu GUI aplikací umožňují, jsou v této analogii chápány jako soběstačný režim a přísluší jim zkratka RMGUI (*retained mode graphical user interface*). S využitím těchto knihoven definuje aplikační kód jednotlivé ovládací prvky, u kterých obvykle specifikuje jejich identifikátory, vzájemné rozložení, a případné callback metody. Knihovna pak ve své interní smyčce řeší vykreslování ovládacích prvků a reakce na události. Mezi zástupce tohoto přístupu patří např. knihovny Qt, GTK, nebo WPF. Oproti tomu aplikační kód, který využívá knihovnu s přístupem IMGUI, neobsahuje žádnou inicializační fázi, kde by se definovaly jednotlivé ovládací prvky, ani callback metody, které by reagovaly na určité události. Aplikační kód zde definuje pouze hlavní smyčku, která obsahuje jak příkazy pro vykreslování ovládacích prvků, tak reakce na případné události. Definice přístupu IMGUI explicitně zmiňuje pouze tento rozdíl z pohledu programátora koncové aplikace, implementace samotných knihoven mohou být různé. [3]

2.1 Demonstrace rozdílu mezi přístupy RMGUI a IMGUI

Rozdíl mezi přístupy RMGUI a IMGUI je dobře vidět na ukázce aplikačního kódu. Pro demonstraci byl zvolen jednoduchý program, který dokáže čítat od jedné do pěti a pozdravit uživatele. Program obsahuje jednu stavovou proměnnou, která reprezentuje stav čítače. Její výchozí hodnota je rovna jedné. Dále program obsahuje čtyři ovládací prvky: dvě tlačítka (*button*) a dva textové popisky (*label*). Jeden textový popisec vždy zobrazuje hodnotu čítače. Druhý popisec zobrazuje pozdrav v podobě řetězce „Hello!“, je ale viditelný pouze v případě, kdy je hodnota čítače rovna třem.

Tlačítka jsou označena symboly „+“ a „–“. Po kliknutí na tlačítko se symbolem „+“ se přičte jednička k hodnotě čítače a kliknutím na tlačítko se symbolem „–“ se jednička naopak odečte. Jelikož se má čítat pouze od jedné do pěti, tak je tlačítko se symbolem „–“ neaktivní (*disabled*) v případě, kdy je hodnota čítače rovna jedné. Podobné pravidlo platí pro tlačítko se symbolem „+“, které je neaktivní zase tehdy, pokud je hodnota čítače rovna pěti.

Z důvodu zvýšení přehlednosti neobsahují ukázky logiku pro nastavení vzhledu a rozmístění ovládacích prvků, je zde pouze to důležité pro pochopení principu daného přístupu. Kód B.2 představuje přístup RMGUI v knihovně Qt. Metoda `init` obsahuje logiku inicializační fáze, která postupně nastaví čtyři ovládací prvky a stavovou proměnnou. Všechny ovládací prvky jsou vytvořeny a přidány do rozložení hlavního okna, které je reprezentované proměnnou `layout`. U tlačítek je pomocí funkce `connect` nastavena jejich callback metoda, která se volá po kliknutí na dané tlačítko. Nakonec je čítač nastaven na svou výchozí hodnotu a volá se metoda `on_counter_update`, která zajistí, aby textové popisky a stav tlačítek (zdali jsou nebo nejsou aktivní) odpovídaly aktuální hodnotě čítače. Callback metody upravují hodnotu čítače, a následně opět volají metodu `on_counter_update`. Všechny čtyři ovládací prvky v této ukázce jsou dynamické, a proto se musí jednat o členy třídy `App`, aby k nim bylo možné přistupovat z callback metod.

Kód B.1 představuje přístup IMGUI v knihovně *Dear ImGui*. Metoda `update` obsahuje logiku hlavní smyčky, která je obvykle volána se stejnou frekvencí, jako je obnovovací frekvence zobrazovacího zařízení. Nejprve je čítač nastaven na svou výchozí hodnotu, kterou si díky klíčovému slovu `static` drží i při opakovaném volání metody. Vybraná knihovna se chová jako stavový automat, což je vidět na kódu, který následuje: po zavolání metody `Begin(x)` je vytvořeno okno s názvem `x` a všechny další příkazy se provádí relativně k tomuto oknu, dokud není zavolána metoda `End`. To samé platí pro dvojici metod `BeginDisabled(p)` a `EndDisabled`. Je-li splněna podmínka `p`, pak jsou veškeré ovládací prvky, jejichž vykreslení je definováno mezi voláními těchto dvou metod, označeny jako neaktivní. Metoda `Button` říká, že má být vykresleno dané tlačítko. Zároveň vrací pravdivostní hodnotu, která vyjadřuje, zdali bylo v tomto průchodu smyčkou tlačítko stisknuto. Metoda `Text` pak vykresluje textové popisky. V ukázce přístupu RMGUI je textový popisec s pozdravem objekt pod správou knihovny Qt, u kterého se přepíná viditelnost. Oproti tomu je v přístupu IMGUI vykreslení textu s pozdravem jednoduše podmíněno.

Z rozdílů mezi řešeními pomocí RMGUI a IMGUI, které byly představeny v této ukázce, vyplývají některé přednosti a úskalí obou přístupů. Nejprve je ale třeba zmínit, že problematika není takto černobílá. Jednotlivé architektury a knihovny se mohou lišit v drobných detailech, kombinovat oba přístupy, nebo problém řešit přístupem zcela odlišným. Například, oproti zde představenému řešení v knihovně Qt, by v architektuře MVVM mohla být hodnota textového popisku svázána přímo s proměnnou reprezentující hodnotu čítače. Při jakékoliv změně proměnné `counter` by se pak hodnota popisku `label_counter_value` aktualizovala automaticky.

Vzdálenějším příkladem jsou knihovny pro tvorbu GUI v jazyce JavaScript, jako je např. React. Zde si knihovna obvykle uchovává strom programátorem definovaných komponent ve stylu soběstačného režimu. Z pohledu aplikačního kódu se pak ale logika jednotlivých komponent místy podobá spíše režimu okamžitému. [6]

2.2 Důsledky přístupu IMGUI

Výhody přístupu IMGUI se projevují především při psaní aplikačního kódu. Knihovna neustále přijímá příkazy pro vykreslování samotných ovládacích prvků, a díky tomu si nemusí uchovávat jejich stav. Toto platí alespoň z pohledu programátora koncové aplikace. Nemusí to nutně znamenat, že si knihovna pracující v okamžitém režimu nemůže o ovládacích prvcích uchovávat žádná data [7].

Příkladem myšleného stavu u tlačítka může být jeho textový popis, nebo informace o tom, zdali je tlačítko aktivní. Tento stav je tlačítku předán v každém průchodu hlavní smyčkou jako součást volání příkazů pro jeho vykreslení. Všechny stavové hodnoty jsou uloženy a spravovány pouze na straně aplikačního kódu, a není proto nutné je synchronizovat se stavem, který by si udržovala knihovna.

To, že všechna důležitá data vlastní pouze aplikační kód, vede společně s absencí callback metod na styl kódu, který je oproti přístupu RMGUI kratší a umožňuje držet související logiku více pohromadě. Vlastnost kratšího kódu se projevuje především u dynamických uživatelských rozhraní, kdy je možné měnit stav a existenci ovládacích prvků s každým průchodem hlavní smyčky.

Z hlediska programování samotných knihoven přicházejí s přístupem IMGUI určité implementační výzvy. Překreslování statického GUI šedesátkrát (nebo i vícekrát) za vteřinu by zbytečně zatěžovalo zařízení, a proto je vhodné, aby knihovna dokázala překreslovat jen ty ovládací prvky, které se od minulého průchodu změnily. To znamená, že i když to knihovna nedává nijak najevo, musí si o vykreslovaných ovládacích prvcích interně udržovat určité informace, které přetrvávají déle než jeden průchod hlavní smyčkou.

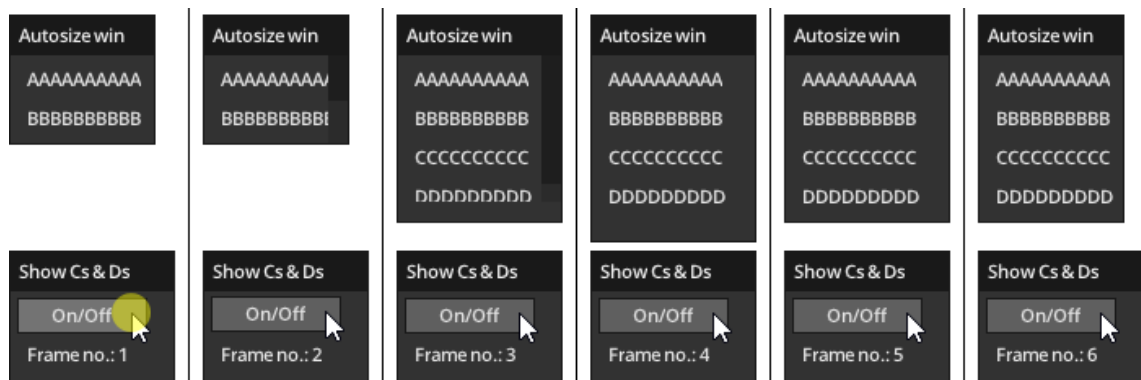
Udržovat si určité informace je potřeba i z důvodu podpory komplexnějšího rozložení ovládacích prvků. Jednoduchým příkladem je nemodální okno¹, které si automaticky určuje svou velikost podle svého obsahu. Je-li jeho obsahem jeden textový popis, pak počet řádků tohoto popisku určuje výšku celého dialogového okna. Chová-li se knihovna jako stavový automat, pak je nejprve potřeba vykreslit dialogové

¹Nemodální okno (*modeless window*) je takové okno, které po svém zobrazení neblokuje interakci s hlavním oknem aplikace.

okno a až poté relativně k němu vykreslit textový popisek. Pokud by se o ovládacích prvcích neuchovávala žádná data, pak by v době vykreslování dialogového okna nebyla existence textového popisku, který se v něm nachází, vůbec známa. Problému nenapomáhá ani fakt, že textový popisek může být dynamický a měnit tak svůj obsah každý snímek.

Řešením může být uchování velikosti textového popisku z předchozích iterací, kde je předpoklad takový, že velikost popisku v aktuální iteraci je stejná, jako byla v iteraci předchozí. Změna velikosti textového popisku se s tímto přístupem projeví na velikosti rodičovského okna se zpožděním minimálně jednoho snímku. Alternativně lze pro každý vykreslený snímek projít hlavní smyčkou dvakrát, kdy první průchod řeší korektní rozložení ovládacích prvků, a druhý průchod pak řeší jejich vykreslení a reakce na události. Dále je možné přidat logiku, která určuje, zdali je pro další snímek nutné projít hlavní smyčkou dvakrát, nebo jen jednou. [3, 8]

Obrázek 2.1 ukazuje šest po sobě jdoucích snímků aplikace, ve které jsou s využitím knihovny *microui* vykreslena dvě okna. Vrchní okno, které na začátku obsahuje dva řádky textu, má nastavený příznak `MU_OPT_AUTOSIZE`. Ten říká, že má dané okno automaticky měnit svou velikost podle svého obsahu. Spodní okno obsahuje tlačítko, které po kliknutí přidá do vrchního okna další dva řádky textu. Dále obsahuje textový popisek, který zobrazuje číslo aktuálního snímku. V prvním snímku je zaregistrováno kliknutí na tlačítko. V druhém snímku už vrchní okno obsahuje čtyři řádky textu, které ale v tu chvíli ještě nejsou vidět. Velikost vrchního okna je ustálena až na snímku číslo šest.



Obrázek 2.1: Zpoždění při aktualizaci okna s automatickou velikostí

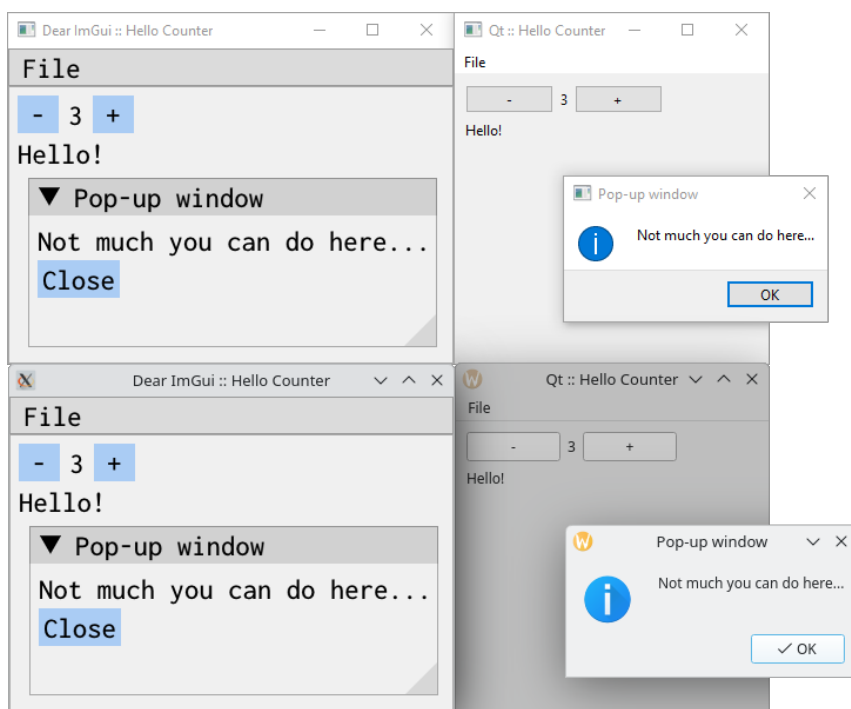
Jak již bylo zmíněno, původní definice ImGui se zaměřuje hlavně na to, jak má vypadat aplikační kód, který využívá knihovny poskytující tento přístup. Teoreticky tedy mohou jednotlivé knihovny využívat libovolných technologií. V praxi mají ale všechny jedno společné: pro zobrazování GUI využívají některého z existujících rozhraní pro vykreslování počítačové grafiky, jako je např. OpenGL, Vulkan, nebo DirectX. V terminologii přístupu ImGui bývá takovéto rozhraní nazýváno *backend*.

Z využití multiplatformního grafického API vyplývá několik vlastností. Výsledná aplikace by měla podporovat všechny platformy, které jsou podporované vybraným backendem, a zároveň bude na všech platformách vypadat stejně. Stejný vzhled na všech platformách nemusí být nutně výhodou. Koncoví uživatelé mohou být zvyklí

na to, že jejich aplikace vypadají „nativně“, tedy že využívají ovládacích prvků poskytnutých daným operačním systémem. Díky tomu, že knihovna zobrazuje GUI posíláním příkazů pro vykreslení vybranému backendu, může být vzhled ovládacích prvků výsledné aplikace velmi přizpůsobitelný včetně možnosti snadno jej měnit i za běhu aplikace. To umožňuje mimo jiné imitovat vzhled nativních aplikací.

To, že bude výsledná aplikace vypadat na všech platformách stejně, platí až na tzv. dekorace okna, jejichž vzhled je ovlivněn operačním systémem a desktopovým prostředím. Knihovny s přístupem ImGui často vykreslují všechny ovládací prvky do jednoho „hostitelského“ okna, které jim poskytuje vybraný backend, a to včetně dialogových oken. Dodatečná okna jsou v tomto případě vykreslována a spravována samotnou knihovnou jako součást hostitelského okna, ze kterého je nelze přesunout „ven“. Nemodální okno bývá základním stavebním prvkem ImGui knihoven s tím, že pro simulaci hlavního okna aplikace se vytvoří dialogové okno o takové velikosti, aby zabralo celý prostor okna hostitelského. U tohoto hlavního okna je nastaveno, aby bylo vždy na pozadí, nevykreslovalo své záhlaví apod.

Obrázek 2.2 ukazuje čtyři instance demonstrační aplikace popsané v sekci 2.1 (čítač s pozdravem), která zde navíc zobrazuje dialogové okno. U instancí implementovaných v knihovně Qt (sloupec vpravo) není dialogové okno součástí okna hlavního, a proto jej lze přesunout na libovolné místo. Zároveň je ale vzhled tohoto okna závislý na vybrané platformě².



Obrázek 2.2: Knihovny Dear ImGui a Qt v prostředích Windows 10 a KDE Plasma

²Zde je třeba zmínit, že knihovna Qt již od verze 5 nabízí multiplatformní styl *Fusion* [9]. Lze tedy docílit stejného vzhledu aplikace na všech platformách. Ne vždy je ovšem tento styl výchozí.

Vzhled aplikace není jediná vlastnost, na kterou mohou být uživatelé zvyklí. Dalším příkladem může být horní lišta nabídek (*menu bar*), u které se očekává možnost navigace pomocí klávesových zkratk. Ne každá aplikace sice potřebuje disponovat lištou nabídek, každopádně její případné zobrazení je např. v knihovně Qt záležitostí volání jedné metody. U knihoven s přístupem ImGui toto není zaručeno a logika lišty nabídek musí být explicitně implementována buď ze strany knihovny, nebo až v aplikačním kódu. Totéž platí např. pro možnosti přístupu, jako je respektování velikosti písma nastavené v rámci operačního systému, nebo podpora odečítače obrazovky (*screen reader*).

Knihovny s přístupem ImGui se často používají pro přidání ovládacích prvků do existující interaktivní aplikace, která využívá stejný backend. Taková aplikace už má definovanou svou vykreslovací smyčku, do které lze snadno přidat příkazy pro vykreslování GUI. To je jeden z důvodů, proč bývá u těchto knihoven základním stavebním prvkem právě nemodální okno. Tato okna jsou vykreslována nad existující interaktivní aplikací, která o nich v podstatě „neví“, a obsahují např. ovládací prvky pro ladění nebo změnu určitých parametrů za běhu.

2.3 Existující knihovny pro tvorbu GUI aplikací s přístupem ImGui

I přesto, že přístup ImGui je poměrně starý koncept a na poli tvorby „běžných“ GUI aplikací dominuje přístup RMGUI, jsou některé ImGui knihovny stále ve vývoji a nachází svá uplatnění. Tabulka 2.1 představuje knihovny, které jsou součástí této rešerše. Existuje mnoho projektů, které nějakým způsobem využívají knihovnu Dear ImGui ve smyslu, že ji buď rozšiřují, nebo převádí do ekosystému jiného programovacího jazyka. Tato rešerše se soustředí na knihovny, které přichází s kompletně vlastním řešením. Kromě projektu microui, který měl poslední aktualizaci v srpnu roku 2024, jsou v době psaní této rešerše všechny knihovny v aktivním vývoji.

Tabulka 2.1: Přehled knihoven pro tvorbu GUI aplikací s přístupem ImGui

Název	Jazyk	Licence	Vznik	Popularita*	Nedávná**
Dear ImGui	C++	MIT	2014	69 315	2 582
egui	Rust	MIT, Apache-2.0	2018	27 111	1 253
Nuklear	C	MIT, Unlicense	2015	10 554	376
microui	C	MIT	2018	5 140	507
raygui	C	Zlib	2014	4 437	296
Gio	Go	MIT, Unlicense	2019	2 059	79
dvui	Zig	MIT	2022	993	273

* počet hvězd na platformě GitHub ke dni 12. 11. 2025

**nárůst počtu hvězd na platformě GitHub mezi dny 12. 11. 2025 a 12. 3. 2026

Protože ukázkovou aplikací této práce je editor diagramů, kde je diagram vykreslen na základě zadaného textového popisu, zaměřuje se řešerše knihoven více na ty aspekty, které budou při tvorbě takové aplikace určitě potřeba. Je pravděpodobné, že všechny knihovny budou disponovat ovládacími prvky jako tlačítko nebo textové pole (*text box*). Otázkou ale je, jaké mají tyto prvky funkce. U textového pole se např. hodí zvýrazňování syntaxe, hledání v textu, našeptávání, klávesové zkratky pro kopírování a vkládání apod. Druhou důležitou částí vedle textového editoru je kreslicí plátno (*canvas*), které obsahuje vykreslený diagram. Pro jeho realizaci by knihovna měla poskytovat funkce kreslení grafických primitiv.

2.3.1 Knihovna Dear ImGui

Dear ImGui [7] je knihovna pro tvorbu GUI s přístupem IMGUI implementovaná v jazyce C++. Její kód splňuje standard *C++11* a v základu nepoužívá žádné standardní knihovny. Dear ImGui se často využívá ve videoherním průmyslu k tvorbě dynamických nástrojů pro ladění. Knihovna je psaná tak, aby šla snadno integrovat do existujícího projektu s libovolným backendem. Základním stavebním prvkem je proto nemodální okno. Knihovna umožňuje vytvořit okno bez dekorací, které zůstává na pozadí a zabírá celý prostor okna hostitelského.

Knihovna nemá žádnou oficiální dokumentaci, která by pokrývala všechny její funkce. Poskytuje ale metodu pro zobrazení ukázkového okna, které obsahuje dostupné ovládací prvky v různých nastaveních. Nastavením ovládacího prvku se v tomto případě myslí dodání proměnné s určitými příznaky (*flags*). U textového pole existují např. příznaky typu „pouze pro čtení“ nebo „zalamuj dlouhé řádky“. Najde-li programátor v ukázkovém okně chtěnou funkci, její implementaci pak snadno dohledá ve zdrojovém kódu tohoto okna. Pomoci mohou také občasné komentáře ve zdrojovém kódu samotné knihovny, nebo dokumentace některé z odvozených knihoven. I přes absenci oficiální dokumentace nebývá díky relativně velké komunitě problém najít odpovědi na základní problémy. Velká komunita okolo knihovny Dear ImGui se projevuje také existencí řady doplňků. Časté jsou doplňky pro animace, grafy, prohlížení souborů, nebo pokročilejší textové editory.

Dear ImGui poskytuje ovládací prvek víceřádkového textového pole, který lze zobrazit voláním metody `InputTextMultiline`. Oproti většině ostatních knihoven s přístupem IMGUI má zdejší textové pole relativně hodně funkcí. Ve své výchozí podobě vykresluje obsah staticky alokovaného pole znaků, součástí knihovny je ale i volitelná implementace pro třídu `std::string`. Je-li použito vlastní písmo, které podporuje znaky české abecedy, není pak problém tyto znaky zobrazit v textovém poli. Výchozí písmo, které je součástí knihovny, totiž všechny znaky české abecedy neobsahuje. Zvýrazňování syntaxe tento ovládací prvek nepodporuje, veškerý text může mít pouze jednu společnou barvu.

V textovém poli je možné myší označovat text a ten pak pomocí klávesových zkratek vyjmout, kopírovat, nebo vložit. Klávesami kurzorových šipek lze posouvat textovým kurzorem (karetou), kdy se s přidržením klávesy *Ctrl* přeskakují celá slova a s klávesou *Shift* se text označuje. Klávesu *Ctrl* lze také použít v kombinaci

s klávesou *Backspace* nebo *Delete* pro mazání celých slov. Dále jsou k dispozici klávesové zkratky pro zrušení poslední změny (*undo*) a její obnovení (*redo*).

Má-li editovaný dokument větší množství řádků, objeví se na pravé straně textového pole vertikální posuvná lišta (*scrollbar*). Horizontální posuvná lišta ale není podporována. Pokud je tedy nějaký řádek textu příliš dlouhý, tak se na jeho konec uživatel nejlépe dostane posunem textového kurzoru pomocí kláves. Přidání horizontální posuvné lišty by se údajně příliš negativně podepsalo na výkonu. Textové pole interně využívá „jednohlavičkovou“ (*header-only*) knihovnu *stb-textedit* a v plánu je jeho kompletní přepsání [10]. Ve verzi 1.92 (červen 2025) byla každopádně přidána možnost zalamovat příliš dlouhé řádky.

Pro vykreslování grafických primitiv je možné si vyžádat ukazatel na strukturu *ImDrawList*, která obsahuje vykreslovací příkazy pro své okno. Tato struktura poskytuje metody pro přidání dalších objektů k vykreslení. Jedná se např. o pravidelný mnohoúhelník, kruh, elipsu, úsečku, nebo text.

2.3.2 Knihovna egui

Egui [8] je knihovna pro tvorbu GUI s přístupem IMGUI implementovaná v jazyce Rust. Její kód je napsán v edici jazyka *Rust 2024* a používá několik dalších závislostí. Knihovna je psaná tak, aby šla integrovat do existujícího projektu s libovolným backendem. Od stejného autora je k dispozici také knihovna *eframe*, která slouží jako oficiální „framework“ pro knihovnu egui. Tento framework podporuje dva backendy: OpenGL s využitím knihovny *glow* a WebGPU s využitím knihovny *wgpu*. Kromě tří hlavních desktopových systémů umožňuje *eframe* také kompilaci pro OS Android a do webové aplikace pomocí WebAssembly. K dispozici je ještě balík *egui-extras*, který přidává další funkce. Je ale označen jako experimentální a nese s sebou větší množství závislostí.

Pro přidání uživatelského rozhraní do existující interaktivní aplikace je opět k dispozici nemožné okno. K tvorbě GUI aplikace, která má zabírat celý prostor hostitelského okna, pak lze použít tzv. panely. Ty umožňují rozdělit rozhraní na oblasti typu „horní lišta“, „spodní lišta“, nebo „hlavní obsah“. Panely mohou takto rozdělovat libovolné okno včetně toho hostitelského. Do panelů pak již lze umísťovat vybrané ovládací prvky.

Knihovna je z velké části zdokumentovaná, a pokud programátor narazí na neznámou funkci, neměl by být problém dohledat její význam. Obtížnější mohou být situace, kdy si programátor není jist, jaké funkce přesně použít pro vyřešení konkrétního problému. K dispozici je však několik ukázkových projektů, které pokrývají alespoň základní funkce knihovny. Případné problémy lze řešit v sekci diskuzí na platformě GitHub. I zde je k dispozici řada komunitních doplňků a integrací pro různé backendy.

Egui poskytuje ovládací prvek víceřádkového textového pole, který lze zobrazit voláním metody *multiline* ze struktury *TextEdit*. Textové pole vykresluje obsah řetězce, který dostává jako měnitelnou (*mutable*) referenci na typ *String*. Z pohledu počtu podporovaných funkcí konkuruje tento ovládací prvek textovému poli z knihovny Dear ImGui. K dispozici jsou všechny zmíněné klávesové zkratky

a výchozí písmo podporuje všechny znaky české abecedy. Ve své základní podobě prvek nepodporuje zvýrazňování syntaxe a automaticky zalamuje dlouhé řádky bez možnosti zobrazení horizontální posuvné lišty. Balík *egui-extras* pak obsahuje modul pro zvýrazňování syntaxe.

K vykreslování grafických primitiv je možné využít pomocnou strukturu `Painter`, která slouží pro kreslení tvarů a textu v určité oblasti. Výběr tvarů je podobný jako u struktury `ImDrawList` v knihovně Dear ImGui.

2.3.3 Knihovna Nuklear

Nuklear (dříve *Zahnrad*) [11] je knihovna pro tvorbu GUI s přístupem ImGui implementovaná ve standardu C89 jazyka C. Podobně jako Dear ImGui je psána tak, aby šla integrovat do projektu s libovolným backendem. Základním prvkem je také nemodální okno, které lze opět nastavit tak, aby zabíralo celý prostor okna hostitelského. Nuklear je jednohlavičková knihovna o zhruba 18 tisících řádcích kódu. Oproti předchozím dvěma knihovnám má tedy značně užší rozsah.

Ke knihovně existuje automaticky generovaná dokumentace z komentářů ve zdrojovém kódu pomocí nástroje *Doxygen*. Velká část kódu ale okomentovaná není a samotná dokumentace v podobě webové stránky není příliš přehledná. Pro pochopení knihovny je vhodnější komunitní wiki na platformě GitHub [11], kde jsou mimo jiné představeny všechny dostupné ovládací prvky. Tato wiki není nikde explicitně zmiňována a může tak být programátorem snadno přehlédnuta.

Nuklear poskytuje ovládací prvek víceřádkového textového pole, který lze zobrazit voláním funkce `nk_edit_string_zero_terminated`. Ten vykresluje obsah staticky alokovaného pole znaků. Po specifikování správných rozsahů a importování vhodného písma dokáže knihovna vykreslovat znaky české abecedy. Textové pole umožňuje pohybovat s textovým kurzorem a označovat text pomocí myši. Označené znaky pak lze hromadně mazat.

Textové pole nepodporuje některé běžné klávesy, jako je např. *Home* a *End*. Výše zmíněné klávesové zkratky také nejsou k dispozici. Z posuvných lišt je opět vykreslena pouze ta vertikální a zalamování dlouhých řádků není podporováno. U některých kláves je problém, že ihned po svém stisknutí berou vstup jako „opakovaný“. Chovají se tedy tak, jakoby uživatel klávesu držel. Toto se děje mimo jiné u klávesy *Backspace*, a je proto např. obtížné smazat jen jeden znak v textu. Knihovna má dále připravenou strukturu `nk_text_edit`, která by měla umožnit definovat robustnější textové pole s podporou schránky a historie změn, čímž by se ovládací prvek více vyrovnal předchozím dvěma knihovnám. Její konfigurace už je ale složitější a není příliš zdokumentovaná, proto jsem se jí dále nevěnoval.

2.3.4 Knihovna microui

Microui [12] je knihovna pro tvorbu GUI s přístupem ImGui implementovaná v jazyce C. Jedná se o minimalistickou knihovnu, jejíž kód má zhruba 1100 řádků a lze ji integrovat do projektu s libovolným backendem. Jediné oficiálně poskytnuté řešení je integrace pro OpenGL v kombinaci s knihovnou SDL. Jelikož má knihovna

microui minimální rozsah, tak jako dokumentace postačí základní popis a ukázky kódu v jejím repozitáři.

Ovládací prvky lze v knihovně microui vykreslovat přímo do „prázdného“ hostitelského okna, je ale možné vytvářet i nemodální okna. Víceřádkové textové pole není podporováno a k dispozici je pouze jednořádková varianta, kterou lze vykreslit voláním funkce `mu_textbox`. V tomto textovém poli lze pouze přidávat nebo ubírat znaky. S textovým kurzorem není možné pohybovat a k dispozici nejsou ani žádné jiné funkce. Znaký české abecedy nejsou podporovány a bez dodání vlastní logiky není možné importovat jiné písmo.

2.3.5 Knihovna raygui

Raygui [13] je jednohlavičková knihovna pro tvorbu GUI s přístupem ImGui implementovaná v jazyce C. Knihovna byla vytvořena primárně jako modul do projektu *raylib*, je ale možné ji integrovat s libovolným backendem. Knihovna nemá žádnou dokumentaci, její zdrojový kód je však dostatečně okomentovaný.

Ovládací prvky lze v raygui vykreslovat přímo do prázdného hostitelského okna a funkce pro zobrazení dodatečných dialogových oken jsou omezené. Jedná se o jedinou knihovnu v této rešerši, která neposkytuje žádnou formu dynamického rozložení, a tak je nutné pozice a velikosti všech ovládacích prvků definovat absolutně. Od stejného autora je pak k dispozici vizuální editor *rGuiLayout*.

Víceřádkové textové pole není podporováno a k dispozici je pouze jednořádková varianta, kterou lze vykreslit voláním funkce `GuiTextBox`. V textovém poli je kromě přidávání a ubírání znaků možné měnit pozici textového kurzoru. Po importování vlastního písma a nastavení správných rozsahů lze vykreslovat znaký české abecedy.

2.3.6 Knihovna Gio

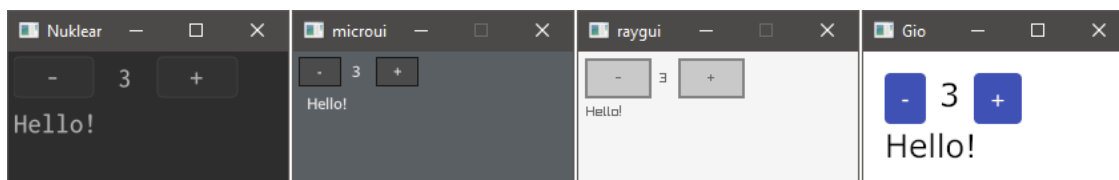
Gio [14] je knihovna pro tvorbu GUI s přístupem ImGui implementovaná v jazyce Go. Knihovna se zaměřuje výhradně na tvorbu běžných aplikací pro koncové uživatele, a nelze ji tak snadno integrovat do existujícího projektu. Práce s backendem je před programátorem skryta a ovládací prvky se vykreslují přímo do hostitelského okna. Ke knihovně existuje automaticky generovaná dokumentace podle definic ve zdrojovém kódu. Na oficiálním webu jsou k dispozici i ukázkové kódy, které ovšem nejsou příliš dobře uspořádané.

Gio poskytuje ovládací prvek víceřádkového textového pole v podobě struktury zvané `Editor`, která automaticky zalamuje dlouhé řádky. Vertikální posuvná lišta není k dispozici, a tak lze s textem posouvat pouze kolečkem myši nebo změnou pozice textového kurzoru. Textové pole umožňuje zrušení a obnovení poslední změny a výchozí písmo podporuje všechny znaký české abecedy. Je také možné označovat text a manipulovat s ním, ale nelze přeskakovat či mazat celá slova. Zvýrazňování syntaxe není k dispozici.

V knihovně Gio se styl psaní aplikačního kódu nejvíce odlišuje od ostatních zmíněných knihoven. Před hlavní smyčkou je nejprve nutné definovat všechny ovládací

prvky, které se v aplikaci vyskytují. V hlavní smyčce jsou pak oddělené fáze aktualizace stavu a vykreslení. Do první fáze patří např. obsluha stisknutých tlačítek. V druhé fázi se definuje rozložení a vzhled ovládacích prvků.

Rozložení ovládacích prvků se definuje relativně robustním způsobem pomocí vnořování anonymních funkcí. I pro vytvoření menší aplikace je tak potřeba větší množství kódu. Např. u demonstrační aplikace čítače s pozdravem (obrázek 2.3) bylo pro definici rozložení čtyř ovládacích prvků potřeba více než 40 řádků kódu.



Obrázek 2.3: Knihovny Nuklear, microui, raygui a Gio ve svém výchozím vzhledu

2.3.7 Knihovna dvui

Dvui [15] je knihovna pro tvorbu GUI s přístupem ImGui implementovaná v jazyce Zig. Knihovna je psaná tak, aby šla integrovat do existujícího projektu s libovolným backendem, ale zároveň cílí i na tvorbu běžných aplikací pro koncové uživatele. Ovládací prvky lze tedy vykreslovat přímo do hostitelského okna, nebo do dodatečných nemodálních oken. Podle definic ve zdrojovém kódu existuje ke knihovně automaticky generovaná dokumentace. Dále jsou k dispozici ukázkové kódy, které pokrývají její základní funkce.

Dvui poskytuje ovládací prvek víceřádkového textového pole, který lze zobrazit voláním metody `textEntry`. Tento ovládací prvek je relativně schopný. Dokáže zobrazit obě posuvné lišty a volitelně zapnout zalamování řádků. Je v něm možné označovat text, manipulovat s ním a přeskakovat či mazat celá slova. Při zvolení vhodného písma není problém s vykreslováním znaků české abecedy. Textové pole nepodporuje zrušení a obnovení poslední změny. K dispozici není také zvýrazňování syntaxe.

2.4 Knihovna Qt

Qt [16] je knihovna pro tvorbu GUI s přístupem RMGUI implementovaná v jazyce C++. Společnost *Qt Group* zastřešuje vývoj knihovny a poskytuje k ní různé komerční i open-source licence, z nichž nejvíce permissivní je licence LGPL. Jedná se o široce používanou knihovnu, a proto je v této práci vybrána jako zástupce přístupu RMGUI.

Existuje více způsobů, jak pomocí knihovny Qt definovat uživatelské rozhraní. Časté je použití programu *Qt Creator*, který obsahuje vizuální editor. Jelikož jsou z výše uvedených knihoven Dear ImGui a raygui jediné, pro které existují vizuální editory, zaměřuji se v této práci na definici uživatelského rozhraní z aplikačního kódu. I v knihovně Qt bude tedy GUI definováno pomocí jazyka C++.

3 Návrh ukázkové aplikace

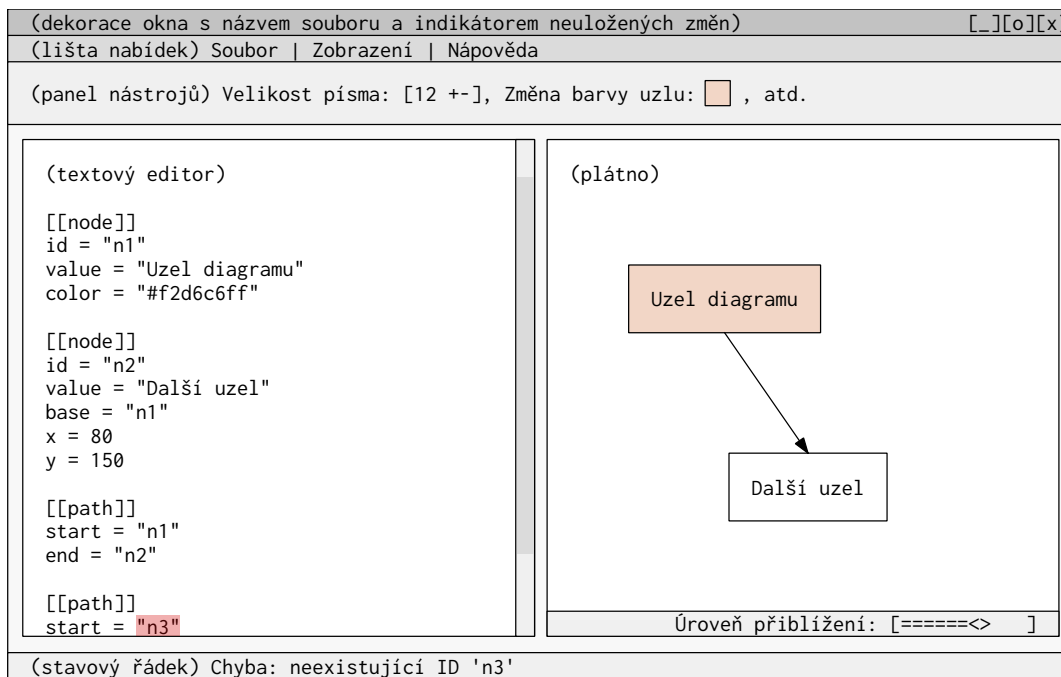
Navrhovanou ukázkovou aplikací této práce je program pro tvorbu a editaci diagramů. Diagram je vcelku obecný pojem a cílem není pokrýt všechny jeho možné varianty. V kontextu této práce se pojmem *diagram* myslí především vývojový diagram, nebo blokové schéma.

Navrhovaná aplikace na základě zadaného textového popisu vykreslí odpovídající diagram. Dělí se tedy na dvě hlavní části: víceřádkové textové pole a kreslicí plátno. Textové pole je základní ovládací prvek, který je ale relativně komplexní, a proto je vhodné jej použít jako jeden z hlavních stavebních kamenů ukázkové aplikace. Plátno pak pomůže prozkoumat schopnost knihoven vykreslovat programátorem definované objekty. Tedy vykreslování i něčeho jiného, než jsou základní ovládací prvky, které knihovna poskytuje.

Při programování GUI aplikací se ale často pracuje právě se základními ovládacími prvky, a proto by je ukázková aplikace měla využívat. Nemělo by se však jednat pouze o „galerii ovládacích prvků“, kterou lze často najít jako příklad v dokumentaci dané knihovny. Použité ovládací prvky by měly dávat smysl v kontextu editoru diagramů. Proto navrhovaná aplikace obsahuje např. celočíselné vstupní pole (*spinbox*) pro změnu velikosti písma, posuvník (*slider*) pro změnu úrovně přiblížení pohledu na diagram, nebo výběr barvy (*color picker*), který umožní změnit barvu uzlu bez nutnosti zásahu do zdrojového textu daného diagramu.

Navrhovaná aplikace dále disponuje funkcemi, které by uživatel od programu tohoto typu mohl očekávat. Tvorba diagramu zde spočívá v editaci textového řetězce, a je tedy na místě, aby měl uživatel možnost jej uložit nebo nahrát ze souboru. Proto je k dispozici lišta nabídek obsahující akce, které pro tento účel vyvolají nativní dialogové okno. Záhloví hlavního okna pak obsahuje cestu k právě editovanému souboru a zároveň indikuje neuložené změny pomocí znaku hvězdičky. Při pokusu o zavření aplikace je v případě neuložených změn uživatel vyzván k potvrzení, zdali chce změny uložit, nebo zahodit. Další akcí v liště nabídek je vyvolání okna pro nastavení. Zde lze především přepnout vzhled aplikace do tmavého režimu, což je mezi uživateli často žádanou funkcí [17]. Aplikace by také měla bez kompromisů podporovat operační systémy Windows a Linux.

Krom toho, že aplikace poslouží pro porovnání vybraných knihoven a disponuje očekávanými funkcemi, by se měla blížit k reálně použitelnému programu pro tvorbu diagramů. Proto je k dispozici funkce, která popsany diagram exportuje do formátu SVG. Dále je potřeba navrhnout, jak bude diagram pomocí textu přesně popisován.



Obrázek 3.1: Drátěný model navrhované ukázkové aplikace

3.1 Princip popisu diagramu pomocí textu

Existující aplikace, které umožňují tvorbu diagramů, lze rozdělit do dvou kategorií. První jsou programy využívající přístup WYSIWYG, kde uživatel interaguje přímo s objekty diagramu vykreslenými na plátně. Do této kategorie se řadí např. programy *draw.io* nebo *yEd*. Druhou kategorií je přístup WYSIWYM. V něm uživatel píše kód, který popisuje, co by měl daný diagram vyobrazovat. Do této kategorie patří např. programy *Mermaid* nebo *Graphviz*. Jelikož kód popisuje především význam daného diagramu, tak nemá uživatel velkou kontrolu nad jeho vzhledem. Kód může např. popisovat vztahy mezi jednotlivými uzly. Přesné umístění těchto uzlů v diagramu už ale určuje samotný program.

Ukázková aplikace této práce se snaží o unikátní přístup k tomuto problému. Diagram je také generován na základě dodaného kódu, tento kód ovšem popisuje, jak by měl diagram přesně vypadat. Popis přesného vzhledu v podstatě probíhá i při použití formátu SVG. Export do tohoto formátu je navíc dle návrhu jediný způsob, jak může uživatel vykreslit svůj diagram mimo program. Přímé použití formátu SVG pro popis diagramu je ale nevhodné. Z důvodu své všestrannosti a syntaxe jazyka XML má tento formát vyšší režijní náklady ve smyslu délky kódu, který je potřeba pro popsání libovolného diagramu.

Nejideálnější by bylo použít doménově specifický jazyk, který by byl od začátku navržen pro účely této aplikace. Tuto myšlenku jsem ale v rané fázi vývoje opustil. Důvodem byla těžko odhadnutelná časová náročnost návrhu a implementace takového jazyka. Nástroj ANTLR, který bych pravděpodobně použil, navíc

```
# Dva uzly diagramu odlišného typu, formát TOML:
[[Uzel]]
typ = "obdélník"

[[Uzel]]
typ = "elipsa"

# Dva uzly diagramu odlišného typu, formát JSON:
{
  "Uzel": [
    {"typ": "obdélník"},
    {"typ": "elipsa"}
  ]
}
```

Zdrojový kód 3.1: Rozdíl mezi formáty TOML a JSON

nemá v ekosystému jazyka Rust velkou podporu. Proto jsem se rozhodl pro použití existujícího jazyka.

Jelikož se jedná o unikátní přístup k popisu diagramů a návrh vlastního jazyka neuvažuji, dává smysl použít existující jazyk pro popis struktury dat. Tyto jazyky jsou strojově čitelné, je zde ovšem nutné klást důraz i na to, jak přívětivé bude pro uživatele zdrojový text číst a editovat. Jazyk XML je robustní a umožňuje tvorbu vlastního schématu, které pak lze snadno validovat. Navíc by bylo možné definovat šablonu pro transformaci dokumentu do formátu SVG. Jak již ale bylo zmíněno, ruční psaní tohoto jazyka je bez větší podpory od textového editoru relativně pomalé.

Formát JSON je z pohledu počtu znaků stručnější, ale stále obsahuje určité režijní náklady. Tím je myšlena např. povinnost umisťovat názvy klíčů do dvojitých uvozovek. U složitějších struktur navíc dochází ke vnořování objektů, kdy je pro zpřehlednění obvykle používáno odsazení. To buď aplikuje textový editor automaticky, nebo si jej musí hlídat uživatel. Není-li odsazení použito, může se uživatel snadno „ztratit“ ve velkém množství uzavírajících složených závorek.

Nejprve jsem zvažoval použití formátu YAML, nakonec jsem se ale rozhodl pro formát TOML. Ten se pro definici struktur nespolehá na bílé znaky a poskytuje typ zvaný tabulka (*table*) [18]. Pomocí tabulek lze dokument rozdělit do sekcí poměrně čitelným a stručným způsobem. Každá sekce pak může např. popisovat vlastní uzel diagramu, což je naznačeno v kódu 3.1.

Po zvolení jazyka TOML následoval návrh schématu. Je třeba zmínit, že pro tento jazyk neexistují zavedené nástroje pro definici a validaci schémat, jako je tomu např. u jazyka XML. Schématem se zde myslí pouze to, jaké tabulky jsou na vstupu očekávány. Pro každý možný typ tabulky se pak očekávají určité názvy klíčů a datové typy jejich hodnot. Validace schématu tak musí probíhat až v aplikačním kódu při procházení stromu, který poskytne knihovna pro překlad jazyka TOML.

3.1.1 Návrh schématu pro popis diagramu v jazyce TOML

Diagram v navrhované aplikaci se skládá z objektů dvou typů: uzel (*node*) a cesta (*path*). Tyto objekty jsou součástí příslušné kolekce tabulek a ve zdrojovém textu začíná jejich definice řádkem `[[node]]` nebo `[[path]]`. Popis diagramu by měl být „agilní“. Pozdější zásah do jeho struktury by tedy neměl vyžadovat velké množství přepisování již existujícího kódu. Proto jsou na objekty kladeny určité požadavky.

Uzel je ve své základní podobě obdélník obsahující text. Pro tvorbu jednoduchého blokového schématu je potřeba mít pro každý uzel možnost specifikovat jeho umístění, velikost a textový obsah. Pro snadnější editaci existujících diagramů je zde požadavek, aby bylo možné definovat pozici uzlu relativně k jinému uzlu. Tím vznikají vztahy typu *rodič–potomek* a při změně pozice rodičovského uzlu se změní i pozice všech jeho potomků. Pokud by bylo možné definovat umístění uzlů pouze absolutně, velmi by to zkomplikovalo editaci existujících diagramů. Dalším požadavkem je, aby bylo možné uzly definovat v libovolném pořadí, tedy aby mohl být ve zdrojovém textu potomek definován dříve než jeho rodič. To sice zkomplikuje logiku na straně aplikačního kódu, ale zlepší uživatelský zážitek.

Pozice uzlu je určena nastavením hodnot pro klíče *x* a *y*, které odpovídají souřadnicím kreslicího plátna. Co ale přesně souřadnice pro daný uzel znamenají? Mohl by zde být umístěn jeho střed, nebo také některý z rohů vykresleného obdélníka. Proto jsem přidal klíč *pivot*, kde lze toto chování pro každý uzel nastavit. Hodnotou tohoto klíče je řetězec, např. `"center"` nebo `"top-left"`. V doménově specifickém jazyce by bylo možné mít pro tento účel dedikovaná klíčová slova, a díky tomu by byl neplatný *pivot* zachycen již během překladu. V tomto případě je nutné kontrolovat správnost argumentu až v pozdější fázi.

Otázka přesného umístění vykreslených uzlů nastává také u relativních souřadnic. Pokud by souřadnice potomka byly relativní např. ke středu rodiče, byla by vyřešena problematika přesunů, ale ne změn velikosti. Příkladem je diagram se dvěma uzly ve vztahu rodič–potomek, kde je rodičovův *pivot* nastaven na střed. Uživatel si přeje, aby byl potomek umístěn pět jednotek pod svým rodičem. Změna pozice rodiče by toto zachovala, ale po změně jeho výšky by bylo nutné zvýšit ypsilonovou souřadnici potomka, protože se střed rodiče nijak neposunul. Proto jsem přidal možnost definovat *pivot* rodiče, ke kterému jsou souřadnice potomka relativní. Obrázek 3.2 ukazuje popsání scénář a také prozrazuje další vlastnosti navrženého schématu.

Definice vztahů rodič–potomek fungují na bázi identifikátorů typu textového řetězce. Potomek uvádí identifikátor a *pivot* svého rodiče, který se tak stává počátkem pro jeho souřadnice. Dále je vidět, že pokud není pro uzel nastavena velikost explicitně, použije se velikost implicitní, která je určena textovým popiskem uzlu. K dispozici jsou pak další parametry pro úpravu vzhledu uzlu. Je možné např. změnit jeho tvar nebo barvu výplně.

Po navržení první verze schématu pro popis uzlů diagramu jsem jej otestoval. Zkusil jsem v něm popsat vybrané blokové schéma a došel k několika závěrům. Z testování vyšlo najevo, že je potřeba vyřešit situace, kdy se více uzlů překrývá. Pořadí vykreslování překrývajících se uzlů by šlo určit např. ze zdrojového textu, kdy uzel, který je definován dříve, je také vykreslen dříve. Rozhodl jsem se ale pro nový

TOML schéma verze 1	TOML schéma verze 2	Odpovídající diagram
<pre>[[node]] id = "parent_id" value = "Rodič" pivot = "center" x = 50 y = 50 width = 150 height = 100 [[node]] value = "Potomek" pivot = "top" base = ["parent_id", "bottom"] x = 0 y = 5</pre>	<pre>[node."parent_id"] value = "Rodič" pivot = "center" xy = [50, 50] size = [150, 100] [node."child_id"] value = "Potomek" pivot = "top" xy = ["parent_id", "bottom", 0, 5]</pre>	

Obrázek 3.2: Schéma pro popis uzlů diagramu v jazyce TOML

parametr z , který toto určí explicitně. Uzel s vyšší hodnotou parametru překryje uzly, které mají hodnotu nižší. Díky tomu lze snadno přepínat „hloubku“ uzlů bez nutnosti přesouvat větší množství řádků ve zdrojovém textu. Pokud ale hodnota parametru z není nastavena, pak pořadí vykreslování není nijak definováno a může být závislé na konkrétní implementaci.

Z testu také vyšlo najevo, že popis diagramu je sice relativně stručný a přehledný, ale zabírá velké množství řádků. Pro danou výšku textového pole tedy nebylo najednou zobrazeno příliš mnoho informací. Proto jsem navrhl druhou verzi schématu, která seskupuje některé parametry dohromady, a tím snižuje nutnost posouvat řádky. Např. místo parametrů x , y a $base$ je zde jeden parametr xy , který očekává buď dvouprvkové pole se souřadnicemi, nebo čtyřprvkové pole, které navíc obsahuje informace o rodičovském uzlu. Uzel má v této verzi na prvním řádku své definice uveden i svůj identifikátor, ze kterého se proto stává povinný údaj. Rozdíl mezi dvěma verzemi návrhu je vidět na obrázku 3.2.

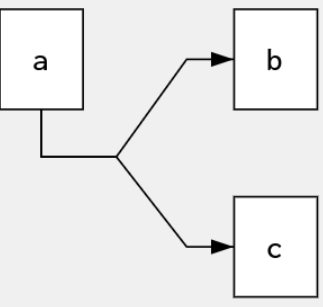
Poslední poznatek z testování je ten, že by bylo vhodnější mít možnost nastavit dva různé rodičovské uzly, kdy jeden rodič určuje umístění potomka na ose x a druhý na ose y . Tím by se zjednodušila tvorba a úprava diagramů, kde mají být uzly rozmístěny v pravidelné mřížce. Tento problém jsem v kontextu celé práce vyhodnotil jako nepřilíh zásadní a pokračoval na návrh popisu cest.

Cesty v diagramu jsou definovány svými body. Pro validní cestu je nutné definovat její startovní a cílový bod. Mezi nimi pak může být definován libovolný počet mezilehlých bodů. Cesta je vykreslena jednoduše postupným propojením bodů od toho startovního, přes případné mezilehlé body, až do bodu cílového. Jsou tedy podporovány pouze rovné čáry bez možnosti definovat křivky.

Startovní a cílový bod jsou definovány stejným způsobem jako souřadnice uzlu, tedy buď dvěma, nebo čtyřmi hodnotami. Dvě hodnoty určují absolutní pozici bodu a čtyři hodnoty určují pozici bodu relativně k pivotu nějakého uzlu. Cesty se obvykle používají k propojování uzlů, a tak bude pravděpodobně nejčastější použití druhé varianty s nulovým relativním posunem.

Pro mezilehlé body je již možné definovat jejich umístění tak, že na každé ose referují pivot jiného uzlu. Implementace by zde měla být jednodušší, protože fáze zpracování cest bude následovat až po kompletním zpracování všech uzlů. Dále mohou jednotlivé body referovat start, cíl, nebo předchozí bod své cesty. Nakonec může být umístění mezilehlého bodu určeno i pomocí absolutních souřadnic. Pro snazší zpracování na straně aplikačního kódu je každý mezilehlý bod definován šestiprvkovým polem, kdy první tři prvky odpovídají umístění na ose x a zbylé tři prvky umístění na ose y. Všechny tři prvky jsou potřeba definovat pouze při referenci pivotu, v ostatních případech jsou nevyužité prvky nastaveny na prázdný řetězec.

Obrázek 3.3 představuje další nástroje pro popis cest: Pro jednu cestu je možné definovat více koncových bodů. To znamená, že je pomocí jedné společné definice v podstatě vykresleno více cest. Díky tomu, že mezilehlé body se mohou odkazovat na koncový bod, lze v takovéto definici popsat rozvětvení. Dále je k dispozici parametr *shift*, který definuje „odskok“ startovního a cílových bodů od svých uzlů. Tento parametr tak dokáže nahradit krajní mezilehlé body.

Diagram	Popis cesty diagramu	Popis cesty s využitím shift
	<pre>[[path]] start = ["a", "bottom", 0, 0] ends = [["b", "left", 0, 0], ["c", "left", 0, 0],] points = [["", "start", 0, "", "start", 25], ["", "prev", 40, "", "prev", 0], ["", "end", -25, "", "end", 0]]</pre>	<pre>[[path]] start = ["a", "bottom", 0, 0] ends = [["b", "left", 0, 0], ["c", "left", 0, 0],] points = [["", "prev", 40, "", "prev", 0]] shift = 25</pre>

Obrázek 3.3: Schéma pro popis cest diagramu v jazyce TOML

Cesty mají také nastavitelný parametr *z*. Je zaručeno, že se cesta vykreslí nad uzly, které mají hodnotu *z* nižší nebo rovnou. Uzly s vyšší hodnotou tohoto parametru pak danou cestu překryjí. Nakonec lze u cest nastavit jejich barvu, textový popis a zdali mají končit či začínat šipkou.

Poslední typ tabulky, který navrhované schéma podporuje, jsou proměnné. Uživatel může přidat sekci začínající řádkem `[variables]` a v ní definovat klíče s přiřazenými číselnými hodnotami. Na určitých místech v kódu, kde je očekávána číselná hodnota, je pak možné místo čísla zadat název proměnné v podobě řetězce. Je tak možné např. nastavit jednotnou šířku pro všechny uzly v diagramu, kterou lze měnit editací jednoho čísla ve zdrojovém textu.

Velkým omezením je, že s proměnnými nelze provádět žádné aritmetické operace. Při popisu diagramu by se jistě hodilo mít možnost definovat, že např. uzel *a* je třikrát širší než uzel *b*. Zakomponovat takovéto funkce do schématu by sice bylo možné, ale jednalo by se o kompromis mezi všestranností a stručností zápisu. Navíc by se tím dále zkomplikoval aplikační kód, který by musel vše kontrolovat. Tento požadavek zkrátka přesahuje záměr a možnosti jazyka TOML.

4 Implementace ukázkové aplikace

Ukázkovou aplikaci této práce jsem implementoval ve třech knihovnách v pořadí *Dear ImGui*, *egui* a *Qt*. První dvě knihovny se v rešerši jeví jako vhodné pro realizaci ukázkové aplikace a zároveň se aktuálně jedná o dvě nejpopulárnější knihovny s přístupem IMGUI. Knihovna *Qt* je pak jednou z nejvíce používaných technologií pro tvorbu aplikací s grafickým uživatelským rozhraním, a jedná se tak o vhodného zástupce přístupu RMGUI.

Následující text odpovídá pořadí, v jakém byly ukázkové aplikace implementovány. První sekce popisuje realizaci v knihovně *Dear ImGui* a zároveň i řešení problémů, které jsou společné pro všechny tři implementace. Zbylé dvě sekce se již zabývají odlišnostmi a specifickými problémy v daných knihovnách.

4.1 Implementace v knihovně Dear ImGui

Před tím, než jsem začal implementovat ukázkovou aplikaci v knihovně *Dear ImGui*, bylo potřeba zprovoznit „minimální funkční projekt“, který obsahuje všechny potřebné závislosti a lze jej sestavit na operačních systémech Windows a Linux. Pro sestavení jsem se rozhodl použít program *CMake* v kombinaci s nástrojem *Ninja*. O překlad se pak stará *GCC* nebo *MSVC* podle dané platformy.

Jako alternativu jsem zvažoval systém *build2*. Jedná se o sadu nástrojů pro sestavování, která mimo jiné obsahuje vlastního správce balíčků a je dobře zdokumentovaná. Usoudil jsem ale, že vyšší komplexita tohoto systému by prodloužila dobu vývoje a z důvodu relativně malé komunity by se také hůře řešily případné problémy.

Poté, co jsem se rozhodl, jak bude projekt sestavován, zbývalo vyřešit otázku přidávání závislostí. Prvním adeptem byl správce balíčků *vcpkg*, který ale v tu dobu neměl k dispozici nejnovější verzi knihovny *Dear ImGui*. Jedná se o verzi 1.92, která přinesla určitá vylepšení ohledně vykreslování textu a kterou jsem proto v tomto projektu vyžadoval. Alternativně by šel použít správce balíčků *Conan*, nebo funkce `FetchContent` programu *CMake*, která dokáže knihovnu stáhnout z online repozitáře.

Nakonec jsem se ale rozhodl nepoužívat žádného správce balíčků, a místo toho jsou zdrojové kódy všech závislostí součástí projektu. Konkrétně se kód aplikace nachází v adresáři `App` a kód závislostí v adresáři `Dependency`. Kořenový adresář projektu pak obsahuje hlavní soubor `CMakeLists.txt`, který programu *CMake* říká, jak má být projekt sestaven. Některé závislosti mají svůj vlastní soubor tohoto typu,

který stačí do toho hlavního nainportovat. Pro závislosti, které program CMake přímo nepodporují, jsem tento soubor vytvořil.

V projektu s takto řešenými závislostmi je sice oproti správci balíčků složitější aktualizovat a přidávat nové knihovny, tento přístup má ale i své výhody. Pro sestavení projektu na novém zařízení stačí pouze spustit program CMake a není nutné zprovozňovat žádného správce balíčků. Projekt si veškerý potřebný kód „nese s sebou“, a není tedy závislý na žádné online platformě. Je také možné upravit zdrojový kód knihoven pro potřeby konkrétní aplikace, což není ovšem vždy považováno za příklad dobré praxe.

Pro zprovoznění knihovny Dear ImGui je potřeba do minimálního projektu přidat nějaký backend. Z důvodu předchozí zkušenosti jsem zvolil OpenGL s knihovnou *Glad* pro načtení funkcí a *GLFW* pro správu oken. Dále jsem do projektu přidal knihovnu *FreeType* pro vykreslování textu, kterou je možné v Dear ImGui použít namísto výchozí knihovny *stb-truetype*. FreeType je sice o dost větší knihovna, nabízí ale kvalitnější vykreslování písma. Knihovna *stb-truetype* měla navíc v době psaní poslední aktualizaci v roce 2021. Jako písmo pro celý projekt jsem zvolil *Inconsolata*. Jedná se o písmo, které má permissivní licenci a je neproporcionální (*monospace*).

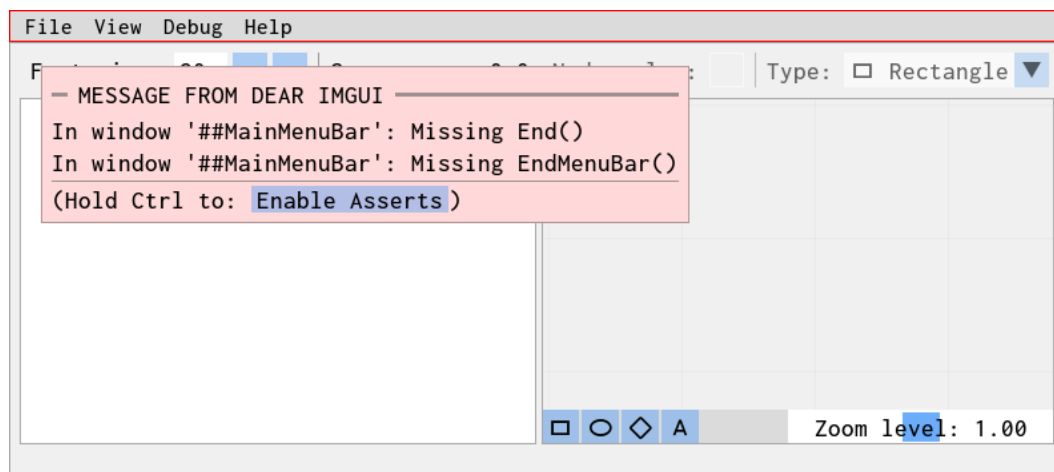
Výchozím písmem knihovny byl totiž v té době bitmapový font *ProggyClean*. Ten není ve větších velikostech příliš čitelný a při škálování dochází k jeho rozmazání. Od verze 1.92.6 (únor 2026) knihovna nabízí nový vektorový font *ProggyForever*, který tyto problémy řeší. Při jeho pozdějším testování jsem ale zjistil, že se v kombinaci s knihovnou FreeType nevykresluje korektně: znaky jsou rozmazané a vertikálně odsunuté. Toto se děje pouze v případě použití verze písma, která je vložena v kód knihovny. Je-li stejné písmo importováno jako externí soubor ve formátu TrueType, tak problém mizí. Tuto skutečnost jsem nahlásil jako [issue #9348](#) na platformě GitHub. Podle autora knihovny se jedná o důsledek více problémů a vyřešení bylo nastaveno jako jeden z milníků (*milestone* na platformě GitHub) pro budoucí verzi 1.93.

Po dokončení minimálního projektu bylo možné začít implementovat jednotlivé funkce aplikace, kde většinu z nich včetně hlavní smyčky pokrývá třída **App**, jejíž logika je rozdělena do menších celků v podobě metod. Je-li potřeba, aby více metod přistupovalo ke stejným datům, musí se jednat o členy jejich třídy. Pro data, která jsou potřeba pouze v jedné konkrétní metodě, lze použít klíčové slovo **static**. Toho se využívá v metodách, které se starají o určitou část GUI, a jsou tedy volány pravidelně s průchodem hlavní smyčky.

Kromě třídy **App** obsahuje projekt další dvě samostatné třídy **Parser** a **Exporter**. První třída se stará o převod textu, který popisuje diagram v jazyce TOML, na kolekci struktur. Podle informací obsažených v těchto strukturách je pak diagram vykreslen. O samotný překlad jazyka TOML se stará knihovna *toml++*, úkolem třídy je dle navrženého schématu výstup dále zpracovat a hlásit případné chyby. Druhá třída umožňuje exportovat diagram do formátu SVG. Obě třídy jsou napsány tak, aby byly nezávislé na knihovně Dear ImGui a šly znovu použít v projektu s knihovnou Qt.

4.1.1 Tvorba uživatelského rozhraní v knihovně Dear ImGui

Knihovna Dear ImGui se chová jako stavový automat a poskytuje tak řadu „párových metod“ pro tvorbu základního rámce uživatelského rozhraní. Příkladem jsou metody `BeginMainMenuBar` a `EndMainMenuBar`. Po zavolání první metody se všechna další volání knihovny vztahují ke hlavní liště nabídek. Jakmile je práce s touto lištou v aplikačním kódu dokončena, je třeba zavolat metodu druhou. Není-li „uzavírající“ metoda zavolána, pak knihovna tuto chybu hlásí za běhu uvnitř uživatelského rozhraní.



Obrázek 4.1: Dear ImGui: oznámení chyby za běhu

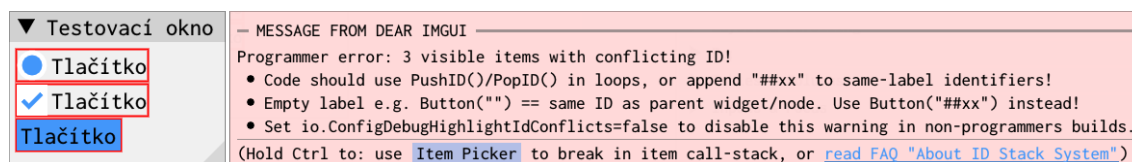
V Dear ImGui může mít každé okno svou vlastní lištu nabídek a ta hlavní je interně řešena tak, že je vytvořeno speciální okno obsahující pouze tuto lištu. Proto chybová hláška oznamuje neuzavření lišty (`EndMenuBar`) a neuzavření okna (`End`). Alternativně by šlo vytvořit pouze hlavní okno a do něj přidat lištu vlastní. Řešení s nezávislou hlavní lištou se ale objevuje v oficiálním příkladu použití knihovny, a proto jsem jej zvolil.

Lišta nabídek je v tomto projektu definována jako nezávislý prvek a až po jejím zpracování následuje definice hlavního okna. Z pohledu knihovny se jedná o klasické nemodální okno, jehož obsah je také definován mezi voláními párových metod. Před voláním první z nich jsou ale ještě použity metody `SetNextWindowPos` a `SetNextWindowSize`, pomocí nichž lze nastavit, kde se bude nacházet následující okno a jak bude velké. V tomto případě jsou metodám předány takové parametry, aby hlavní okno vyplňovalo celý prostor okna hostitelského. Tedy až na část, kde už se nachází lišta nabídek. Dále je párové metodě, která značí začátek okna, předáno několik příznaků, např. aby okno nemělo záhlaví, nebo aby nebylo možné jej přesouvat.

V kontextu okna už lze vykreslovat ovládací prvky. Knihovna interně pracuje s „kreslícím kurzorem“, který po vytvoření okna ukazuje do jeho levé horní části. Je-li zavolána metoda pro přidání ovládacího prvku, tak je tento prvek vykreslen na pozici kreslícího kurzoru. Po přidání daného prvku se kreslící kurzor posouvá na

novou pozici a je připraven na další prvek. Pokud je např. dvakrát zavolána metoda `Button`, budou se v okně pod sebou nacházet dvě vykreslená tlačítka.

Knihovna si uchovává o vykreslených ovládacích prvcích určité údaje a ke každému má přidělen identifikátor. Ten musí být unikátní a je vypočten jako hash *popisku* a *rodiče*. V případě okna s tlačítkem se jedná o textový popis tlačítka a název daného okna. To znamená, že když jedno okno obsahuje více tlačítek se stejným popiskem, dochází ke kolizi. Kolize je opět oznámena jako chyba za běhu a může se projevit nefunkčností některého z dotčených ovládacích prvků. Vypočtený hash navíc nebere v potaz typ objektu, a ke kolizi může proto dojít i při použití více druhů ovládacích prvků (obrázek 4.2). Tento problém se dá vyřešit dvojicí metod `PushID` a `PopID`, nebo speciálním formátem popisku. Obsahuje-li popisek dva po sobě jdoucí znaky mřížky (`##`), pak od této části již není jeho text vykreslen. Díky tomu lze definovat více prvků, které mají odlišné popisky, ale jejich vykreslená část je totožná.



Obrázek 4.2: Dear ImGui: konflikt identifikátorů pro ovládací prvky

S voláním více metod pro vykreslení ovládacího prvku se tedy dané prvky vykreslí pod sebe. Pro vykreslení prvků vedle sebe je možné mezi jejich volání přidat metodu `SameLine`, která v podstatě přesune kreslicí kurzor zpět na předchozí řádek a přidá k němu malý horizontální posun. Takto lze docílit základního rozložení ovládacích prvků: pokládat je pod sebe shora dolů a případně vedle sebe zleva doprava. Nemí-li okno dostatečně vysoké, aby dokázalo viditelně vykreslit všechny své prvky, automaticky se v něm zobrazí vertikální posuvná lišta. U horizontální posuvné lišty se toto chování musí zapnout pomocí příznaku, i bez něj je ale možné obsah okna horizontálně posouvat kombinací klávesy *Shift* a kolečka myši.

Pro tvorbu složitějšího rozložení je potřeba pracovat s kreslicím kurzorem. K dispozici jsou metody pro získání a nastavení jeho souřadnic. Díky tomu lze vykreslovat na libovolné místo v okně. Další užitečná metoda je `GetContentRegionAvail`, která vrací informaci o tom, kolik volného místa od aktuální pozice kreslicího kurzoru v okně ještě zbývá. Metoda vrací dva prvky, jeden pro osu *x* a druhý pro osu *y*, a často se používá v kombinaci s podokny.

Podokno (*child window*) je samostatná oblast, kterou lze umístit do normálního okna. Ve své výchozí podobě má pevnou velikost a při přidávání prvků se chová stejně jako normální okno. Jakmile se do něj tedy prvky „nevejdu“, objeví se v něm posuvná lišta. Podokna mají své speciální příznaky jako `ResizeX`, který umožňuje uživateli měnit jeho šířku.

V ukázkové aplikaci je rozložení textového editoru a kreslicího plátna řešeno právě pomocí podoken, kterým je stanovena pevná velikost. Výška obou podoken je rovna volnému místu od kreslicího kurzoru na ose *y* po odečtení výšky stavového řádku. Šířka podokna, které obsahuje textové pole, je při startu aplikace nastavena

na polovinu dostupného místa na ose x. Textové pole tak vždy zabírá polovinu okna i při změně jeho velikosti, protože je tento kód volán s každým průchodem smyčky. Poté je volána metoda `SameLine` a je vytvořeno podokno pro plátno, které si může vzít zbytek dostupné šířky. Nejprve jsem použil pro levé podokno příznak `ResizeX`, aby mohl uživatel měnit poměr šířek mezi textovým polem a plátnem, což je u tohoto typu aplikace běžná funkce. S tímto příznakem je ale při změně velikosti hostitelského okna zachována šířka levého podokna, nikoliv poměr šířek obou podoken. Proto bylo nutné přijít s vlastním řešením.

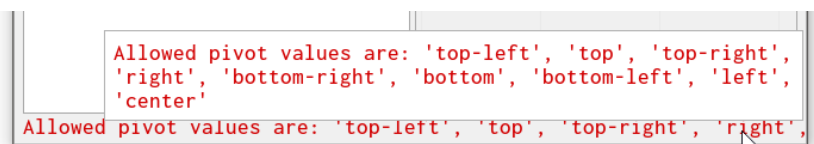
Mezi podokna je metodou `InvisibleButton` přidáno tzv. neviditelné tlačítko. Jedná se o ovládací prvek, se kterým lze interagovat jako s tlačítkem a který zabírá určitý prostor, ale není nijak vykreslen. Zde má tlačítko stejnou výšku jako obě podokna. Pokud je drženo, tak se podle pohybu myši mění hodnota proměnné reprezentující poměr šířek podoken. Zda-li je neviditelné tlačítko drženo, lze zjistit metodou `IsItemActive`. Knihovna se chová jako stavový automat, a proto tato metoda vrací hodnotu vždy pro poslední vykreslený prvek.

V hlavním okně nyní zbývá vykreslit pouze stavový řádek. Ten může zobrazovat text chyby, kterou nahlašuje třída pro zpracování textu popisujícího diagram. Není-li nahlášena žádná chyba, pak není třeba nic vykreslovat. Při absenci chybového textu zde zůstane prázdný prostor, protože výška podoken nad stavovým řádkem je nastavena tak, aby byla co nejvyšší, ale tento prostor nezabírala. Při nahlášené chybě je v tomto prostoru vypsána chybová hláška červeným písmem. Barevného textu lze dosáhnout použitím párových metod `PushStyleColor` a `PopStyleColor`.

Pokud je okno příliš zmenšené, tak se může stát, že se do něj celá chybová hláška nevejde. V takových případech je nutné dávat pozor na horizontální posouvání: Není-li oknu předán příznak `NoScrollWithMouse`, může uživatel pomocí klávesy *Shift* a kolečka myši celým oknem posouvat i přesto, že není zobrazena horizontální posuvná lišta (obrázek 4.3). Posouvání jsem proto zakázal a pro případy malého okna s dlouhou chybovou hláškou přidal kontextovou nápovědu (*tooltip*), která vždy zobrazí celý chybový text (obrázek 4.4). Jelikož je celá aplikace vykreslována do hostitelského okna, nemůže jej kontextová nápověda „opustit“.



Obrázek 4.3: Dear ImGui: problém horizontálního posouvání



Obrázek 4.4: Dear ImGui: kontextová nápověda

4.1.2 Víceřádkové textové pole v knihovně Dear ImGui

Jakmile je vytvořeno podokno, které má sloužit jako rodič víceřádkového textového pole, stačí mezi jeho párovými metodami zavolat metodu `InputTextMultiline`, která textové pole vykreslí. Vlastnosti tohoto ovládacího prvku byly představeny v rešerši, během implementace ukázkové aplikace se však objevily určité obtíže, které s tímto ovládacím prvkem souvisí.

Knihovna Dear ImGui v základu umožňuje ovládat uživatelské rozhraní pomocí klávesnice. Klávesou *Tab* tak lze přepínat, jaký z interaktivních prvků je aktuálně vybraný. Metodě pro vykreslení textového pole je proto předán příznak `AllowTabInput`, aby při psaní textu nebylo možné klávesou *Tab* ovládací prvek opustit. Místo toho je do textu vložen znak tabulátoru a toto chování není možné přenastavit např. na vložení čtyř mezer.

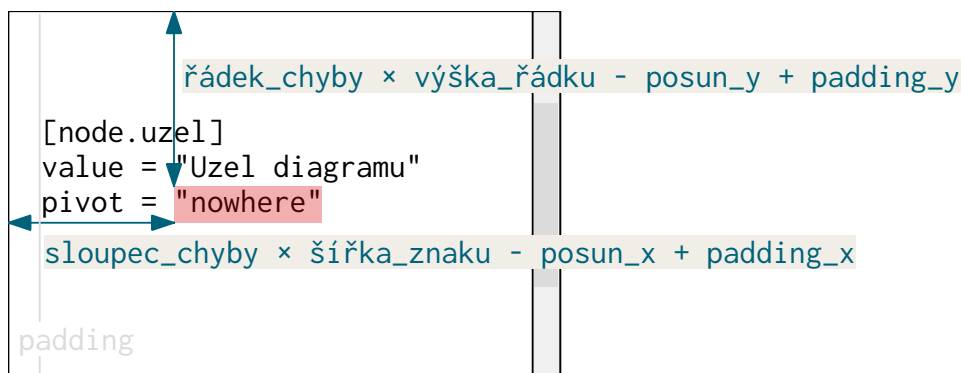
Další klávesou pro navigaci je *Alt*, která slouží pro skok do záhlaví okna a lišty nabídek. Hlavní okno má záhlaví skryto a lišta je umístěna v samostatném okně, i přesto ale stisk klávesy *Alt* ruší interakci s textovým polem. Protože se v této práci dále nevěnuji navigaci pomocí klávesnice a klávesovým zkratkám, vypnul jsem tyto funkce v aplikaci globálně.

Jak již bylo zmíněno, ve víceřádkovém textovém poli nelze zobrazit horizontální posuvnou lištu. Posouvat pohled je možné pouze s využitím kláves pro změnu pozice textového kurzoru. Při pokusu o posunutí pomocí klávesy *Shift* a kolečka myši jsem objevil drobnou vizuální chybu. Tímto způsobem sice bylo možné text horizontálně posouvat, ale jen do chvíle, kdy počáteční písmena „narazila“ na okraj ovládacího prvku. Kolečkem myši bylo tedy možné v podstatě jen vynulovat vnitřní okraj (*padding*) textového pole. Tuto chybu jsem nahlásil jako [issue #9249](#) na platformě GitHub a ve verzi 1.92.6 byla opravena autorem knihovny.

Dále jsem do aplikace přidal funkci zvýraznění nahlášené chyby v textovém poli. Samotný ovládací prvek žádné zvýraznění textu nepodporuje, a tak je tato funkce realizována pomocí vykreslování grafických primitiv. Nad částí textu, která obsahuje chybu, je vykreslen poloprůhledný červený obdélník. Knihovna `toml++` poskytuje metodu `source`, která pro jednotlivé zpracované tabulky, pole a hodnoty vrací informace o tom, kde se nachází jejich definice ve vstupním textu. Výstupem metody je struktura, ze které lze pro začátek a konec dané oblasti získat číslo řádku a číslo znaku v tomto řádku („číslo sloupce“).

Tato data jsou převedena na pozici a velikost červeného obdélníka. Výpočet pozice od levého horního rohu podokna ukazuje obrázek 4.5. Řešení počítá s neproporcionálním písmem, jinak by byla potřeba dodatečná logika pro měření šířky určitých úseků textu. Pro zjištění vertikálního posunu (`posun_y` v obrázku) je použita metoda

`GetScrollY`, která tuto hodnotu vrací pro poslední vykreslený prvek. Zjištění horizontálního posunu (`posun_x` v obrázku) je složitější a využívá definic ze souboru `imgui_internal.h`. Není proto zaručeno, že toto řešení bude fungovat i v budoucích verzích knihovny. Pro účely této práce je to však dostačující, a jelikož jsou zdrojové kódy knihovny součástí projektu, tak nehrozí použití nekompatibilní verze vývojářem.



Obrázek 4.5: Dear ImGui: výpočet pozice zvýraznění chyby

Jednou z žádoucích funkcí aplikace je zvýrazňování syntaxe, kterou ale výchozí textové pole nepodporuje. Proto jsem do projektu přidal doplněk třetí strany *ImGuiColorTextEdit* [19]. Tento doplněk není v aktivním vývoji a v době psaní už se pravděpodobně nejedná o nejlepší způsob, jak do projektu v knihovně Dear ImGui přidat zvýrazňování syntaxe. Když jsem ale implementoval ukázkovou aplikaci, jednalo se o jediný doplněk tohoto typu, který byl vystaven v sekci doplňků na *Dear ImGui wiki*. Od té doby zde přibyl odkaz na fork¹ tohoto projektu, který je v aktivním vývoji. Problémy, které jsou později zmíněny, proto už mohou být v této verzi vyřešeny. Při výběru doplňku jsem kontroloval i jeho forky, ale bylo jich několik aktivních a usoudil jsem, že programátor se nejčastěji setká právě s originálním repozitářem. Nově doporučovaná verze doplňku je navíc forkem jiného forku, takže není ve výchozím zobrazení na platformě GitHub vidět.

Tento doplněk se nechová jako standardní ovládací prvek v přístupu IMGUI a jeho rozhraní spíše připomíná přístup RMGUI. Není zde tedy metoda pro vykreslení ovládacího prvku, která by dostala jako parametr referenci na řetězec se zdrojovým textem a případné příznaky. Místo toho pracuje aplikace s instancí třídy `TextEditor`, u které se s každým průchodem smyčky volá metoda `Render`. Ovládací prvek si udržuje stav textového obsahu u sebe a pro práci s tímto textem v aplikačním kódu je nutné využít metod `GetText` a `SetText`.

Doplněk jsem mírně upravil pro potřeby ukázkové aplikace. Jelikož je jeho zdrojový kód součástí projektu a pravděpodobně pro něj již nevyjdou žádné aktualizace, upravil jsem kód přímo bez využití záplat (*patch files*). Jednou ze změn je např. chování při stisku klávesy `Tab`, kdy jsou do textu namísto znaku tabulátoru vloženy

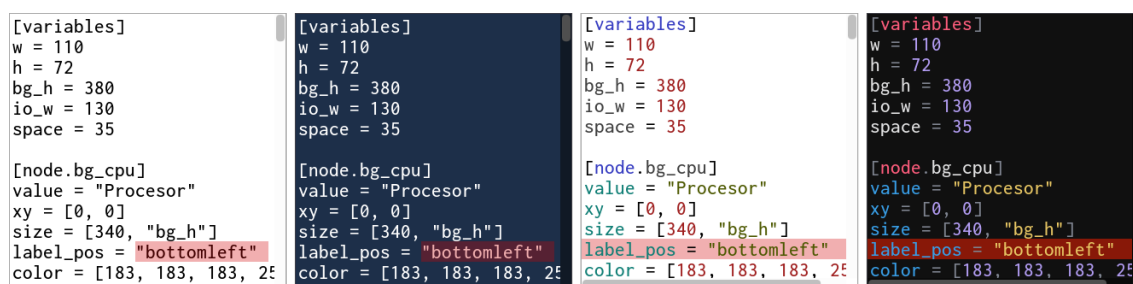
¹„Fork“ je na platformě GitHub nezávislá kopie nějakého repozitáře. Oproti větvím (*branches*) je možné s touto kopií volněji experimentovat bez jakéhokoliv zásahu do původního projektu [20]. Pro tento termín neexistuje zavedený český překlad.

mezery. Pro zvýrazňování syntaxe jsem přidal vlastní definici jazyka, aby byla zvýrazňována pouze klíčová slova z navrhovaného schématu pro jazyk TOML. K tomu jsem definoval vlastní barevná schémata pro světlý a tmavý režim aplikace, která mají oproti těm výchozím lepší poměr kontrastu mezi popředím a pozadím.

Hlavní motivací pro použití tohoto doplňku, která je zároveň i prvním viditelným rozdílem oproti výchozímu textovému poli, je schopnost zvýrazňovat syntax. Při testování doplňku jsem ale narazil na další rozdíly a problémy tohoto ovládacího prvku:

- Má funkci pro zvýraznění chyb v kódu. Zvýrazněn je vždy celý řádek a není možné zvýraznit pouze jeho část.
- Nepodporuje mazání celých slov pomocí klávesových zkratk *Ctrl + Backspace* a *Ctrl + Delete*.
- Nepodporuje zalamování dlouhých řádků.
- Držením levého tlačítka myši lze vybírat text. Aby se ale s pohybem myši aktualizovala vybraná oblast, musí se kurzor nacházet uvnitř ovládacího prvku.
- Dokáže zobrazit horizontální posuvnou lištu, i tak je ale horizontální posouvání problémové. Pohled na text je totiž posouván automaticky a zdá se být nekonzistentní. V určitých případech je pohled posouván tak, že textový kurzor není vidět, protože je posun příliš malý. Uživatel sice může využít posuvné lišty, aby textový kurzor do pohledu dostal, jakmile ale začne psát, pohled se opět přesune na automaticky určené místo. Uživatel tedy nemusí vidět text, který aktuálně píše.

Z důvodu posledního bodu, který značně zhoršuje uživatelský zážitek, jsem přidal do nastavení aplikace možnost přepínat mezi výchozím ovládacím prvkem a prvkem třetí strany. Obrázek 4.6 ukazuje, jak vypadají obě varianty ve světlém a tmavém režimu.



Obrázek 4.6: Dear ImGui: víceřádkové textové pole

4.1.3 Zpracování vstupního textu v jazyce TOML

Vedle víceřádkového textového pole je nejdůležitějším prvkem ukázkové aplikace kreslicí plátno, ve kterém je zobrazen diagram odpovídající textovému popisu. V něm

se využívá metod pro vykreslování grafických primitiv, které poskytuje daná knihovna. Nejprve je ale nutné textový vstup na volání těchto metod převést. Převod probíhá ve dvou krocích, kdy se o první krok stará třída `Parser`, která je nezávislá na použité GUI knihovně. Druhý krok je již součástí kódu kreslicího plátna a využívá metody z knihovny Dear ImGui. I v druhém kroku probíhá určité zpracování dat, a nejedná se tedy o prosté vykreslování podle kompletně připravených hodnot.

Výchozí podobou uzlu diagramu je obdélník, ve kterém je umístěn text. Nemá-li uzel nastavenou explicitní velikost, pak je určena implicitně tak, aby byl celý text uzlu obsažen v daném obdélníku s přidáním vnitřním okrajem. V diagramu může nastat situace, kdy má takový uzel nějaké potomky a jeho velikost ovlivňuje umístění těchto potomků. Pro určení přesných pozic a velikostí všech uzlů je tedy nutné znát přesné velikosti vykreslených textů. Knihovna Dear ImGui poskytuje metodu `CalcTextSizeA`, která pro dané písmo, velikost a textový obsah vrátí informace o tom, jak by byl takto vykreslený text velký.

Třída `Parser` ale musí být nezávislá na knihovně Dear ImGui a nelze v ní tedy přímo volat metodu pro zjištění velikosti vykresleného textu. Toto by šlo řešit různě, ale já se rozhodl právě pro postup s dvěma kroky. V prvním kroku třída `Parser` s využitím knihovny `toml++` převede vstupní text na struktury pro krok druhý. Třída „udělá vše, co dokáže“ bez nutnosti použít knihovnu Dear ImGui a jakmile její krok skončí, dále už se v tomto průchodu hlavní smyčkou nepoužije. Proto je v druhém kroku potřeba provádět určité dodatečné výpočty.

Úkolem prvního kroku je tedy zpracovat textový vstup do vlastních struktur. Nejprve se použije knihovna `toml++` pro překlad dokumentu. Zahlásí-li knihovna chybu, tak je chybová hláška přeposlána do stavového řádku v uživatelském rozhraní. Na plátně v tomto případě zůstane vykreslen diagram, který z pohledu jazyka TOML odpovídá poslednímu validnímu vstupu. Při psaní kódu uživatelem totiž pravidelně dochází k nevalidním vstupům, např. když se definuje nová tabulka a zatím nebyla uzavřena hranatá závorka. Pokud by nebyl uchovávan poslední validní diagram, tak by při psaní vstupního textu kreslicí plátno často „problikávalo“, protože z nevalidního vstupu nelze získat žádná data.

V případě validního vstupu vrací knihovna `toml++` strom dokumentu, v němž jsou postupně zpracovány uživatelské proměnné, uzly a cesty. Tomu odpovídají očekávané tabulky `[variables]`, `[node.x]` a `[[path]]`. I v této fázi zpracování vstupu jsou hlášeny chyby. Zde se již jedná o aplikačně specifické chyby, které hlásí porušení navrženého schématu v jazyce TOML. Z důvodu ulehčení logiky na straně aplikačního kódu nejsou hlášeny případy, kdy uživatel definuje neočekávané tabulky. Jakmile se ale v tabulce očekávaného typu objeví neznámý parametr, nebo známý parametr s neočekávanou hodnotou, je chyba zahlášena. V tomto případě probíhá zpracování dokumentu dál a diagram je vykreslen tak, jakoby byl chybný řádek ignorován.

Hodnoty uživatelských proměnných jsou uloženy v mapě `m_variables`. Jako *mapa* se zde myslí kolekce typu `std::unordered_map`. Z důvodu přehlednosti je logika zpracování dokumentu rozdělena do více implementačních souborů, a proto je mapa členem třídy `Parser`. Pokud je v dokumentu nalezena tabulka `[variables]`, tak jsou postupně všechny její páry typu *klíč-hodnota* přidány do mapy. Návrh umožňuje pouze proměnné s číselnou hodnotou, a tak jsou řádky s jiným datovým typem

ignorovány. U prvního výskytu takového řádku je nahlášena chyba. Třída **Parser** totiž hlásí pouze první nalezenou chybu.

Zpracování uzlů už je složitější, protože schéma podporuje relativní souřadnice a uzly mohou být definovány v libovolném pořadí. Definice potomka („závislého“ uzlu) se tedy ve vstupním dokumentu může objevit dříve než definice jeho rodiče. Jak již bylo zmíněno, v prvním kroku zpracování dokumentu nelze vypočítat finální souřadnice závislých uzlů.

Aby bylo možné vypočítat souřadnice závislého uzlu v kroku druhém, musí být nejprve kompletně zpracován jeho rodič. Úkolem prvního kroku je proto všechny uzly seřadit, aby je druhý krok mohl bez problému zpracovat. Dalším úkolem prvního kroku je kontrolovat, zdali všechny závislé uzly referují nějakého existujícího rodiče a zdali neexistuje posloupnost referencí, která tvoří kruh. Je třeba zdůraznit, že závislý uzel může být zároveň rodičem i několika jiných uzlů, a tak na hierarchii těchto vztahů lze pohlížet jako na stromy.

Pro každý uzel je během jeho zpracování vytvořena struktura **Node** a její parametry jsou nastavovány podle hodnot očekávaných klíčů v příslušné tabulce. Určité klíče mohou mít jako hodnotu nastaven řetězec s názvem proměnné. V tom případě je parametru přiřazena hodnota z již připravené mapy **m_variables**. Případná neexistující proměnná je hlášena jako chyba. Zpracované uzly jsou postupně vkládány do mapy **m_result_nodes**, kde klíčem je jejich identifikátor.

Každému uzlu je přidělena celočíselná proměnná **draw_batch_number**, která značí pořadové číslo dávky pro vykreslení. Čím vyšší je číslo dávky, o to později musí být daný uzel v druhém kroku zpracován. Mají-li dva uzly toto číslo stejné, tak na pořadí jejich zpracování nezáleží. Až na jednu optimalizaci, kterou zmíním později, je u všech uzlů nejprve nastaveno číslo dávky na nulu.

Dále jsou během zpracování plněny dvě pomocné kolekce. Mapa **refs** obsahuje dvojice identifikátorů pro všechny existující vztahy typu rodič–potomek. Množina **stable_nodes** pak obsahuje identifikátory všech „stabilních“ uzlů. Stabilním uzlem se myslí takový uzel, jehož číslo dávky je finální. Nezávislé uzly, jejichž souřadnice jsou určeny absolutně, jsou už během svého zpracování označeny za stabilní a jejich číslo dávky tedy zůstává nulové. Algoritmus pro určení čísel dávky u zbylých uzlů je proveden až po úvodním zpracování, kdy jsou naplněny pomocné kolekce.

Algoritmus iteruje přes páry identifikátorů rodič–potomek v mapě **refs** a kontroluje při tom účast v množině **stable_nodes**. Objeví-li se pár, kdy je rodič stabilní a potomek nestabilní, pak je číslo dávky potomka nastaveno o jedna vyšší, než je číslo rodiče. Tento potomek je nyní označen jako stabilní přidáním do množiny **stable_nodes**. Mapa je takto opakovaně procházena do té doby, než nastane iterace, kdy nebyl žádný nový uzel označen jako stabilní. V tomto cyklu zároveň probíhá kontrola, zdali neexistuje uzel s referencí na neplatný identifikátor. Pokud po ukončení cyklu existují ještě nějaké nestabilní uzly, znamená to, že reference mezi nimi tvoří kruh.

Výše bylo naznačeno, že pro tento postup existuje ještě jedna optimalizace, která probíhá během zpracování uzlů. Pokud je při zpracování závislého uzlu zjištěno, že jeho rodič je označen jako stabilní, pak je možné nastavit číslo dávky závislého uzlu okamžitě. Díky tomu je tento uzel také označen jako stabilní a daný pár nemusí být

přidán do mapy `refs`. V případě, že uživatel poskytne takový vstupní text, kde se definice každého potomka nachází vždy až za definicí jeho rodiče, pak pokryje tato optimalizace všechny závislosti a mapa `refs` bude po celou dobu prázdná.

Uzly jsou do této chvíle uloženy v mapě `m_result_nodes`, aby bylo možné k nim snadno přistupovat a měnit jejich číslo dávky. Jakmile jsou všechna čísla určena, je potřeba podle nich objekty seřadit. Nejdříve jsem struktury uzlů přesouval pomocí `std::move` do prioritní fronty, ze které si je hlavní třída `App` v druhém kroku postupně vybírala metodou `pop`. Pokud ale byla ve vstupním textu udělána chyba, nebylo pak možné vykreslit poslední validní diagram, protože prioritní fronta byla prázdná. Proto do fronty vkládám pouze dvojici s číslem dávky a identifikátorem. V druhém kroku jsou identifikátory z fronty postupně vybírány a použity jako klíč do původní mapy `m_result_nodes`. V případě chyby není obsah této mapy vymazán, a prioritní fronta tak může být znovu vytvořena.

Po zpracování všech uzlů jsou nakonec zpracovány cesty. Pozice jednotlivých bodů každé cesty mohou být také definovány relativně. Referovat lze ale pouze uzly, které jsou v tuto chvíli již vyřešené, nebo jiné body na své vlastní cestě. Není tedy potřeba řešit žádné pořadí zpracování a cesty mohou být ukládány rovnou do kolekce typu `std::vector`.

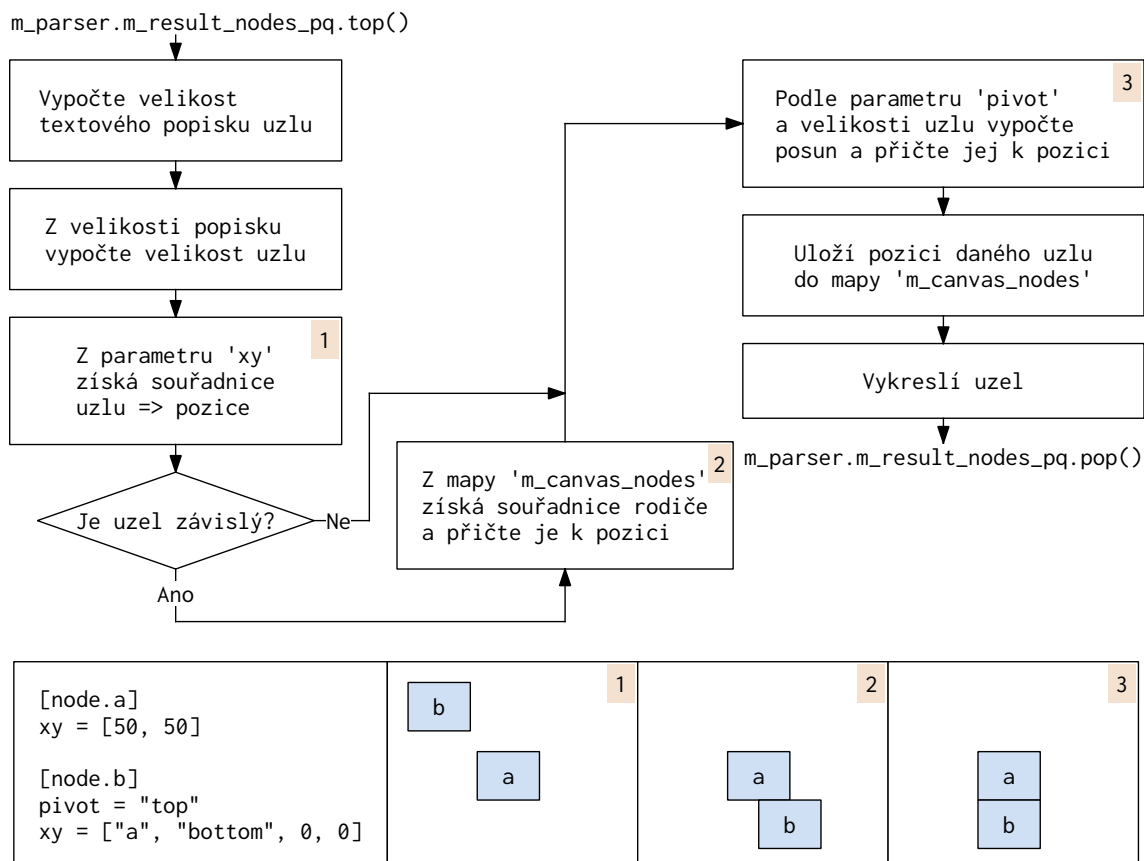
Třída `Parser` je psána jako stavový automat. Aplikace obsahuje instanci této třídy a jako součást průchodu hlavní smyčkou volá její metodu `Parse`, kde vstupním parametrem je reference na řetězec s popisem diagramu. Kolekce se zpracovanými uzly a cestami jsou veřejnými členy této třídy, a tak k nim aplikace může přistupovat. Dalším veřejným členem je případná chybová hláška. Dojde-li k chybě již při překladu jazyka TOML, není obsah kolekcí vymazán, a lze tak použít data z předchozích iterací. Díky tomu, že metoda pro zpracování textového vstupu nic nevrací a třída si místo toho udržuje výsledky u sebe, není třeba řešit žádné kopírování, přesouvání, nebo vlastnictví a životnost většího objemu dat. Takto si celé řešení vystačí s referencemi.

4.1.4 Vykreslení diagramu v knihovně Dear ImGui

Poté, co má třída `App` k dispozici informace o objektech v aktuálním diagramu, může jej začít vykreslovat. Obrázek 4.7 naznačuje, jak jsou dopočítány přesné souřadnice těch uzlů, jejichž velikost je určena implicitně podle velikosti textového popisku. V této části kódu je již možné použít metody z knihovny Dear ImGui, a tak je metodou `CalcTextSizeA` spočtena velikost textového popisku, ze které je pak odvozena velikost celého uzlu. Nemá-li uzel nastavený parametr *pivot*, pak je na zadaných souřadnicích vykreslen jeho levý horní roh. Pokud je hodnota *pivotu* nastavena např. na `"center"`, pak je uzel před vykreslením posunut o polovinu vypočtené velikosti, aby se na zadaných souřadnicích nacházel jeho střed.

U závislých uzlů ještě dochází k posunu podle přesných souřadnic rodiče, které jsou v danou chvíli již známy. Jednotlivé uzly jsou totiž zpracovány v pořadí určeném třídou `Parser`. Po zpracování každého uzlu jsou pro tento účel vypočítané souřadnice uloženy do mapy `m_canvas_nodes`. Konkrétně jsou ukládány pozice levého horního rohu, pravého dolního rohu a středu. Z těchto tří bodů lze snadno dopočítat všechny

ostatní pozice, které by mohl případný potomek referovat. Spodní část obrázku ukazuje, jak je v jednoduchém diagramu postupně určena pozice závislého uzlu. Po stanovení přesné pozice je již možné tento uzel vykreslit.



Obrázek 4.7: Princip určení přesných pozic uzlů v diagramu

Uzly diagramu jsou vykresleny pomocí metod struktury `ImDrawList`. Např. k vykreslení uzlu ve tvaru obdélníka je třeba volat: `AddRectFilled` pro výplň, `AddRect` pro okraj a nakonec `AddText` pro textový popis. Tyto metody jsou pro každý uzel volány hned po určení přesných souřadnic, a tak pořadí vykreslování odpovídá pozici v prioritní frontě třídy `Parser`.

Schéma pro popis diagramu ale povoluje parametr `z`, který slouží pro přepínání „hloubky“ jednotlivých uzlů. Toho je docíleno pomocí tzv. kanálů (*channels*), které jsou součástí struktury `ImDrawList`. Během vykreslování je možné přepínat číslo aktuálního kanálu. Nehledě na pořadí volání jsou objekty s nižším číslem kanálu vykresleny pod těmi s číslem vyšším.

Následně jsou vykresleny cesty. Zde je zpracování mírně složitější, protože mezilehlé body mohou mít pro každou osu jiného rodiče. Rodičem navíc může být i jiný bod na této cestě. Zpracování může dále zkomplikovat textový popis, cesta s více koncovými body, nebo parametr *shift*. Každopádně jsou pro každou cestu postupně zpracovávány její body a metoda `AddPolyline`, která slouží k vykreslení lomené čáry, pak přijímá kolekci těchto bodů.

Diagram je vykreslován na kreslicí plátno, což v případě knihovny Dear ImGui není žádný speciální ovládací prvek. V podstatě se kreslí na prázdné místo v hlavním okně, které zde zbylo po přidání víceřádkového textového pole. V této fázi průchodu hlavní smyčkou je dočasně nastaveno, aby vykreslování pomocí struktury `ImDrawList` probíhalo pouze v této prázdné oblasti. Příliš velký diagram tedy bude „useknut“, aby se nevykresloval přes ostatní části aplikace.

Uživatel může v kreslicím plátně posouvat pohled držením pravého tlačítka myši a měnit úroveň přiblížení pomocí kolečka myši. Toho je docíleno ovládacím prvkem neviditelného tlačítka, které zabírá celou oblast plátna. Tlačítko slouží pouze pro zachycení interakcí. Samotné posunutí a změna velikosti diagramu jsou realizovány dodatečnými výpočty během určování přesných pozic uzlů a cest. Výše popsaný princip tyto transformace pro zjednodušení nezmiňoval.

Při posouvání pravým tlačítkem myši se podle pohybu kurzoru mění hodnota proměnné, která značí aktuální posun pohledu. Tato hodnota je přičítána k souřadnicím vykreslovaných uzlů až po jejich uložení do mapy `m_canvas_nodes`. Jedná se tedy o poslední operaci před samotným vykreslením. Kolečkem myši se pak mění hodnota proměnné, která určuje úroveň škálování. Touto hodnotou je hned na začátku zpracování vynásobena pozice a velikost uzlu, čímž dochází ke zvětšení či zmenšení celého vykresleného diagramu.

Hodnoty v mapě `m_canvas_nodes` tedy berou v potaz úroveň přiblížení, ale ne posun pohledu. Posun v mapě neuchovávám, protože to za mě vede na čistší kód. Uzly a jednotlivé body cest mohou mít své souřadnice určeny různými způsoby s tím, že v některých případech by bylo nutné k nim hodnotu posunu přičíst, a v některých ne. Např. u závislého uzlu by přičtení vedlo k dvojitému posunu, protože souřadnice rodiče by již posunuty byly. Ve zvoleném přístupu stačí hodnotu posunu přičíst vždy jako poslední krok před vykreslením.

Mapa `m_canvas_nodes` slouží kromě určování přesných pozic také pro kontrolu, zdali se na souřadnicích nějakého uzlu právě nenachází kurzor myši. Konkrétně je zkoumáno, zdali se kurzor na obou osách nenachází mezi levým horním a pravým dolním bodem nějakého uzlu. Proto je výhodné uchovávat souřadnice uzlů škálované. Této kontroly se využívá v některých dalších funkcích ukázkové aplikace.

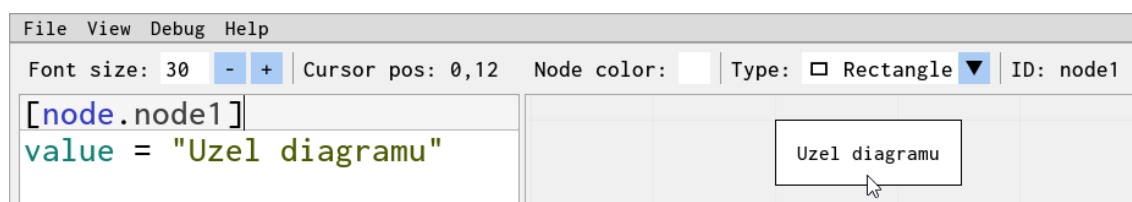
4.1.5 Další funkce aplikace v knihovně Dear ImGui

Ukázková aplikace dále obsahuje panel nástrojů, jehož funkce využívají různých ovládacích prvků a měly by dále prohloubit poznání knihovny Dear ImGui. V části nad víceřádkovým textovým polem se v panelu nástrojů nachází textový popisek, který informuje o aktuální pozici textového kurzoru v tomto poli. Konkrétně je vypsáno číslo řádku a číslo sloupce, což je informace často zobrazovaná textovými editory. Tato funkce je ovšem dostupná pouze při použití textového pole z doplňku třetí strany. Výchozí ovládací prvek tyto informace veřejně neposkytuje.

Dále se v panelu nástrojů nachází celočíselné vstupní pole, kterým lze měnit velikost písma v textovém poli. Knihovna pro tento účel poskytuje metodu `InputInt`, která vykreslí vstupní pole doprovázené dvěma tlačítky. Hodnotu dodané celočíselné proměnné lze měnit pomocí těchto tlačítek, nebo přímou editací textu ve vstupním

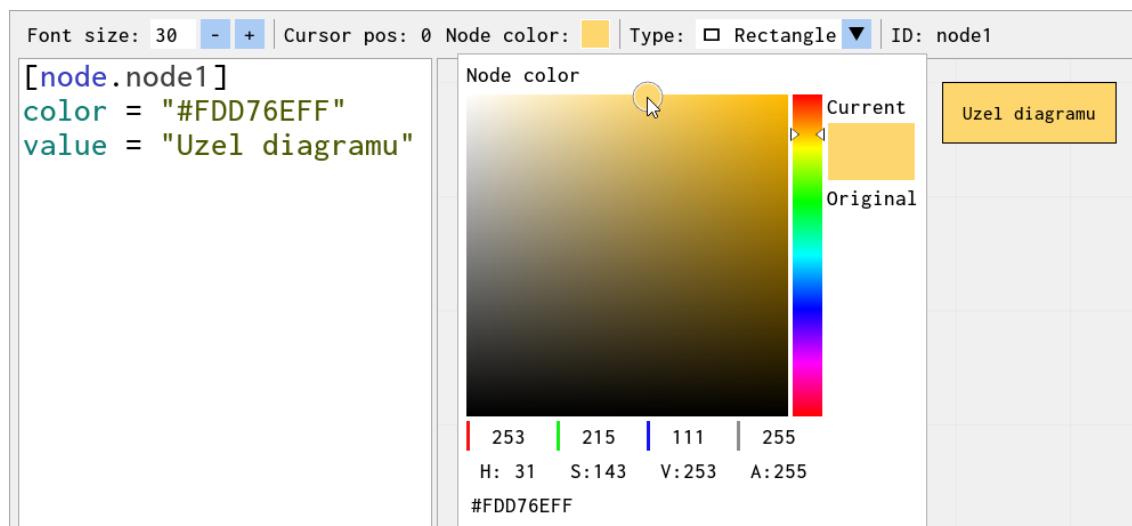
poli. Omezení rozsahu musí být řešeno vlastní logikou, viz kód B.3. Hodnota proměnné je pak použita při volání párových metod `PushFont` a `PopFont`, které kromě dočasné změny písma umožňují také měnit jeho velikost. Mezi voláními těchto metod se nachází veškerá obsluha víceřádkového textového vole, a proto velikost jeho písma odpovídá nastavené hodnotě.

V části nad kreslicím plátnem se v panelu nástrojů nachází ovládací prvky, které umožňují editaci uzlů diagramu bez nutnosti zásahu do vstupního dokumentu. Uživatel musí nejprve kliknout na nějaký uzel, aby jej označil jako „vybraný“. Takový uzel je pak možné určitým způsobem editovat. Konkrétně se v této části panelu nástrojů nachází textový popis, který zobrazuje identifikátor právě vybraného uzlu. Dále je zde výběr barev a rozbalovací seznam (*drop-down list*).



Obrázek 4.8: Dear ImGui: panel nástrojů, výběr uzlu

Ovládací prvek *výběr barev* slouží ke změně barvy vybraného uzlu a je vykreslen metodou `ColorEdit4`, kde číslo 4 značí, že lze nastavit i hodnotu alfa kanálu (průhlednost). Obrázek 4.9 ukazuje, jak může uživatel změnit barvu vybraného uzlu. Vykreslený diagram musí vždy odpovídat vstupnímu dokumentu. Při změně barvy se tedy ve skutečnosti mění pouze vstupní dokument a až v dalším průchodu hlavní smyčkou se tato změna projeví na vykresleném diagramu. Pokud daný uzel ve vstupním dokumentu již obsahuje parametr *color*, pak je jeho hodnota změněna podle uživatelem zvolené barvy. V opačném případě je tento parametr do vstupního dokumentu přidán.



Obrázek 4.9: Dear ImGui: změna barvy uzlu

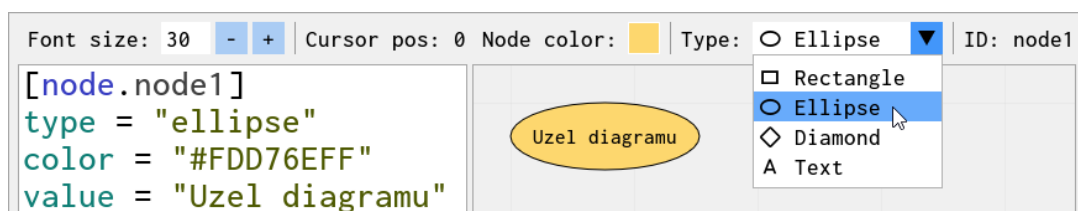
Očekávaná hodnota parametru *color* je řetězec ve tvaru **#RRGGBBAA**, což je stejný formát, který se nachází v dolní oblasti podokna výběru barev. Knihovna ovšem neposkytuje veřejný způsob, jak získat hodnotu tohoto zobrazeného řetězce. K dispozici je pouze čtyřprvková struktura, která je proto na řetězec v žádaném formátu převedena vlastní logikou.

Aby bylo možné upravit hodnotu parametru *color*, je potřeba znát umístění této hodnoty ve vstupním dokumentu. Umístění určitých hodnot je proto během zpracování uzlů ukládáno. K tomu je opět využita knihovna `toml++`, která tyto informace vrací v podobě čísla řádku a čísla sloupce. Funkce `insert` a `replace` typu `std::string`, které slouží pro vkládání a nahrazení částí textu, ale vyžadují na vstupu „absolutní“ pozici znaku. Proto je nutné provést převod, který funguje na principu hledání znaků nového řádku.

Na přidání funkce změny barvy bez zásahu do vstupního dokumentu lze pohlížet jako na kombinování přístupů WYSIWYG a WYSIWYM. Díky tomu je možné v ukázkové aplikaci prozkoumat kromě víceřádkového textového pole i další ovládací prvky. V existujících aplikacích ale taková kombinace přístupů není běžná a z pohledu aplikačního kódu komplikuje synchronizaci mezi jednotlivými prvky.

Problém, který se objevil v této aplikaci, je s vymazáním historie změn. Jakmile uživatel změni barvu uzlu pomocí ovládacího prvku, nelze již v textovém poli vrátit změny klávesovou zkratkou `Ctrl + Z`. Pro zprovoznění této funkce by bylo nutné buď upravit ovládací prvek textového pole tak, aby bral v potaz externí změny, nebo přijít s kompletně vlastním řešením historie změn.

Další parametr, jehož hodnotu lze měnit mimo textový editor, je tvar uzlu. K tomu slouží přidání rozbalovací seznam (obrázek 4.10). Knihovna Dear ImGui pro jeho vykreslení poskytuje párové metody `BeginCombo` a `EndCombo`. Mezi nimi lze volat jakékoliv jiné metody ovládacích prvků, které se pak vykreslí v rozbaleném seznamu. Náhled ovládacího prvku se nastavuje samostatně, a nemusí tak odpovídat právě vybrané položce. Jedná se tedy o relativně všestrannější a složitější přístup, a proto jsem pro účely aplikace implementoval pomocnou funkci. Ta za pomoci šablony (*template*) přijímá pole `std::array` o libovolném počtu řetězců a vykresluje odpovídající „tradiční“ rozbalovací seznam.



Obrázek 4.10: Dear ImGui: změna tvaru uzlu

Jednotlivé položky seznamu mají svou vlastní ikonu. Pro jejich vykreslení jsem využil písmo s ikonami od komunity *Pictogrammers*. Tento font je při startu aplikace sloučen s fontem Inconsolata, takže pro účel kreslení ikon není třeba nic přepínat. Vybrané písmo definuje své ikony v rozsahu, který je podle standardu Unicode [21] nazýván jako *Private Use Area*. Znaky v této oblasti nemají standardem specifikovaný

význam a jsou vyhrazeny pro „soukromé použití“. K usnadnění zápisu používám hlavičkový soubor z repozitáře *IconFontCppHeaders*, který pro jednotlivé znaky definuje makra.

Kód 4.1 ukazuje vykreslení rozbalovacího seznamu, který slouží pro změnu tvaru uzlu. Je zde použita vlastní pomocná funkce, která přijímá pole položek. Jednotlivé položky pak využívají maker pro přidání ikon. Ve výchozím stavu je ovládací prvek příliš široký, a tak je pro něj nastavena šířka explicitně. Knihovna sice poskytuje příznak, který nastaví šířku ovládacího prvku automaticky podle právě vybrané položky, pak se ale s výběrem jiné položky může šířka změnit. To by šlo řešit doplněním kratších řetězců mezerami, zvolené řešení se mi ale zdálo čistší.

```
constexpr std::array node_types = {
    ICON_RECTANGLE_OUTLINE" Rectangle",
    ICON_ELLIPSE_OUTLINE" Ellipse",
    ICON_RHOMBUS_OUTLINE" Diamond",
    ICON_FORMAT_TEXT_VARIANT" Text"
};
static int idx; // Index právě vybrané položky v rozbalovacím seznamu
ImGui::PushItemWidth(160); // Nastavení šířky ovládacího prvku
if (ImGuiCombo("##ComboNodeType", node_types, idx))
    { /* Uživatel vybral jinou položku ... */ }
ImGui::PopItemWidth();
```

Zdrojový kód 4.1: Dear ImGui: rozbalovací seznam s tvarem uzlu

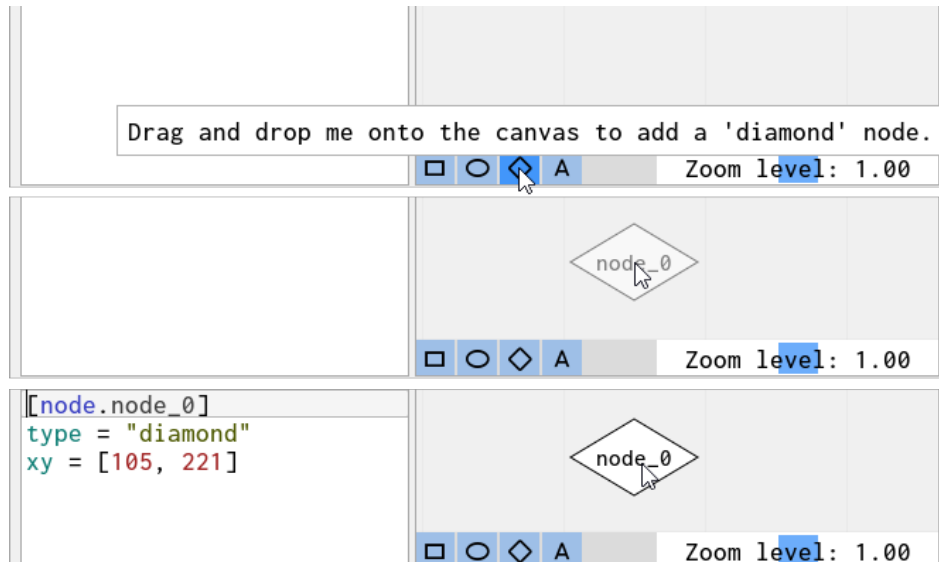
Kromě hlavního panelu nástrojů má aplikace také druhý panel, který je umístěn vespod kreslicího plátna. V jeho pravé části se nachází posuvník pro změnu úrovně přiblížení diagramu. K vykreslení posuvníku v pravé části panelu je potřeba přemístit kreslicí kurzor knihovny tak, aby zbývalo místo pouze pro tento ovládací prvek (kód 4.2).

```
constexpr auto SLIDER_WIDTH = 200;
ImGui::SetCursorPosX(ImGui::GetCursorPosX()
    + ImGui::GetContentRegionAvail().x
    - SLIDER_WIDTH);
ImGui::PushItemWidth(SLIDER_WIDTH);
ImGui::SliderInt( ... );
ImGui::PopItemWidth();
```

Zdrojový kód 4.2: Dear ImGui: posuvník v pravé části panelu

Úrovně přiblížení diagramu jsou odvozeny od velikosti písma, které je použito pro textové popisky uzlů a cest. Každá dostupná úroveň přiblížení odpovídá nějakému celému číslu, protože neceločíselné velikosti písma mohou vést na mírně rozmazaný text. Konkrétní rozsah hodnot je 6–30 s velikostí kroku 4. Posuvník v knihovně Dear ImGui umožňuje omezit rozsah, ale nepodporuje nastavení velikosti kroku. Proto musely být povolené hodnoty přiblížení „namapovány“ na hodnoty posuvníku vlastní logikou.

V levé části sekundárního panelu se nachází čtveřice tlačítek s ikonami, kde každé odpovídá jednomu dostupnému tvaru uzlu. Tato tlačítka mohou být stylem „táhni a pusť“ (*drag and drop*) přetažena na plátno, čímž se na pozici kurzoru myši objeví nový uzel. Během tažení je pod kurzorem vykreslen náhledový uzel, který naznačuje, kde se bude nový uzel po puštění tlačítka myši nacházet (obrázek 4.11).

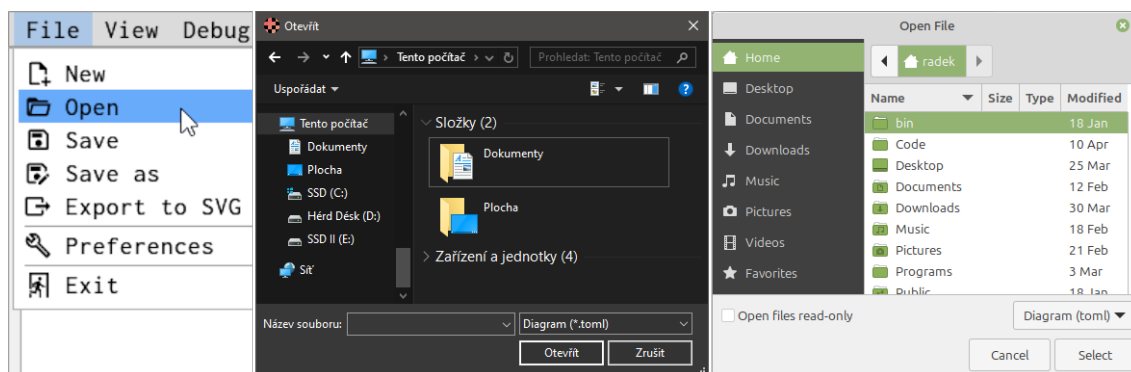


Obrázek 4.11: Dear ImGui: funkce „táhni a pusť“

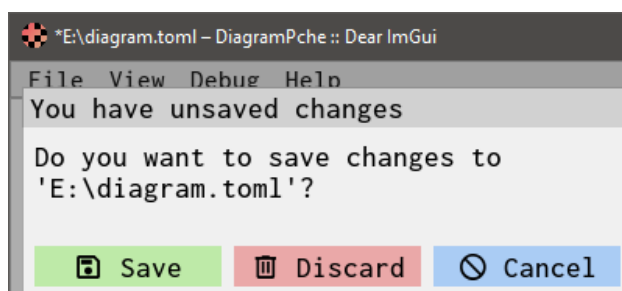
Knihovna Dear ImGui poskytuje *drag and drop* funkce, které uživateli umožňují např. zkopírovat hodnotu z jednoho ovládacího prvku do druhého, nebo měnit pořadí položek v seznamu. Pro účel přidání uzlu do plátna jsem ale poskytnuté funkce nevyužil, protože nejsou v tomto případě potřeba. Jakmile uživatel klikne na některé z tlačítek, nastaví se logická proměnná `m_is_dragndropping_node` na `true`. Tuto hodnotu si proměnná drží, dokud uživatel nepustí levé tlačítko myši, což lze kontrolovat metodou `IsMouseDown`. Pokud k tomu dojde ve chvíli, kdy se kurzor nachází v oblasti plátna, tak je do vstupního dokumentu přidána definice nového uzlu. Hodnotou logické proměnné je podmíněno vykreslení náhledového uzlu, které funguje na stejném principu jako vykreslení všech ostatních uzlů diagramu.

Do aplikace jsem dále přidal možnost uložit a nahrát ze souboru dokument popisující diagram. Tyto akce jsou dostupné v liště nabídek a také využívají písma s ikonami. Pro výběr konkrétního souboru je vyvolán nativní dialog pomocí knihovny *Native File Dialog Extended* (obrázek 4.12). Pro samotnou práci se soubory jsou použity datové proudy ze standardní knihovny jazyka C++.

Záhlaví hostitelského okna obsahuje cestu k právě editovanému souboru a znakem hvězdičky indikuje neuložené změny. Při neuložených změnách a pokusu o nahrání jiného souboru či zavření aplikace je zobrazeno varovné modální okno (obrázek 4.13). Správu textu v záhlaví hostitelského okna nechává knihovna Dear ImGui na starost vybranému backendu. V tomto případě je tedy text upravován pomocí knihovny GLFW. Ta také poskytuje callback metodu, která se zavolá při pokusu o zavření hostitelského okna.



Obrázek 4.12: Dear ImGui: nabídka s ikonami a nativní dialog



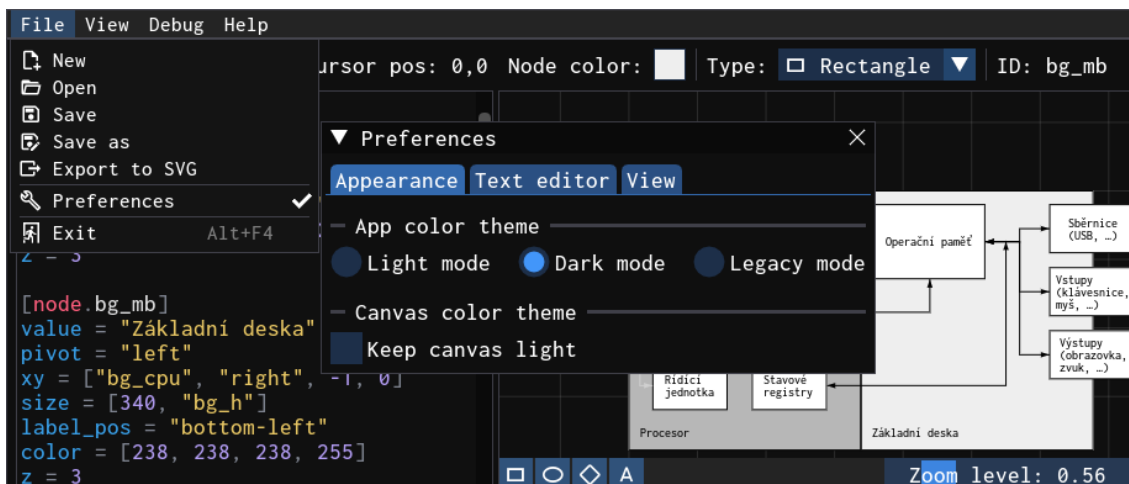
Obrázek 4.13: Dear ImGui: modální okno varující před neuloženými změnami

Aplikace dále obsahuje okno s nastavením, kde lze mimo jiné přepnout její vzhled do tmavého režimu (obrázek 4.14). Knihovna poskytuje metody `StyleColorsLight` a `StyleColorsDark`, jejichž voláním je téma přenastaveno. Interně jsou všechny použité barvy uloženy v poli `Colors`, které je součástí struktury stylů. Zmíněné metody si získají ukazatel na strukturu stylů voláním `GetStyle` a nastaví jednotlivé položky pole na předpřipravené hodnoty. Takto si lze nastavit i vlastní styl, nebo upravit ten existující. V této aplikaci u tmavého režimu přenastavuji např. barvu pozadí nemodálních oken tak, aby nebyla poloprůhledná, čímž je zvýšena čitelnost.

Poslední zmíněnou funkcí aplikace je export diagramu do formátu SVG. Pro sestavení výsledného souboru používám *simple-svg*, což je jednohlavičková knihovna z roku 2010. Tato knihovna má určité problémy, nechtěl jsem ale trávit příliš času s implementací vlastního řešení. Ostatní knihovny, které jsem našel, se zaměřovaly spíše na překlad či vykreslování souborů ve formátu SVG, nikoliv na vytvoření validního dokumentu podle zadaných parametrů.

Knihovna *simple-svg* je obsluhována třídou `Exporter`, která je psána tak, aby byla nezávislá na knihovně Dear ImGui a šla případně použít v projektu s knihovnou Qt. Podobně jako u třídy `Parser` se jedná o stavový automat, jehož instance je členem hlavní třídy `App`. Tuto instanci budu dále nazývat jako „exportér“.

Exportér je aktivován, jakmile uživatel ve vyhrazeném modálním okně potvrdí export do formátu SVG. Po potvrzení bude v dalším průchodu hlavní smyčkou dokument sestaven. Během vykreslování diagramu jsou totiž kromě metod struktury



Obrázek 4.14: Dear ImGui: aplikace v tmavém režimu, okno s nastavením

`ImDrawList` volány také analogické metody exportéra. Pokud není exportér aktivován, pak je volání jeho metod ignorováno.

V opačném případě přidá metoda do interní prioritní fronty objekt, který bude součástí výsledného dokumentu. Např. u uzlu ve tvaru obdélníka se jedná o objekty `svg::Rectangle` a `svg::Text`. Před kreslením je ještě resetován posun pohledu a úroveň přiblížení plátna, aby tyto transformace nemusely být v dokumentu negovány. Po vykreslení celého diagramu jsou do objektu typu `svg::Document` postupně přidávány prvky z prioritní fronty, načež je dokument uložen.

Prioritní fronta je zde použita kvůli hloubce uzlů nastavitelné parametrem `z`. Návrh druhé verze standardu SVG sice zmiňuje nastavení pozic objektů na třech osách, aktuální verze 1.1 ale umožňuje vykreslovat pouze podle pořadí jejich uvedení v dokumentu [22]. Objekty jsou proto podle své hloubky řazeny ve frontě a až poté přidány do výsledného dokumentu.

Dále musí exportér řešit únikové znaky, víceřádkový text a rozměry dokumentu. Knihovnu `simple-svg` jsem si také upravil, aby podporovala poloprůhlednou barvu výplně objektů. Podpora poloprůhledné barvy tahu (*stroke color*) by už vyžadovala větší zásah do knihovny a vhodnější by bylo přijít s kompletně vlastním řešením.

4.2 Implementace v knihovně egui

Projekt v knihovně `egui` je založen na šabloně `eframe template`. Jedná se v podstatě o minimální funkční projekt, který obsahuje hlavní strukturu `App`. Ta má připravené metody `new` pro inicializaci a `update` pro kód hlavní smyčky. Správu závislostí a sestavení projektu řeší nástroj `Cargo`.

Stejně jako v aplikaci s Dear ImGui jsem zde použil OpenGL jako backend. Pro správu nativního okna je interně použita knihovna `winit`. Oproti předchozímu projektu, kde bylo v určitých situacích nutné pracovat přímo s knihovnou GLFW, je zde práce s backendem abstrahována. Pro změnu názvu okna nebo reakci na pokus o jeho zavření se tedy volají připravené metody z knihovny `egui`.

Jako písmo jsem opět použil font Inconsolata. Výchozí písmo knihovny egui je sice dobře čitelné a podporuje velké množství znaků, chtěl jsem ale mít společné písmo napříč všemi třemi aplikacemi.

Struktura projektu je také podobná s řešením v Dear ImGui. Tím je myšleno rozdělení logiky do konkrétních adresářů a souborů. V jazyce Rust se zdá být častější umísťování související logiky do jediného souboru, a projekt proto není v tomto ohledu příliš „idiomatický“.

Jazyk Rust nemá klíčové slovo `static`, které by šlo použít uvnitř metody podobně jako v jazyce C++. Z toho důvodu jsou všechny proměnné, které si musí držet hodnotu přes více průchodů smyčkou, součástí hlavní struktury `App`. Pro lepší členění by bylo nutné jednotlivé části GUI rozdělit do samostatných struktur.

4.2.1 Tvorba uživatelského rozhraní v knihovně egui

V knihovně egui se pro definici uživatelského rozhraní používá vnořování uzávěrů². Kód 4.3 porovnává tento přístup s přístupem použitým v knihovně Dear ImGui. Je zde vytvořena hlavní lišta nabídek, která v nabídce *File* obsahuje akci *Exit* pro ukončení aplikace. V egui se pracuje se dvěma typy struktur: `Context` pro obecná volání knihovny a `Ui` pro pokládání ovládacích prvků. Je třeba zmínit, že v této práci používám verzi knihovny 0.33. V době psaní této práce (březen 2026) totiž vyšla verze 0.34, která v tomto ohledu přináší drobné změny.

```
// egui (Rust)
egui::TopBottomPanel::top("top_panel").show(ctx, |ui| {
    egui::MenuBar::new().ui(ui, |ui| {
        ui.menu_button("File", |ui| {
            if ui.button(concatcp!(ICON_EXIT, " Exit")).clicked() {
                ctx.send_viewport_cmd(egui::ViewportCommand::Close);
            }
        });
    });
});
// Dear ImGui (C++)
if (ImGui::BeginMainMenuBar()) {
    if (ImGui::BeginMenu("File")) {
        if (ImGui::MenuItem(ICON_EXIT " Exit")) {
            glfwSetWindowShouldClose(m_window, GLFW_TRUE);
        }
        ImGui::EndMenu();
    }
    ImGui::EndMainMenuBar();
}
```

Zdrojový kód 4.3: egui a Dear ImGui: uzávěry a stavový automat

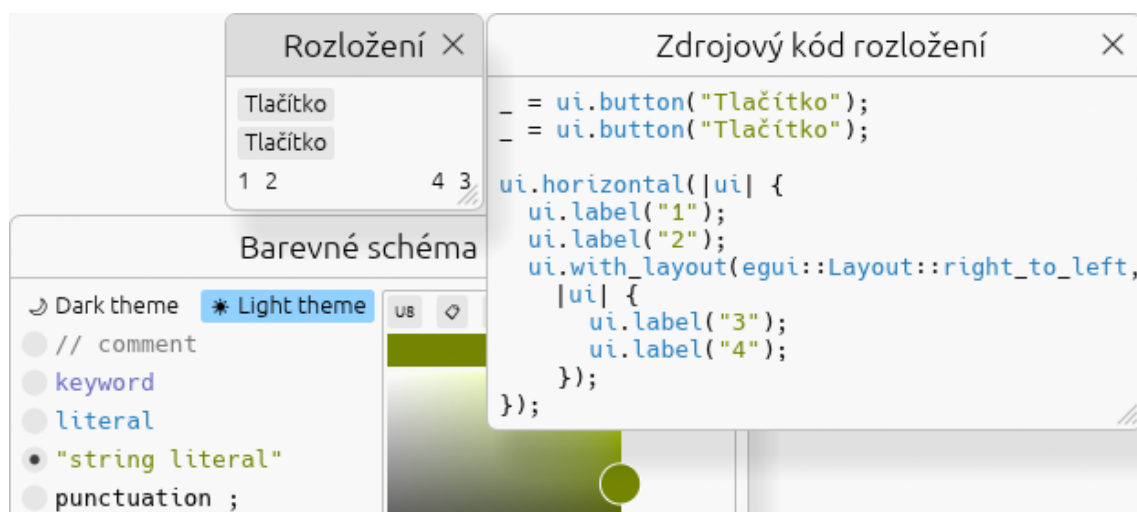
²Uzávěr (*closure*) je v jazyce Rust anonymní funkce, která dokáže pracovat s proměnnými definovanými ve stejném rozsahu platnosti, v jakém je definován samotný uzávěr [23].

Na prvním řádku ukázkového kódu je vytvořen horní panel okna. Protože takových panelů může být v aplikaci více, je třeba dodat unikátní identifikátor v podobě řetězce. Toto volání není součástí žádného nemodálního okna, a tak je panel vytvořen v okně hostitelském.

Metodě pro zobrazení panelu je předán uzávěr, jehož vstupní parametr je typu `Ui` a je zvykem jej pojmenovávat jako `ui`. Metoda o řádek níže, která slouží k vykreslení lišty nabídek, přijímá stejný typ uzávěru. Předchozí proměnná `ui` je překryta, protože je v tomto uzávěru pro „novou“ proměnnou použit stejný název. Nemůže se tak stát, že programátor použije některou z vnějších struktur typu `Ui`, což je v tomto případě žádoucí.

Pak lze již vykreslovat ovládací prvky v liště nabídek: nejprve nabídku *File* a v ní tlačítko s ikonou a popiskem. Pro zobrazení ikon opět používám písmo od *Pictogrammers* a soubor s konstantami z repozitáře *IconFontCppHeaders*. Abych mohl spojit znak ikony se zbylým řetězcem již během kompilace, musím použít `concatcp`, což je makro třetí strany. Řádek sloužící k ukončení aplikace ukazuje, jak obě knihovny řeší volání backendu.

Pro vytvoření hlavního obsahu okna je nejprve stejným způsobem zobrazen panel typu `CentralPanel`. V jeho uzávěru pak už lze vykreslovat ovládací prvky voláním metod struktury `Ui`. Obrázek 4.15 ukazuje, jak lze definovat základní rozložení. Ovládací prvky se implicitně pokládají pod sebe, pomocí dodatečných uzávěrů lze ale snadno prvky pokládat vedle sebe včetně možnosti zarovnání zprava.

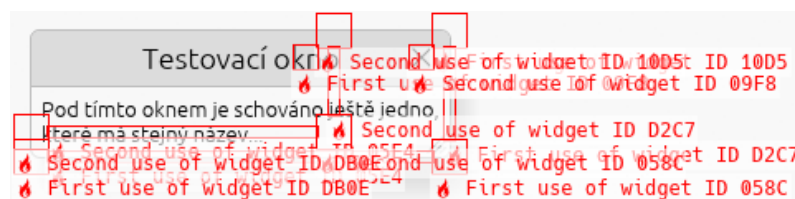


Obrázek 4.15: egui: rozložení ovládacích prvků

Na obrázku je vidět zdrojový kód daného rozložení, který využívá zvýrazňování syntaxe z balíku *egui-extras*. Použité barevné schéma může uživatel editovat za běhu pomocí výběru barev. Neexistuje však veřejný způsob, jak tyto barvy měnit z aplikačního kódu. Při startu aplikace je tak vždy použito výchozí barevné schéma.

V knihovně `egui` není problém s kolizí identifikátorů, pokud mají např. dvě tlačítka stejný textový popisek. V určitých případech, jako je tvorba nemodálního okna nebo horního panelu, je ale nutné poskytnout unikátní název, jinak knihovna nemůže korektně operovat. Takové chyby jsou také hlášeny za běhu, ale oproti chybám

v Dear ImGui nejsou příliš čitelné (obrázek 4.16). Pro zobrazení dvou oken se stejným názvem je možné při jejich definici explicitně specifikovat identifikátor, a díky tomu již nebude pro tento účel použit název okna.



Obrázek 4.16: egui: dvě okna se stejným názvem

V ukázkové aplikaci je rozložení textového editoru a kreslicího plátna řešeno podobně jako v projektu s Dear ImGui. Nejprve je zjištěna zbývající volná oblast v daném okně. Podle jejích rozměrů jsou odvozeny pozice a rozměry dvou podoblastí, které jsou vytvořeny metodou `new_child` struktury `Ui`. Místo neviditelného tlačítka je pomocí metody `interact` vytvořena oblast, ve které lze kontrolovat události jako kliknutí tlačítka myši.

4.2.2 Víceřádkové textové pole v knihovně egui

Ovládací prvek víceřádkového textového pole v knihovně egui sám o sobě nedokáže zobrazit posuvné lišty. Jejich vykreslení je zde řešeno tak, že je celý prvek umístěn do struktury `ScrollArea`, což je oblast, která poskytuje posuvné lišty pro svůj veškerý obsah. Textové pole tedy nijak neřeší, jak zobrazit pouze část svého obsahu a jak s obsahem posouvat. Ovládací prvek je vykreslen celý, a pokud se nevejde do svého vyhrazeného prostoru, tak je „useknut“. S posunutím vertikální lišty se uvnitř textového pole nic nepohybuje, a místo toho se posouvá celá oblast struktury `ScrollArea`.

Stejně jako v Dear ImGui je při definici textového pole potřeba specifikovat, aby stisk klávesy `Tab` během psaní nevedl k vybrání dalšího interaktivního prvku. I v knihovně egui je pak s tímto nastavením po stisku klávesy vložen znak tabulátoru bez možnosti přepnutí na vkládání mezer. Oproti projektu v Dear ImGui se mi zde nepodařilo přijít na to, jak vypnout navigaci pomocí kláves globálně.

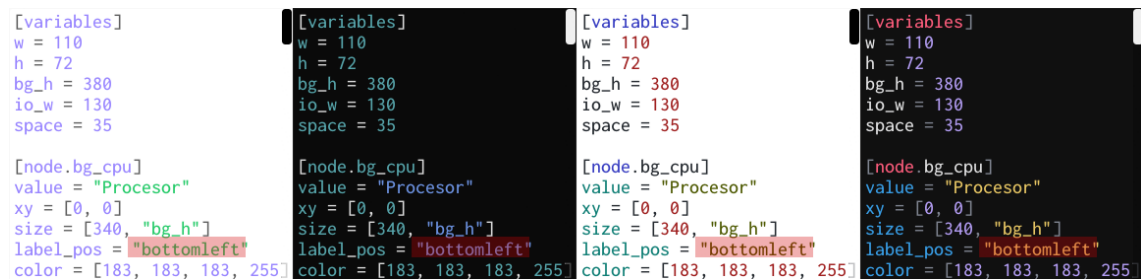
Do projektu jsem přidal balík *egui-extras*, aby bylo možné v textovém poli zvýrazňovat syntax. Zvýrazňování lze zapnout pouze pro podporované jazyky a není možné definovat vlastní pravidla. Jazyk TOML sice podporovaný je, ale využívá pouze tři barev a zvýrazňovány jsou všechny klíče bez ohledu na jejich platnost v navrženém schématu. Jak bylo zmíněno výše, barevné schéma nelze měnit z aplikačního kódu, a tak je použito to výchozí, které má ve světlém režimu špatný poměr kontrastu mezi popředím a pozadím.

Ke zprovoznění zvýrazňování syntaxe je potřeba dodatečná logika. Metodě pro vykreslení víceřádkového textového pole musí být předán uzávěr, který přijímá řetězec s obsahem textového pole a vrací strukturu typu `Galley`. Tato struktura obsahuje veškeré informace o textu připraveném k vykreslení včetně barev jednotlivých znaků. Pro její vytvoření je uvnitř uzávěru použita metoda z balíku *egui-extras*, je

ale možné přijít i s vlastním řešením. V kódu uvnitř uzávěru se mi podařilo vypnout zalamování dlouhých řádků a vynutit tak zobrazení horizontální posuvné lišty.

Zvýraznění nahlášené chyby v textovém poli je zde opět řešeno vykreslením poloprůhledného červeného obdélníka. Knihovna *toml-edit*, kterou zde používám, vrací pozice objektů ve vstupním textu jako absolutní index. Ten je třeba pomocí počítání znaků nového řádku převést na čísla řádku a sloupce nahlášené chyby. V tomto případě není potřeba řešit stav posunu, protože vykreslený obdélník je posouván zároveň s textovým polem jakožto součást struktury **ScrollArea**. K pozici obdélníka také nepřičítám vnitřní okraj (*padding*) textového pole, a proto je vykreslen s mírným posunem doleva nahoru. Hodnotu okraje se mi totiž nepodařilo nijak zjistit. S měnící se velikostí písma se navíc nekonzistentně mění i tato hodnota, a tak není možné ji změřit a nastavit „napevno“.

Pro porovnání jsem do aplikace přidal doplněk třetí strany *egui-code-editor*. Jedná se o modifikaci výchozího víceřádkového textového pole, která umožňuje definovat vlastní pravidla pro zvýrazňování syntaxe. U dodaných klíčových slov je možné nastavit jejich barvu a zároveň pro ně funguje našeptávání. Obrázek 4.17 porovnává výchozí ovládací prvek s prvkem třetí strany ve světlém a tmavém režimu.



Obrázek 4.17: egui: víceřádkové textové pole

4.2.3 Vykreslení diagramu v knihovně egui

Vykreslení diagramu podle vstupního dokumentu v jazyce TOML funguje na stejném principu jako v předchozím projektu. Struktura **Parser** provede první fázi převodu, která je nezávislá na knihovně *egui*, a pak je zpracování předáno do struktury **App**, kde jsou dopočítány přesné souřadnice objektů v diagramu.

V projektu s *Dear ImGui* bylo možné po zjištění souřadnic rovnou začít vykreslovat grafická primitiva. Hloubka uzlů definovaná parametrem *z* se řešila pomocí kanálů, které struktura **ImDrawList** podporuje. V *egui* ale struktura **Painter** žádný takový mechanismus nemá a je podporován pouze „malířův algoritmus“ [24]. Z toho důvodu jsou finální informace pro vykreslení zabaleny do nové struktury a přidány do prioritní fronty, která své prvky řadí podle parametru *z*. Po zpracování všech objektů v diagramu jsou prvky z fronty postupně vybírány a vykreslovány.

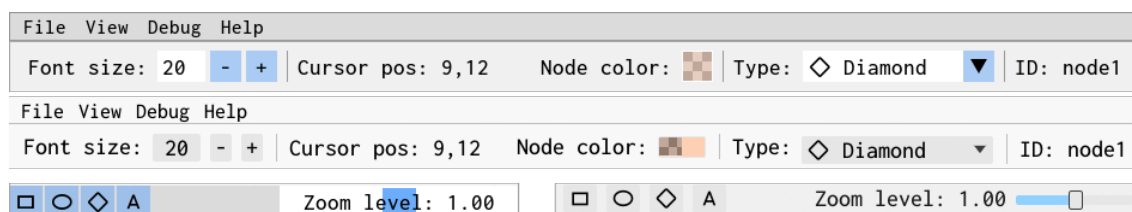
Prvky přidávané do této kolekce implementují určitý *trait* (rozhraní v jazyce Rust) a využívá se tak polymorfismu. Kromě metody pro vykreslení pomocí struktury **Painter** implementují také metodu použitou při tvorbě dokumentu ve formátu SVG. Ten je zde také tvořen během průchodu hlavní smyčkou. U jednotlivých

prvků fronty je v tomto případě kromě metody `draw` volána i metoda `draw_svg`. To v kombinaci s použitím modernějšího balíku `svg` vede na značně čistší implementaci exportu do formátu SVG, než jaká je v předchozím projektu.

Samotné vykreslování a interakce s kreslicím plátnem fungují podobně jako v Dear ImGui. Plátno je oblast, která v okně zbyla po přidání textového pole, a metodou `allocatePainter` struktury `Ui` je tato oblast vyhrazena pro kreslení. Volaná metoda vrací kromě struktury `Painter` také strukturu `Response`, ze které lze např. zjistit, zdali uživatel neposouvá danou oblast pravým tlačítkem myši. Ne-ní zde proto potřeba používat metodu `interact`, která je analogií k neviditelnému tlačítku z knihovny Dear ImGui.

4.2.4 Další funkce aplikace v knihovně egui

Do aplikace jsem dále přidal panel nástrojů. Zde je zobrazena pozice kurzoru v textovém poli, kterou je v tomto případě možné zjistit i z výchozího ovládacího prvku. Opět je ovšem nutné převést absolutní pozici kurzoru na čísla řádku a sloupce.



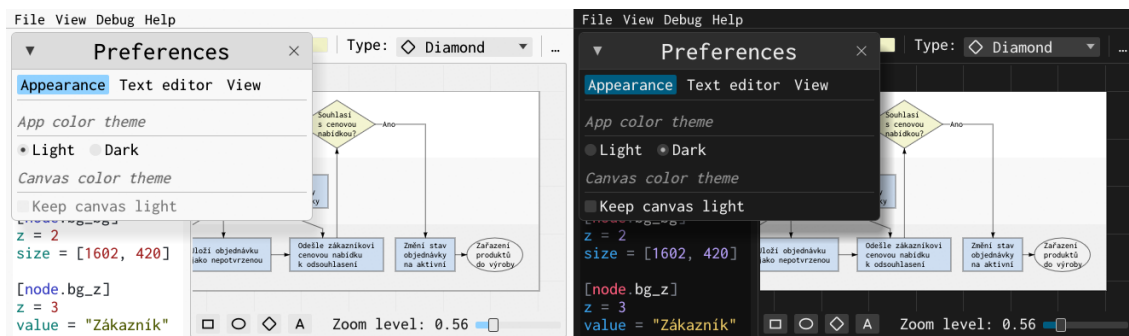
Obrázek 4.18: Dear ImGui a egui: hlavní a sekundární panel nástrojů

Ovládací prvky výběr barev a rozbalovací seznam fungují podobně jako v Dear ImGui. U seznamu je opět nastavena pevná šířka, aby se neměnila podle délky aktuálně vybrané položky. Na obrázku 4.18 je vidět, jak je po vykreslení seznamu zbytek prvků v horizontálním rozložení posunut dolů. Tento vizuální problém byl již nahlášen jiným uživatelem jako [issue #7412](#) na platformě GitHub a zatím nebyl oficiálně vyřešen.

Celočíselné vstupní pole pro změnu velikosti písma je realizováno pomocí prvku `DragValue`, jehož hodnotu lze měnit tažením myši, nebo přepsáním jeho textu. K tomuto prvku jsem přidal dvě tlačítka a po jejich vykreslení řeším velikost kroku a omezení rozsahu, viz kód B.3. Změna velikosti písma je řešena pomocí uzávěru, který je předán metodě `scope` struktury `Ui`. V tomto uzávěru je přenastaven styl aplikace tak, aby velikost písma odpovídala nastavené hodnotě. Pak je vykresleno textové pole a s koncem uzávěru je upravený styl aplikace zahojen.

V sekundárním panelu pod kreslicím plátnem se v pravé části nachází posuvník pro změnu úrovně přiblížení. Zarovnání vpravo je dosaženo uzávěrem, který byl již zmíněn. U posuvníku v egui je možné nastavit jeho rozsah i velikost kroku, a tak je kód pro jeho umístění i logiku oproti předchozímu projektu stručnější.

Na podobném principu, jako v projektu s knihovnou Dear ImGui, pak fungují tlačítka pro přidání nového uzlu do diagramu a funkce přepínání mezi světlým a tmavým režimem. Pro nativní dialogy je použita knihovna `RFD` a práce se soubory využívá funkcí ze standardní knihovny jazyka Rust.



Obrázek 4.19: egui: aplikace ve světlém a tmavém režimu, okno s nastavením

4.3 Implementace v knihovně Qt

Aplikace využívající knihovnu Qt je psaná v jazyce C++ a pro její sestavení opět využívám nástroje CMake a Ninja. Knihovna Qt je relativně velká, a proto zde není vhodné mít zdrojové kódy všech závislostí jako součást projektu. Z toho důvodu jsem zvolil časté řešení, kdy se na stroj vývojáře nainstaluje celé SDK a cesta k němu je předána programu CMake. Při distribuci hotové aplikace jsou pak potřebné komponenty knihovny Qt obvykle dodávány jako samostatné soubory, které se linkují dynamicky.

Struktura projektu je zde odlišná od předchozích dvou aplikací. Definice hlavního okna se nachází ve vlastní třídě `GUIMainWindow`, která dědí ze třídy `QMainWindow` poskytnuté knihovnou. V konstruktoru vlastní třídy je postupně definováno veškeré uživatelské rozhraní hlavního okna. Pro pokládání ovládacích prvků jsou k dispozici různé typy rozložení. V hlavním okně je použito vertikální rozložení `QVBoxLayout`, v němž se pod sebou nachází prvky `QSplitter` a `QLabel`. První objekt zapouzdřuje víceřádkové textové pole s kreslicím plátnem a umožňuje uživateli měnit poměr jejich šířek. Druhý objekt je textový popis reprezentující stavový řádek.

Všechny ovládací prvky a rozložení jsou vytvářeny klíčovým slovem `new`, a odkaz na ně je tedy uchován v proměnných typu ukazatele (*pointer*). O životní cyklus objektů se sice stará knihovna Qt, i přesto může programátor v určitých případech napsat „nebezpečný“ kód. Příkladem je pokus o odstranění již odstraněného objektu z grafické scény, nebo špatné použití dynamického přetypování. Na přidání ovládacích prvků jsou navěšeny callback metody, a pokud je potřeba s nimi pracovat i v jiných částech kódu, musí být jejich ukazatel členem třídy `GUIMainWindow`.

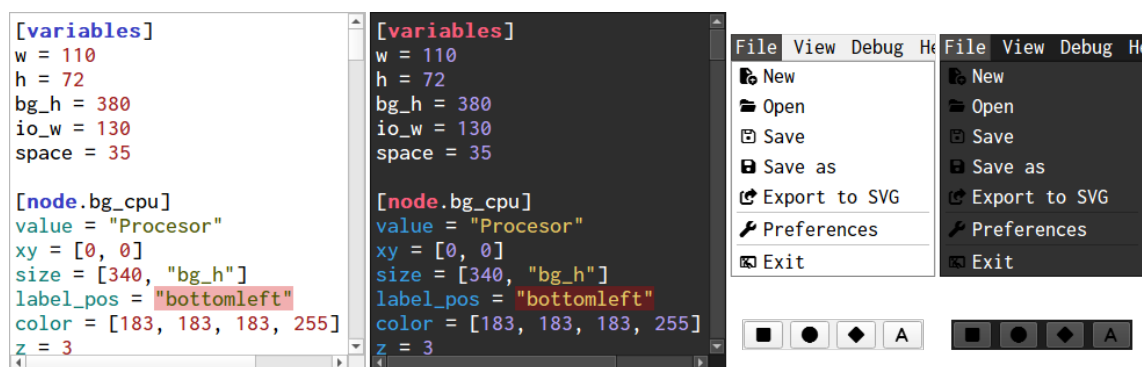
Pro přidání víceřádkového textového pole je použit objekt typu `QPlainTextEdit`. Ten podporuje všechny základní funkce a klávesové zkratky, jako textové pole z předchozích projektů. Bez složitých modifikací dokáže zobrazit obě posuvné lišty a lze pro něj definovat pravidla zvýrazňování syntaxe. Jako jediný dokáže tento ovládací prvek při stisku `Ctrl + Z` korektně vrátit změny i v případě, kdy byl jeho obsah změněn „zvenčí“. Tedy když byla u nějakého uzlu změněna barva výplně pomocí výběru barev, nebo jeho tvar pomocí rozbalovacího seznamu.

Stejně jako v předchozích případech zde nelze snadno nastavit, aby stisk klávesy `Tab` vkládal mezery namísto znaku tabulátoru. Výchozí šířka odsazení je navíc

o velikosti osmi znaků. Zvýrazňování chyb v textovém poli je řešeno pomocí pomyslného textového kurzoru. Ten se chová jako stavový automat a jeho pozice je nejprve nastavena na začátek nahlášené chyby. Poté je zvýrazněna patřičná část textu a barva zvýraznění je nastavena na červenou.

Zvýrazňování syntaxe je zde řešeno vytvořením vlastní třídy, která dědí ze třídy `QSyntaxHighlighter`. V jejím konstruktoru jsou definována očekávaná klíčová slova a k nim příslušné barvy. Následně se z víceřádkového textového pole získá ukazatel na jeho dokument, a ten je metodou `setDocument` předán instanci „zvýrazňovače“.

V tomto řešení není možné snadno nastavit, aby se s přepnutím aplikace do tmavého režimu změnilo i barevné schéma pro zvýrazňování syntaxe. Nejjednodušší způsob, na který jsem přišel, je mít pro každé téma aplikace samostatnou třídu a přepínat mezi nimi.



Obrázek 4.20: Qt: víceřádkové textové pole a problém s ikonami v tmavém režimu

Obrázek 4.20 ukazuje textové pole a prvky s ikonami ve světlém a tmavém režimu. V projektu s Qt nelze k výchozímu písmu snadno přidat font s ikonami, a tak používám knihovnu *QtAwesome*, která umožňuje v aplikaci používat ikony z knihovny *Font Awesome*. Jak je ale vidět, ikony po přepnutí do tmavého režimu nezmění svou barvu, a nejsou tak dobře viditelné. Nepodařilo se mi najít žádný jednoduchý způsob, jak tomuto předejít. Dále je v bezplatné verzi *Font Awesome* k dispozici o dost méně ikon než v knihovně od *Pictogrammers*, a tak je např. místo ikony elipsy použita ikona kruhu.

Nejdůležitější callback metoda v této aplikaci je ta, která reaguje na změnu obsahu ve víceřádkovém textovém poli. Tato metoda mimo jiné nastaví, že dokument má neuložené změny. Pro tento účel má knihovna připravenou metodu `setWindowModified`, která se stará i o zobrazení znaku hvězdičky v záhlaví okna. Po přidání této funkce se ale objevil problém, kdy byl dokument označen jako změněný hned po startu aplikace.

Příčinou tohoto chování je třída pro zvýrazňování syntaxe. Po tom, co zvýrazňovač dostane odkaz na textové pole, v něm provede určité změny. I když se obsahově dokument nijak nezmění, tak pro zavolání callback metody to stačí. Snadným řešením by tak mělo být na konci konstruktoru označit dokument jako nezměněný. Jak se ale ukázalo, změny dokumentu probíhají až po dokončení logiky konstrukturu v interní smyčce knihovny Qt. Proto jsem musel na konci konstrukturu spustit časovač

```

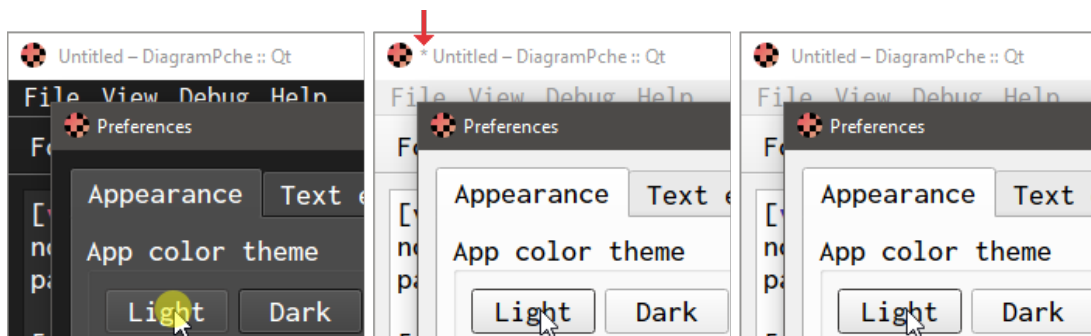
GUIMainWindow::GUIMainWindow() {
    // ...
    m_source = new QPlainTextEdit(); // Víceřádkové textové pole
    m_highlighter_light = new Highlighter(); // „Zvýrazňovač“
    m_highlighter_light->setDocument(m_source->document());
    // ...
    QTimer::singleShot(0, this, [this] { setWindowModified(false); });
}

```

Zdrojový kód 4.4: Qt: výňatek z konstruktoru hlavního okna

`QTimer`, který má po svém uběhnutí označit dokument jako nezměněný (kód 4.4). Doba trvání časovače může být nastavena na 0 milisekund, protože obsluha časovačů probíhá v interní smyčce až poté, co je obsloužen zvýrazňovač.

Řešení s časovačem muselo být použito i v okně nastavení. Zde může uživatel vypnout zvýrazňování syntaxe, nebo přepnout aplikaci do tmavého režimu, což vede i na přepnutí zvýrazňovače. V obou těchto případech je tedy dokument opět označen jako změněný. Na obrázku 4.21 je vidět, jak přepnutí vzhledu aplikace přidá znak hvězdičky do záhlaví okna. Na dalším snímku již indikátor není, protože byla provedena logika časovače `QTimer`.



Obrázek 4.21: Qt: nevhodný indikátor neuložených změn v záhlaví okna

Ve větších projektech se pro textový editor pravděpodobně často používá vlastní řešení, nebo doplněk třetí strany, jako je např. *QScintilla*. Do ukázkové aplikace jsem ale žádný další doplněk nepřidával, protože výchozí textové pole pokrylo všechny funkce z předchozích projektů. Výjimkou je funkce našeptávání, kterou podporuje pouze doplněk třetí strany v knihovně `gugi`.

4.3.1 Vykreslení diagramu v knihovně Qt

Pro realizaci kreslicího plátna by bylo možné vytvořit vlastní ovládací prvek dědícím ze třídy `QWidget`. V něm by se pak přepsaly metody, které reagují na uživatelské interakce myši, společně s metodou `paintEvent`, která by se starala o vykreslení diagramu. Jednalo by se o přístup na nižší úrovni, který se více podobá řešením z předchozích dvou projektů. V této aplikaci ale využívám grafickou scénu knihovny Qt,

což je běžně používaný prvek, který se objevuje i v ukázkových projektech oficiální dokumentace. Scéna je vytvořena děděním ze třídy `QGraphicsScene`.

Jakmile uživatel upraví vstupní dokument popisující diagram, spustí se výše zmíněná callback metoda. Obsah textového pole je předán třídě `Parser`, která provede první fázi zpracování stejně jako v předchozích dvou projektech. Následně jsou za pomoci knihovny Qt dopočítány přesné souřadnice uzlů a pro každý z nich je vytvořena instance vlastní třídy, která dědí ze třídy `QGraphicsItem`.

Každý prvek ve scéně musí implementovat dvě metody. První metoda se stará o jeho vykreslení pomocí instance třídy `QPainter`, která poskytuje metody pro vykreslování grafických primitiv. Druhá metoda, kterou je potřeba implementovat, vrací obdélníkovou oblast ohraničující tento prvek. Veškeré vykreslování musí probíhat právě v této oblasti. U prvků ve scéně je možné nastavit jejich hloubku metodou `setZValue`, takže není potřeba řešit dodatečné řazení.

Pro každý objekt diagramu je vytvořena instance takovéto třídy a ukazatel na ni je předán grafické scéně, která od té chvíle přebírá nad objektem kontrolu. Scéna podle aktuálního pohledu na plátno určuje, jaké objekty jsou viditelné, a mají tedy být vykresleny. Pokud je vstupní dokument jakkoliv změněn, pak je celá scéna vyčištěna a následně znovu vybudována. Logika, která by hlídala, jaké uzly ve scéně se změnilly a jaké byly přidány či odebrány, by byla zbytečně komplexní a náchylná na chyby. Dokud ovšem není vstupní dokument upraven, tak je obsah scény neměnný. To je rozdíl oproti projektům s přístupem `IMGUI`, kde je zpracování vstupu a následné vykreslení součástí každého průchodu hlavní smyčkou.

Aby mohla být grafická scéna zobrazena uživateli, musí být ještě celá „zabalena“ do další vlastní třídy, která dědí z `QGraphicsView`. Tato třída reprezentuje pohled na danou scénu a ve výchozím nastavení ji vykresluje s posuvnými lištami. Lišty jsou v ukázkové aplikaci skryty a velikost výřezu scény, kde se může pohled pohybovat, je nastavena na velké číslo. Chování aplikace díky tomu odpovídá předchozím projektům, kde lze s pohledem posouvat „donekonečna“.

Třída obalující scénu dále implementuje logiku posouvání pohledu při držení pravého tlačítka myši. V projektech s přístupem `IMGUI` je podle pohybu myši upravena hodnota proměnné, která reprezentuje posun pohledu a přičítá se k souřadnicím všech objektů diagramu. Zde jsou podle pohybu myši upravovány hodnoty posuvných lišt, které jsou sice skryty, ale stále určují pozici pohledu na scénu.

4.3.2 Další funkce aplikace v knihovně Qt

V aplikaci je opět možné vybrat uzel diagramu kliknutím levého tlačítka myši. U vybraného uzlu pak lze měnit jeho barvu a tvar. Obsluha kliknutí se řeší v grafické scéně přepsáním metody `mousePressEvent`. Scéna dokáže vrátit kolekci objektů, které se nachází na určitých souřadnicích. Kolekce je seřazena podle parametru `z`, a tak stačí její první prvek označit jako vybraný.

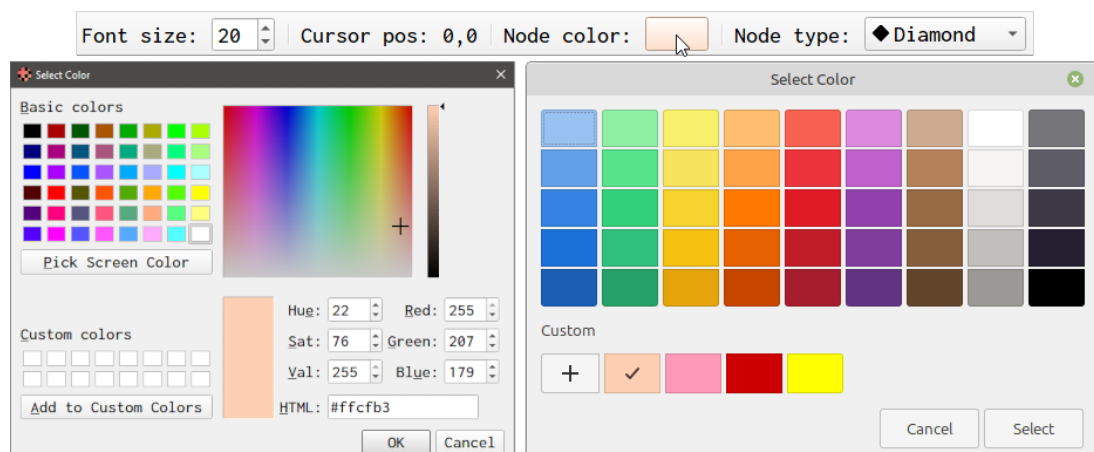
Logika změny barvy a tvaru vybraného uzlu ale probíhá ve třídě hlavního okna, protože ta má odkaz na textové pole, jehož obsah je potřeba měnit. Scéna tedy potřebuje nějak oznámit oknu, na které nemá referenci, že byl vybrán určitý uzel. K tomu v knihovně Qt slouží tzv. signály, jejichž princip ukazuje kód 4.5. Jakmile

uživatel klikne na nějaký uzel ve scéně, tak je zavolána metoda `NodeClicked`, což lze chápat jako vyslání signálu. Interní smyčka knihovny Qt se pak stará o to, aby reakcí na vyslaný signál bylo spuštění metody `OnNodeClick` v hlavním okně.

```
class GUIScene : public QGraphicsScene {
    void NodeClicked(const std::string& id);
}
class GUIMainWindow : public QMainWindow {
    void OnNodeClick(const std::string& id);
}
GUIMainWindow::GUIMainWindow() {
    m_scene = new GUIScene( ... );
    connect(m_scene, &GUIScene::NodeClicked,
           this, &GUIMainWindow::OnNodeClick);
}
```

Zdrojový kód 4.5: Qt: princip signálů

Po přijetí signálu si hlavní okno získá informace o uzlu, na který uživatel klikl, a aktualizuje podle nich panel nástrojů. Pro účel výběru barev má knihovna Qt předpřipravené dialogové okno. Do panelu nástrojů jsem umístil vlastní tlačítko, které dědí ze třídy `QPushButton` a jehož barva odpovídá právě vybranému uzlu. Po kliknutí na toto tlačítko je zobrazen dialog výběru barev. Na obrázku 4.22 je vidět, že i když je použit multiplatformní styl *Fusion*, tak se vzhled tohoto dialogu liší v závislosti na platformě. Vybere-li uživatel v dialogu novou barvu, pak je ze třídy barevného tlačítka vyslán signál do hlavního okna, které odpovídajícím způsobem upraví obsah textového pole. Díky navěšené callback metodě se po úpravě textového pole překreslí grafická scéna.



Obrázek 4.22: Qt: výběr barev v prostředích Windows 10 a Cinnamon

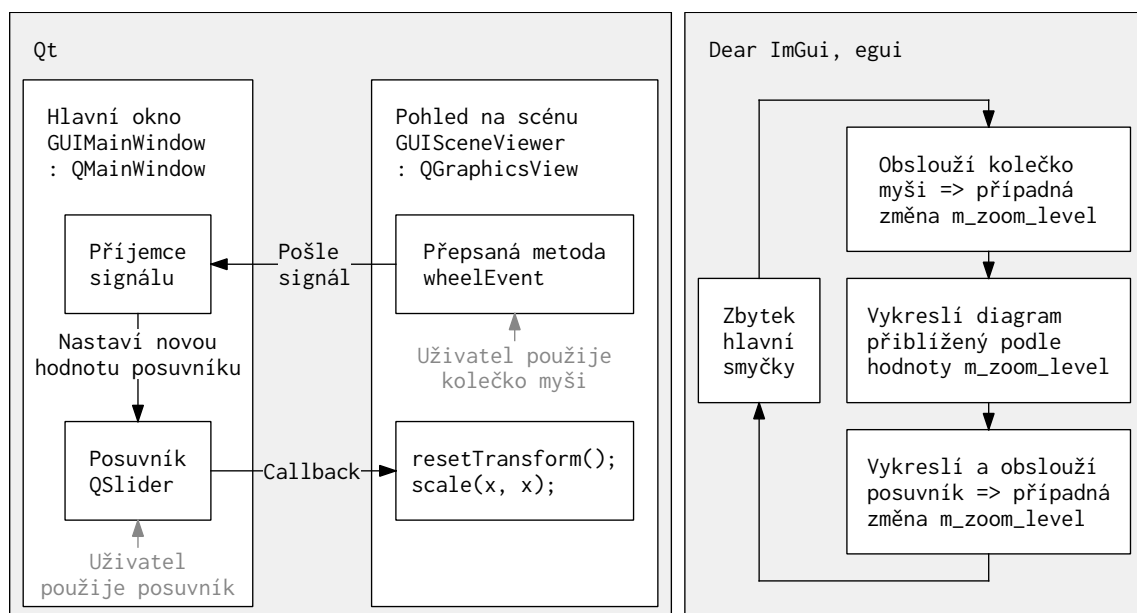
Pod vykresleným diagramem se nachází čtveřice tlačítek pro přidání nového uzlu a posuvník pro změnu úrovně přiblížení vykresleného diagramu. Tyto ovládací prvky se nacházejí v horizontálním rozložení `QHBoxLayout` a pomocí metody `addStretch`

lze snadno nastavit, aby se posuvník nacházel v pravé části rozložení. Tato metoda vloží do daného rozložení oddělovač levé a pravé strany, a je tedy volána mezi přidáním tlačítek a posuvníku.

V projektech s přístupem ImGui byla proměnná reprezentující úroveň přiblížení diagramu členem hlavní třídy a její hodnota mohla být upravována ovládacím prvkem posuvníku i kolečkem myši. V tomto projektu je ale úroveň přiblížení držena na dvou místech: ve třídě, která reprezentuje pohled na grafickou scénu, a jako součást stavu posuvníku.

Tyto dvě hodnoty je potřeba nějak synchronizovat pomocí signálů, protože jedna se nachází ve „třídě pohledu“ a druhá v hlavním okně. Místo pokusu o obousměrnou synchronizaci jsem zvolil jednosměrný přístup. Posuvník má na sebe navěšenou callback metodu, která při změně jeho hodnoty změní úroveň přiblížení. Hlavní okno, a tedy i callback metoda, má přístup ke třídě pohledu, a tak může její hodnotu přiblížení změnit napřímo. Hodnota ve třídě pohledu je ta, která určuje skutečnou úroveň přiblížení vykreslené scény. Logika posuvníku je tímto tedy zprovozněna a zbývá vyřešit kolečko myši.

Otáčeli-li uživatel kolečkem myši nad plátnem, pak je volána poskytnutá metoda `wheelEvent` právě ve třídě pohledu. Než aby ale třída rovnou upravila svou hodnotu přiblížení a následně ji propagovala do posuvníku, vyšle pouze signál do hlavního okna. Po přijetí signálu upraví hlavní okno hodnotu posuvníku, čímž se spustí jeho callback metoda, která se postará o zbytek. Obrázek 4.23 ukazuje schéma popsané synchronizace a porovnává ji s řešením v předchozích projektech.



Obrázek 4.23: Princip změny úrovně přiblížení vykresleného diagramu

Pokud by třída pohledu při reakci na otáčení kolečkem myši rovnou měnila svou úroveň přiblížení, tak by nastala komplikace při pokusu o aktualizování hodnoty posuvníku. Callback metoda, která je navěšená na posuvníku, by totiž měnila hodnotu přiblížení podruhé. V této metodě nelze nijak zjistit, zdali změnu hodnoty

provedl uživatel, nebo aplikační kód. Jediným řešením by tak bylo dočasné vypnutí obsluhy callback metod pomocí volání metody `blockSignals`.

Zprovoznění tlačítek typu „táhni a pusť“ s sebou také neslo určité komplikace. Stejně jako v Dear ImGui a egui, jsou i v knihovně Qt poskytnuté *drag and drop* funkce zbytečně komplexní pro scénář v ukázkové aplikaci. Proto jsem implementoval další typ tlačítka, které vysílá dva signály: jeden pro začátek tažení a druhý pro jeho puštění. Tyto signály přijímá grafická scéna, kde jsem zamýšlel, že po prvním signálu do sebe přidá nový objekt náhledového uzlu. Dokud nepřijde druhý signál, tak aktualizuje pozici náhledového uzlu tak, aby se nacházel pod kurzorem myši. S druhým signálem pak scéna pošle signál do hlavního okna, aby patřičně změnilo vstupní dokument, a náhledový uzel smaže.

Pro zjištění pozice kurzoru, která poslouží k určení souřadnic náhledového uzlu, jsem chtěl použít metodu `mouseMoveEvent` v grafické scéně, která se má zavolat při pohybu kurzoru nad scénou. Zde ale nastal problém: pokud uživatel táhne nějaké tlačítko, tak se tato metoda nikdy nezavolá. Další informace, kterou je potřeba znát, je zdali se při puštění tlačítka nachází kurzor nad plátnem. Je-li kurzor mimo plátno, tak to znamená, že se má operace přidání nového uzlu zrušit. Pro zjištění této informace by měla sloužit metoda `underMouse` ve třídě pohledu. Ta ale podléhá podobnému problému: při tažení tlačítka vrací vždy *false* neohledně na pozici kurzoru.

Logika přidání nového uzlu se tedy v tu chvíli nacházela ve čtyřech různých třídách: vlastní tlačítko, hlavní okno, scéna a pohled na scénu. Z toho dvě části nefungovaly tak, jak jsem podle dokumentace očekával. Proto musím v konstruktoru třídy pohledu volat metodu `installEventFilter`, což umožňuje reagovat na veškeré interní události knihovny přepsáním metody `eventFilter`. V této metodě je již možné získat souřadnice kurzoru myši, i když uživatel táhne tlačítko.

Metoda `eventFilter` je spouštěna s vysokou frekvencí a získané souřadnice kurzoru jsou počítány absolutně od levého horního rohu obrazovky. Třída pohledu má v sobě uložen ukazatel na grafickou scénu a zároveň poskytuje metody pro přepočítání souřadnic. Pokud třída pohledu od scény zjistí, že uživatel právě pokládá nový uzel, přepočte absolutní souřadnice kurzoru na jeho souřadnice ve scéně. Ty jsou posílány zpět do scény, která podle nich posune náhledový uzel.

Důvod, proč není metoda `eventFilter` implementována rovnou ve scéně, je ten, že grafická scéna nemá k dispozici metodu pro přepočítání souřadnic kurzoru. Třída pohledu navíc obsahuje ukazatel na scénu, takže je snadné k ní odsud přistupovat. Opačný směr je mírně složitější, ale i v tomto řešení jsem jej nakonec musel využít. Jakmile uživatel pustí levé tlačítko myši, musí scéna zkontrolovat, zdali se nad ní nachází kurzor. To ale ve scéně zjistit nelze, a tak je nutné dotázat se třídy pohledu.

Pokud třída pohledu potvrdí, že se kurzor nachází nad plátnem, je možné poslat signál hlavnímu oknu, aby do vstupního dokumentu přidalo definici nového uzlu. Po odeslání signálu jsem se snažil náhledový uzel ze scény smazat. Nedošlo mi ale, že změna vstupního dokumentu znamená kompletní vyčištění scény. Snažil jsem se tedy smazat neexistující objekt, což vedlo k pádu aplikace. Po opravení této chyby byl celý mechanismus již funkční.

Implementace této funkce by se pravděpodobně dost usnadnila, pokud by byl nový uzel přidáván pomocí dvou kliknutí namísto tažení. Uživatel by např. stiskl

tlačítko s obdélníkem a to by zůstalo „zamáčké“. Druhým kliknutím do scény by se pak na toto místo přidal obdélníkový uzel. Chtěl jsem ale, aby se aplikace chovala co nejpodobněji předchozím projektům s přístupem ImGui, kde implementace této funkce nebyla zdaleka tak problémová.

Co se naopak v knihovně Qt implementovalo velmi snadno, byl export diagramu do formátu SVG. Knihovna má totiž k dispozici třídu `QSvgGenerator`, která dokáže do tohoto formátu uložit obsah grafické scény. Další funkce, pro kterou má knihovna připravené řešení, je zobrazení nativního dialogu pro výběr souboru. V předchozích projektech se toto řeší dodatečnou závislostí, zatímco v knihovně Qt je to záležitost volání jedné metody.

5 Porovnání přístupů a ukázkových aplikací

5.1 Porovnání přístupů IMGUI a RMGUI z pohledu funkcí a aplikačního kódu

Implementace ukázkových aplikací ve vybraných knihovnách poskytla hlubší vhled do problematiky přístupů IMGUI a RMGUI. Opět je ale třeba zmínit, že definice rozdílů mezi těmito přístupy není příliš striktní, a některé detaily v architekturách vybraných knihoven proto nemusí nutně vypovídat o daném přístupu jako takovém. Např. v přístupu RMGUI je nutné pro reakci na stisk tlačítka použít callback metodu, ale komunikace mezi částmi aplikace nemusí být nutně řešena pomocí signálů, jako tomu je v knihovně Qt.

Z pohledu dostupných funkcí je knihovna Qt více připravená k tvorbě „běžných“ aplikací pro koncové uživatele. V aplikačním kódu lze snadno upravit text v záhlaví okna, nastavit ikonu aplikace, nebo zobrazit nativní dialogové okno. Pro vykreslení diagramu, nebo jiné libovolné grafiky, je zde připravena grafická scéna, která umožňuje svůj obsah exportovat do formátu SVG. K dispozici je dále např. kontejner `QSplitter`, který v ukázkové aplikaci obsahuje víceřádkové textové pole a kreslicí plátno. Tento kontejner umožňuje uživateli upravovat poměr šířek mezi svými ovládacími prvky, a není tak nutné přicházet s vlastním řešením.

Víceřádkové textové pole je jeden ze standardních ovládacích prvků, ale i tak má knihovna Qt v tomto ohledu náskok. Použité knihovny s přístupem IMGUI jsou v aktivním vývoji, a tak se toto může časem zlepšit, momentálně jsou však dostupná řešení pro textová pole zatížena drobnými chybami a chybějícími funkcemi. V knihovnách s přístupem IMGUI bylo naopak jednodušší přepínat mezi barevnými styly aplikace a lépe se mi zde pracovalo s fontem ikon.

Co se týče vzhledu, tak aplikace využívající přístup IMGUI by měly vypadat vždy stejně neohledně na cílovou platformu. Knihovna Qt může na některých místech využívat nativní ovládací prvky poskytnuté operačním systémem, a na jiných místech vykreslovat vlastní prvky, které tento vzhled imitují. I při použití multiplatformního vzhledu *Fusion* se může stát, že bude vzhled určitých částí aplikace platformně závislý.

Knihovny s přístupem RMGUI mají obvykle k dispozici robustnější řešení pro rozložení ovládacích prvků. V knihovně Qt není problém umístit ovládací prvky do mřížky, která aktualizuje rozměry svých řádků a sloupců na základě velikosti okna aplikace. Funkce pro „responzivní“ rozložení ovládacích prvků jsou v knihovnách s přístupem IMGUI často omezené.

Hlavní rozdíl mezi přístupy IMGUI a RMGUI je v principu psaní aplikačního kódu. Přístup IMGUI je založen na programování v nekonečné smyčce, kdy je knihovně na základě stavu aplikace popsáno, jak má být vykresleno uživatelské rozhraní. Přístup RMGUI je založen na programování callback metod a synchronizaci stavů.

Při programování v nekonečné smyčce může hrát roli pořadí provedených operací. V ukázkové aplikaci se např. nachází logika, která zajišťuje, aby nemohl být vybrán neexistující uzel. Uživatel totiž může kliknout na uzel diagramu, čímž ho vybere, a následně jeho definici smazat ze vstupního dokumentu. Zmíněná logika zjistí, že identifikátor vybraného uzlu se v dokumentu již nenachází, a výběr zruší. Součástí funkce *drag and drop*, kdy uživatel přetáhne na plátno nový uzel, by pak mělo být vybrání nově přidaného uzlu. Pokud je ale tato funkce obsloužena před výše popsanou logikou, tak je výběr nově přidaného uzlu okamžitě zrušen. Třída `Parser` jej totiž ještě nezpracovala, a tak je jeho identifikátor vyhodnocen jako neexistující. Prohozením pořadí těchto dvou operací se problém vyřeší.

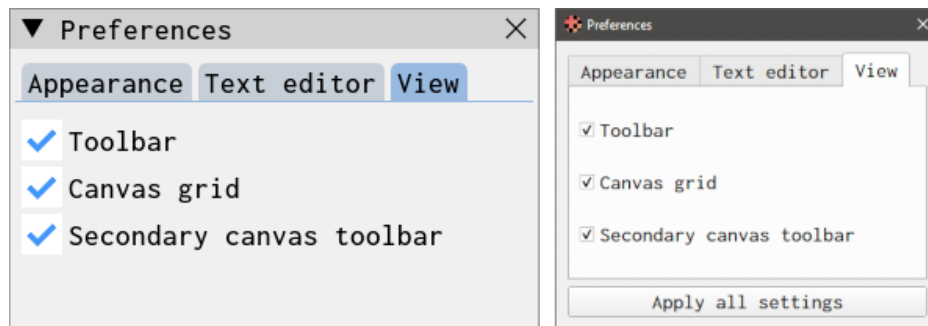
Při práci s callback metodami a se signály v knihovně Qt může také dojít k nečekanému chování aplikace. Pro textové pole je např. dostupný callback `textChanged`. Ten volá dodanou metodu, jakmile uživatel změnil obsah textového pole. V pozdější fázi vývoje může programátor chtít v určité situaci měnit obsah textového pole z aplikačního kódu, a zde je nutné si uvědomit, že zmíněná callback metoda bude opět spuštěna. Pokud je tělo metody složitější, tak může spouštěním dalších callback metod docházet k „dominovému efektu“, který vede na nečekané chování aplikace a hůře se ladí. V ukázkové aplikaci je celkem 60krát volána funkce `connect`, takže práce s callback metodami a signály je v každé větší aplikaci nevyhnutelná.

I v knihovně Qt je někdy potřeba řešit pořadí provedených operací: funkce pro vytvoření nového dokumentu musí nejdříve vyčistit textové pole a až poté nastavit dokument jako nezměněný. Vyčištění pole se totiž také bere jako změna obsahu, takže v opačném pořadí by u nového dokumentu bylo nastaveno, že obsahuje neuložené změny. Situace se dále komplikuje, pokud jsou callback metody nečekaně volány z interní smyčky knihovny, jako tomu bylo u zvýrazňovače syntaxe. Programátor nad tímto nemá přímou kontrolu a tyto problémy se opět hůře ladí. Komplikací také je, když funkce dodané knihovnou, na které se programátor spoléhá, přestanou v určitých chvílích fungovat. Příkladem je zjištění pozice kurzoru při tažení tlačítka. Přístup IMGUI je v tomto ohledu obvykle méně abstraktní, a tyto problémy se v něm proto nevyskytují.

S přístupem RMGUI přichází také problematika synchronizace stavů. V panelu nástrojů ukázkové aplikace lze měnit velikost písma v textovém poli. Pokud by se stejné celočíslné vstupní pole mělo nacházet i v okně s nastavením, bylo by nutné tyto dva ovládací prvky synchronizovat mezi sebou a zároveň se skutečnou hodnotou, která je součástí stavu textového pole. V přístupu IMGUI je velikost písma uchována na jednom místě a podle její hodnoty jsou vykresleny všechny dotčené ovládací prvky. Vykreslení složitějších prvků lze pak snadno zabalit do znovupoužitelných funkcí.

Obecně vede přístup IMGUI na kratší kód a rychlejší vývoj uživatelských rozhraní, což bylo naznačeno v řešerši u demonstračního kódu čítače s pozdravem. Tato

vlastnost se potvrdila během implementace ukázkové aplikace. Rozdíl byl nejvíce patrný při definici uživatelského rozhraní dodatečných oken. Příkladem je záložka nastavení na obrázku 5.1 se zdrojovým kódem B.4.



Obrázek 5.1: Dear ImGui a Qt: záložka *Zobrazení* v okně s nastavením

5.2 Porovnání výkonu ukázkových aplikací

V ukázkových aplikacích jsem implementoval dva testy pro srovnání jejich výkonu. První test sleduje výkon aplikace při vykreslování diagramu s velkým počtem uzlů a druhý test sleduje dobu potřebnou pro zobrazení dialogového okna s velkým množstvím ovládacích prvků. Testy jsem provedl na třech zařízeních:

1. Stolní počítač s OS Windows 10 22H2 x64; CPU Intel Xeon E3-1270 v3 (**2013**) @ 3,5 GHz $\times$ 4; RAM 16 GB; dedikovaná GPU NVIDIA GeForce GTX 1060.
2. Laptop s OS Linux Mint 22.3 Cinnamon x64; CPU Intel Core i7-6700HQ (**2015**) @ 2,6 GHz $\times$ 4; RAM 16 GB. Na tomto zařízení byly testy provedeny dvakrát: jednou s dedikovanou GPU NVIDIA GeForce GTX 960M, podruhé s integrovanou GPU Intel HD Graphics 530.
3. Laptop s OS Linux Mint 22.3 Cinnamon x64; CPU Intel Core i5-2430M (**2011**) @ 2,4 GHz $\times$ 2; RAM 8 GB; integrovaná GPU Intel HD Graphics 3000.

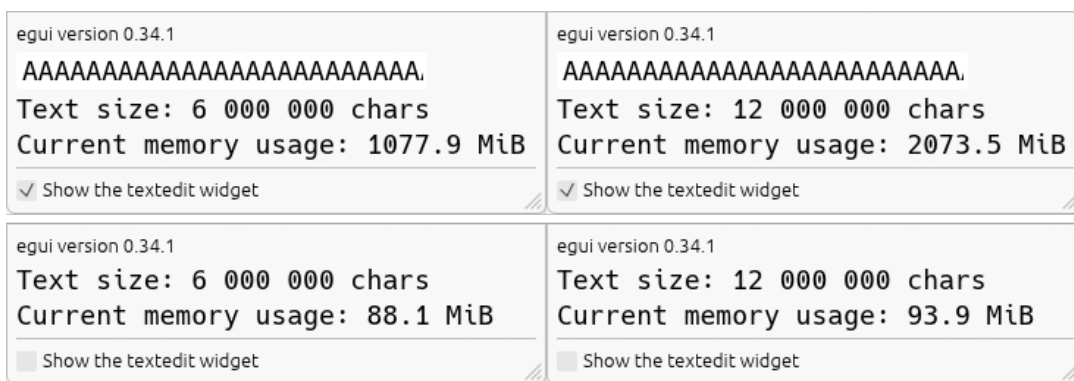
Aplikace implementované v *Dear ImGui* a *Qt* jsou sestaveny na OS Windows pomocí programu Visual Studio 2022 ve verzi 17.14.29 a na OS Linux pomocí sady GCC ve verzi 13.3.0. Aplikace implementovaná v *egui* je na obou systémech sestavena nástrojem rustc ve verzi 1.93.0.

Celkové využití CPU na OS Windows získávám pomocí čítače s cestou "`\\Informace o procesoru(_Total)\\Využití procesoru (%)`", který se nachází v rozhraní *Performance Data Helper*. Využití operační paměti procesem získávám z atributu `WorkingSetSize` v rozhraní *Process Status API*. Na OS Linux využívám jednohlavičkové knihovny, které tyto informace získávají z adresáře `/proc`.

Porovnání výkonu při vykreslování diagramu probíhá ve třech fázích. V první fázi se v diagramu nachází pouze jeden uzel, což by mělo reprezentovat aplikaci v „klidovém“ stavu. V druhé fázi jsou do vstupního dokumentu postupně přidávány definice nových uzlů, a diagram je tedy ve všech třech aplikacích pravidelně překreslován. Na konci druhé fáze se v diagramu nachází zhruba 13 tisíc uzlů. V poslední fázi se do diagramu již nic nepřidává, ale všechny uzly z předchozí fáze zůstávají vykresleny.

Obrázky B.1, B.2 a B.3 ukazují grafy s průběhem využití procesoru, využití operační paměti a doby trvání mezi dvěma měřeními. Jedná se o hodnoty naměřené na zařízení č. 2: laptop (2015) s integrovanou grafickou kartou. Jak je vidět na využití procesoru (B.1), knihovny Dear ImGui a egui mají vyšší režijní náklady (*overhead*) než knihovna Qt. Jejich spotřeba se ale příliš nemění mezi jednotlivými fázemi a knihovna Qt je v průběhu druhé fáze „dožene“.

Benchmark jsem dále spustil se skrytým víceřádkovým textovým polem a obrázek [B.4](#) ukazuje graf naměřeného zpoždění. Zde je u knihovny Qt vidět přibývajících zpoždění ve druhé fázi měření. Logika měření výkonu se v knihovně Qt nachází



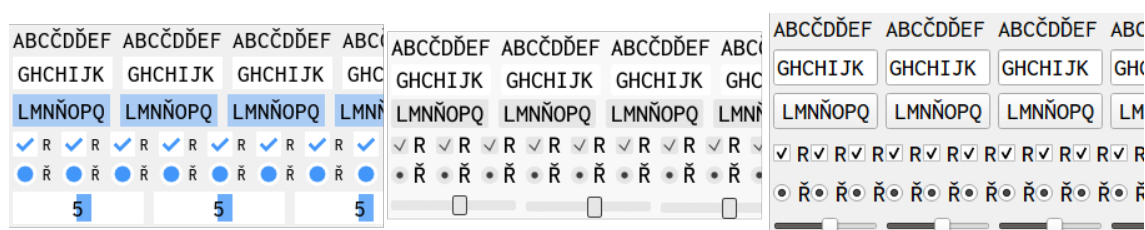
Obrázek 5.3: egui: textové pole a využití operační paměti

v samostatném koprogramu (*coroutine*), zatímco v Dear ImGui a egui je součástí hlavní smyčky. To se projevuje na kolísání doby trvání mezi měřeními ve třetí fázi.

Z důvodu přehlednosti obsahují grafy záznam pouze jednoho měření. Naměřené hodnoty na ostatních zařízeních však vykazovaly stejné trendy, což je vidět na obrázku B.5. Benchmark jsem také zkusil spustit s vypnutým zvýrazňováním syntaxe, na výsledcích se to však nijak neprojevilo. Nakonec jsem zkusil aplikaci v knihovně Dear ImGui upravit tak, aby zpracovávala vstupní dokument jen tehdy, jsou-li v něm provedeny nějaké změny. Toto se na naměřené spotřebě opět nijak neprojevilo.

5.2.2 Porovnání výkonu při vykreslování ovládacích prvků

Druhé porovnání výkonu spočívá ve zobrazení dialogového okna, které obsahuje velké množství ovládacích prvků. Měří se, jak dlouho trvá takové okno zobrazit a zase zavřít. Okno nejprve obsahuje jednu „dávku“ ovládacích prvků, v níž se nachází několik textových popisků, textových polí, tlačítek, zaškrťovacích tlačítek (*checkbox*), přepínačů (*radio button*) a posuvníků. Celkem se v jedné dávce nachází 172 ovládacích prvků. Obrázek 5.4 ukazuje výřez okna ve všech třech knihovnách.



Obrázek 5.4: Benchmark ovládacích prvků v Dear ImGui, egui a Qt

Doba potřebná pro zobrazení a zavření okna je měřena vždy desetkrát. Začíná se na jedné dávce a po každých deseti měřeních je počet dávek zdvojnásoben. Měření končí, jakmile se na zobrazení okna čeká déle než 15 vteřin.

Obrázek B.6 ukazuje výsledky měření. Aplikace v knihovně Qt končí jako první a její spotřeba operační paměti se mírně zvyšuje. Aplikace v egui je v tomto ohledu sice rychlejší, ale opět je zde problém se spotřebou operační paměti. V této knihovně

jsem zkusil také zobrazovat pouze „prázdné“ ovládací prvky bez žádného textu, ale na výsledcích se to neprojevilo. Aplikace v Dear ImGui se dostává do řádu desítek milionů vykreslených ovládacích prvků a jako jediná si drží téměř konstantní spotřebu operační paměti. Na využití procesoru se tento test nijak neprojevil.

Z výsledků vyplývá, že knihovny s přístupem ImGui mají vyšší režijní náklady a nehodí se tak pro aplikace s důrazem na nižší spotřebu. Využití procesoru není ale příliš závislé na aktuálním stavu aplikace, což nelze říci o knihovně Qt. Před implementací libovolné aplikace v knihovně s přístupem ImGui je vhodné prozkoumat chování jednotlivých ovládacích prvků, které se mají nacházet v navrhovaném řešení. Tak se může předem přijít na problémy, jako je spotřeba operační paměti u textového pole v knihovně egui. Poté je možné vyhodnotit, zdali jsou nalezené problémy relevantní pro navrhované řešení.

6 Závěr

Hlavním cílem této práce bylo prozkoumat, jak jsou existující knihovny s přístupem ImGui vhodné k tvorbě aplikací pro koncové uživatele. V teoretické části práce jsem nejprve popsal princip přístupu ImGui a porovnal jej s běžným přístupem pro tvorbu GUI aplikací, který lze označit jako RMGUI. Následně jsem v rešerši představil existující knihovny, které umožňují vývoj s přístupem ImGui. Pro potřeby ukázkové aplikace jsem zkoumal především schopnosti knihoven vykreslit a obsloužit víceřádkové textové pole. Jako nejvhodnější zástupce pro implementaci ukázkové aplikace jsem zvolil knihovny *Dear ImGui* a *egui*. V rešerši také krátce představuji knihovnu *Qt*, kterou jsem vybral jako zástupce přístupu RMGUI.

V praktické části práce jsem začal návrhem ukázkové aplikace. Jedná se o editor diagramů, který na základě uživatelem dodaného textového popisu vykreslí odpovídající diagram. Pokouším se zde o unikátnější přístup, kdy uživatel ve vstupním textu nepopisuje pouze význam diagramu, ale i jeho přesný vzhled. Aplikace je navržena tak, aby bylo možné při její implementaci prozkoumat větší množství funkcí jednotlivých knihoven. Zároveň má ale praktické využití, a nejedná se tak pouze o „galerii ovládacích prvků“.

Navrženou aplikaci jsem postupně implementoval v knihovnách *Dear ImGui*, *egui* a *Qt*. Aplikace v *Dear ImGui* a *Qt* jsou implementovány v jazyce C++ a aplikace v knihovně *egui* je implementována v jazyce Rust. Výsledkem jsou tři programy, ve kterých lze vytvořit blokové schéma či základní vývojový diagram s možností exportu do souboru ve formátu SVG. V této části práce popisují, jak se v jednotlivých knihovnách programuje uživatelské rozhraní a na jaké problémy jsem během implementace narazil. Větší pozornost zde opět dostává víceřádkové textové pole, kde v knihovnách s přístupem ImGui může být problém např. zobrazit horizontální posuvnou lištu, zvýrazňovat části textu, nebo zvýrazňovat syntax.

Během implementace se potvrdilo, že přístup ImGui vede na kratší a přímočařejší kód, a urychluje proto vývoj některých částí aplikace. Tento přístup je založen na programování v nekonečné smyčce, kde může hrát roli pořadí provedených operací. Všechna data aplikace jsou držena na straně aplikačního kódu a knihovně je popisováno, jak má na základě těchto dat vykreslovat uživatelské rozhraní.

Přístup RMGUI je založen na programování callback metod a synchronizaci stavů. Oproti psaní kódu ve smyčce, kde je jasně daná posloupnost operací, zde s přibývajícemi funkcemi přibývá i složitost komunikace mezi jednotlivými prvky. Data aplikace jsou zde součástí stavu ovládacích prvků, které spravuje daná knihovna. To vede na složitější aplikační kód v případech, kdy má být možné upravovat hodnotu jedné proměnné z více částí aplikace.

Z pohledu dostupných funkcí na tom byla knihovna Qt nejlépe. Bez externích závislostí dokáže např. exportovat obsah kreslicího plátna do formátu SVG nebo zobrazit nativní dialogová okna pro výběr souboru. Výchozí víceřádkové textové pole zde také bylo nejschopnější. Zkoumané knihovny s přístupem ImGui jsou však v aktivním vývoji, a tak se v určitých ohledech mohou časem zlepšit.

V knihovně Dear ImGui jsem během implementace objevil dvě nezdokumentované vizuální chyby, které jsem nahlásil jako *issue* na platformě GitHub. První se týkala víceřádkového textového pole, kdy uživatel mohl měnit velikost jeho vnitřního okraje (*padding*). Druhá chyba se týká rozmazaného textu v případě, kdy je použito nové výchozí písmo knihovny v kombinaci s balíkem FreeType. První chyba již byla opravena a druhá je nastavena jako jeden z milníků pro vydání další vedlejší verze (*minor version*).

Nakonec jsem v ukázkových aplikacích implementoval dva testy pro srovnání jejich výkonu. Jeden vykresluje diagram s velkým počtem uzlů a druhý měří dobu potřebnou pro zobrazení dialogového okna s velkým počtem ovládacích prvků. Z výsledků je vidět, že knihovny Dear ImGui a egui mají v základu vyšší využití procesoru a operační paměti. Jejich spotřeba je ale relativně stabilní a knihovna Qt, kde v tomto ohledu více závisí na aktuálním stavu aplikace, je v určitých situacích může svými systémovými nároky překonat.

Nečekaným objevem měření je nadměrné využití operační paměti knihovnou egui v případech, kdy textové pole obsahuje větší množství znaků. Před začátkem vývoje je proto vhodné otestovat chování konkrétních ovládacích prvků. Velká spotřeba operační paměti knihovnou egui se projevila také při testu zobrazení okna s velkým počtem ovládacích prvků. V této kategorii vedla aplikace implementovaná v Dear ImGui, která si navíc jako jediná držela konstantní spotřebu.

Seznam použité literatury

1. MCBRIDE, Mike. *KDE Documentation Visual Dictionnary* [online]. 2003-11. [cit. 2026-05-03]. Dostupné z: <https://web.archive.org/web/20160425112124/http://l10n.kde.org/docs//visualdict/dict.html>.
2. EXCOFFIER, Thierry. *Zero Memory Widgets* [online]. Lyon, 2003-03 [cit. 2025-11-08]. Výzk. zpr., 20030311. Laboratoire d'InfoRmatique en Image et Systèmes d'information. Dostupné z: https://perso.univ-lyon1.fr/thierry.excoffier/ZMW/rr_2003_03_11.pdf.
3. MURATORI, Casey. *Immediate-Mode Graphical User Interfaces (2005)* [online]. 2016-05. [cit. 2025-11-01]. Dostupné z: https://caseymuratori.com/blog_0001.
4. SANCHEZ, Julio a CANTON, Maria P. *DirectX® 3D Graphics Programming Bible*. Foster City: IDG Books Worldwide, 2000. ISBN 0-7645-4633-3.
5. WHITE, Steven. *Retained Mode Versus Immediate Mode* [online]. 2019-08. [cit. 2025-11-01]. Dostupné z: <https://learn.microsoft.com/en-us/windows/win32/learnwin32/retained-mode-versus-immediate-mode>.
6. SARTOR, Sam. *Statefulness in GUIs* [online]. 2023-02. [cit. 2025-11-05]. Dostupné z: <https://samsartor.com/guis-1>.
7. CORNUT, Omar. *Dear ImGui wiki* [online]. 2014-11. [cit. 2025-11-09]. Dostupné z: <https://github.com/ocornut/imgui/wiki>.
8. ERNERFELDT, Emil. *egui docs* [online]. 2020-05. [cit. 2025-11-09]. Dostupné z: <https://docs.rs/egui/latest/egui/index.html>.
9. BACHE-WIIG, Jens. *Cleaning up styles in Qt5 and adding Fusion* [online]. 2012-10. [cit. 2026-03-31]. Dostupné z: <https://www.qt.io/blog/2012/10/30/cleaning-up-styles-in-qt5-and-adding-fusion>.
10. CORNUT, Omar. *10 years of Dear ImGui* [online]. 2024-08. [cit. 2026-04-06]. Dostupné z: <https://github.com/ocornut/imgui/issues/7892>.
11. METTKE, Micha. *Nuklear wiki* [online]. 2016-02. [cit. 2026-04-21]. Dostupné z: <https://github.com/Immediate-Mode-UI/Nuklear/wiki>.
12. *Microui v2: An Implementation Overview* [online]. 2020-04. [cit. 2026-04-22]. Dostupné z: https://rxi.github.io/microui_v2_an_implementation_overview.html.

13. SANTAMARIA, Ramon. *raygui* [online]. 2016-06. [cit. 2026-04-25]. Dostupné z: <https://github.com/raysan5/raygui/blob/master/README.md>.
14. NAUR, Elias. *Gio UI* [online]. 2019-10. [cit. 2026-04-25]. Dostupné z: <https://gioui.org/>.
15. VANDERSON, David. *DVUI* [online]. 2024-03. [cit. 2026-04-25]. Dostupné z: <https://david-vanderson.github.io/>.
16. *About Qt* [online]. 2026-02. [cit. 2026-04-21]. Dostupné z: https://wiki.qt.io/About_Qt.
17. PEDERSEN, Lasse Apalnes; EINARSSON, Svavar Skuli; RIKHEIM, Fredrik Arne a SANDNES, Frode Eika. User interfaces in dark mode during daytime – Improved productivity or just cool-looking? In: *Universal Access in Human-Computer Interaction. Design Approaches and Supporting Technologies*. Cham: Springer International Publishing, 2020, s. 178–187. ISBN 978-3-030-49282-3. Dostupné z DOI: [10.1007/978-3-030-49282-3_13](https://doi.org/10.1007/978-3-030-49282-3_13).
18. PRESTON-WERNER, Tom; GEDAM, Pradyun et al. *TOML v1.0.0* [online]. 2021-11. [cit. 2026-04-02]. Dostupné z: <https://toml.io/en/v1.0.0>.
19. JÁKÓ, Balázs. *ImGuiColorTextEdit* [online]. 2017-10. [cit. 2026-04-10]. Dostupné z: <https://github.com/BalazsJako/ImGuiColorTextEdit/blob/master/README.md>.
20. GITHUB. *About forks - GitHub Docs* [online]. 2021-10. [cit. 2026-04-12]. Dostupné z: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/about-forks>.
21. UNICODE CONSORTIUM. *The Unicode Standard* [online]. South San Francisco, 2025 [cit. 2026-04-19]. Tech. zpr., Version 17.0.0. ISBN 978-1-936213-35-1. Dostupné z: <https://www.unicode.org/versions/Unicode17.0.0>.
22. WORLD WIDE WEB CONSORTIUM. *Scalable Vector Graphics (SVG) 1.1* [online]. Second Edition. 2011-08. [cit. 2026-04-20]. Dostupné z: <https://www.w3.org/TR/SVG11>.
23. KLABNIK, Steve; NICHOLS, Carol a KRYCHO, Chris. *The Rust Programming Language*. 3rd Edition. No Starch Press, 2026. ISBN 9781718504448.
24. ERNERFELDT, Emil. *Z-ordering of shapes* [online]. 2022-04. [cit. 2026-04-24]. Dostupné z: <https://github.com/emilk/egui/issues/1516>.

A Příloha

Praktická část této práce se nachází v několika repozitářích. Odkazy na tyto repozitáře se nachází na webu radekmocek.github.io/DPDoc, který zároveň obsahuje dokumentaci k použití jazyka pro popis diagramů. Obsah všech repozitářů je také součástí přílohy.

B Ukázkové kódy a výsledky porovnání výkonu ukázkových aplikací

```
void App::update() {
    static int counter = 1;

    ImGui::Begin("Hello Counter");

    ImGui::BeginDisabled(counter == 1);
    if (ImGui::Button("-")) {
        counter--;
    }
    ImGui::EndDisabled();

    ImGui::Text("%d", counter);

    ImGui::BeginDisabled(counter == 5);
    if (ImGui::Button("+")) {
        counter++;
    }
    ImGui::EndDisabled();

    if (counter == 3) {
        ImGui::Text("Hello!");
    }

    ImGui::End();
}
```

Zdrojový kód B.1: Kód demonstrující přístup ImGui v knihovně Dear ImGui

```

// Předpokládejme třídu App se členy counter, layout,
// button_counter_decrement, label_counter_value atd.

void App::init() {
    button_counter_decrement = new QPushButton("-");
    layout->addWidget(button_counter_decrement);
    connect(button_counter_decrement, &QPushButton::clicked,
            this, &App::on_button_decrement_clicked);

    label_counter_value = new QLabel("");
    layout->addWidget(label_counter_value);

    button_counter_increment = new QPushButton("+");
    layout->addWidget(button_counter_increment);
    connect(button_counter_increment, &QPushButton::clicked,
            this, &App::on_button_increment_clicked);

    label_hello = new QLabel("Hello!");
    layout->addWidget(label_hello);

    counter = 1;
    on_counter_update();
}

void App::on_button_decrement_clicked() {
    counter--;
    on_counter_update();
}

void App::on_button_increment_clicked() {
    counter++;
    on_counter_update();
}

void App::on_counter_update() {
    label_counter_value->setText(QString::number(counter));

    button_counter_decrement->setEnabled(counter != 1);
    button_counter_increment->setEnabled(counter != 5);

    label_hello->setVisible(counter == 3);
}

```

Zdrojový kód B.2: Kód demonstrující přístup RMGUI v knihovně Qt

```

// Mějme aplikaci se členem `int m_number`,
// jehož hodnotu chceme měnit v GUI.

constexpr auto number_min = 0; // Minimální hodnota člena
constexpr auto number_max = 10; // Maximální hodnota člena
// „Velikost kroku“, která se po stiknutí tlačítka +/- přičte/odečte
constexpr auto step_size = 2;

// --- Dear ImGui (C++) ---
ImGui::InputInt("##spinbox", &m_number, step_size);
m_number = std::clamp(m_number, number_min, number_max);

// --- egui (Rust) ---
ui.horizontal(|ui| {
    ui.add(egui::DragValue::new(&mut m_number));
    if ui.button("-").clicked() {
        m_number -= step_size;
    }
    if ui.button("+").clicked() {
        m_number += step_size;
    }
    m_number += m_number % step_size;
    m_number = m_number.clamp(number_min, number_max);
});

// --- Qt (C++) ---
const QPointer spinbox = new QSpinBox();
spinbox->setMinimum(number_min);
spinbox->setMaximum(number_max);
spinbox->setSingleStep(step_size);
spinbox->setValue(m_number);

main_layout->addWidget(spinbox);
connect(spinbox, &QSpinBox::valueChanged, [this](int value) {
    m_number = value;
});

// Dear ImGui / egui:
// Změna hodnoty proměnné `m_number` jinde v kódu se
// automaticky projeví na hodnotě zobrazené v ovládacím prvku.

// Qt:
// Po změně hodnoty proměnné `m_number` jinde v kódu musíme také
// aktualizovat hodnotu drženou v ovládacím prvku `spinbox`.

```

Zdrojový kód B.3: Porovnání knihoven: celočíselné vstupní pole

```

// --- Dear ImGui --- --- --- --- --- --- --- --- --- --- ---
if (ImGui::BeginTabBar("PreferencesTabBar")) {
    // ... (Zde se nachází definice ostatních záložek nastavení) ...
    if (ImGui::BeginTabItem("View")) {
        ImGui::Checkbox("Toolbar", &m_do_show_toolbar);
        ImGui::Checkbox("Canvas grid", &m_do_show_grid);
        ImGui::Checkbox("Secondary canvas toolbar", &m_do_show_sct);
        ImGui::EndTabItem();
    }
    ImGui::EndTabBar();
}
// Změna stavu zaškrťovacího tlačítka se v aplikaci projeví okamžitě.

// --- Qt --- --- --- --- --- --- --- --- --- --- ---
const QPointer layout = new QVBoxLayout();
const QPointer tabs = new QTabWidget();

// ... (Zde se nachází definice ostatních záložek nastavení) ...

const QPointer layout_view_wrapper = new QWidget();
const QPointer layout_view = new QVBoxLayout(layout_view_wrapper);

m_view_toolbar = new QCheckBox("Toolbar");
m_view_toolbar->setChecked(m_state.do_show_toolbar);
layout_view->addWidget(m_view_toolbar);

m_view_canvas_grid = new QCheckBox("Canvas grid");
m_view_canvas_grid->setChecked(m_state.do_show_canvas_grid);
layout_view->addWidget(m_view_canvas_grid);

m_view_secondary_toolbar = new QCheckBox("Secondary canvas toolbar");
m_view_secondary_toolbar->setChecked(m_state.do_show_sct);
layout_view->addWidget(m_view_secondary_toolbar);

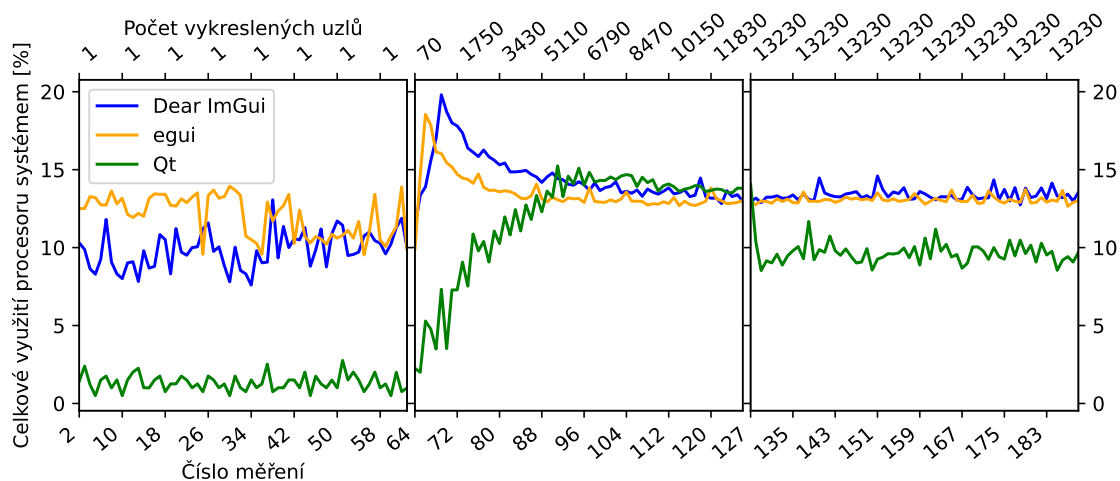
tabs->addTab(layout_view_wrapper, "View");
layout->addWidget(tabs);

// Okno s nastavením obsahuje tlačítko, které aplikuje všechny změny.
// Implementovat chování podobné IMGUI by kód dále zkomplikovalo.
const QPointer button_apply = new QPushButton("Apply all settings");
layout->addWidget(button_apply);
connect(button_apply, &QPushButton::clicked, [this] { ... });

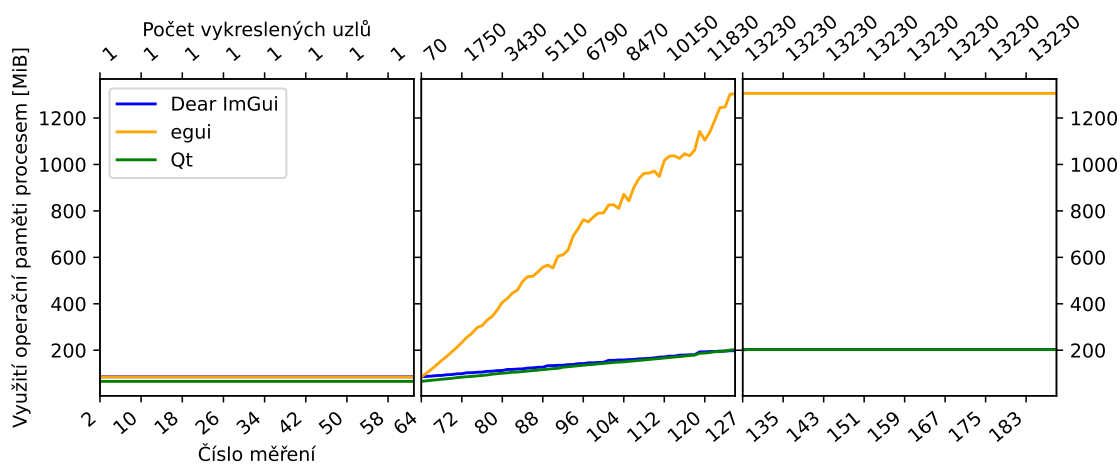
setLayout(layout);

```

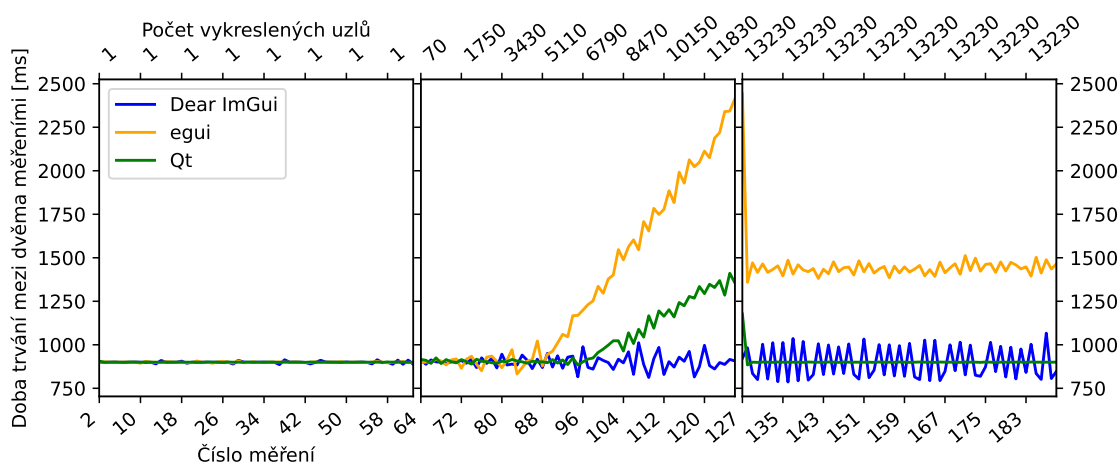
Zdrojový kód B.4: Dear ImGui a Qt: záložka *Zobrazení* v okně s nastavením



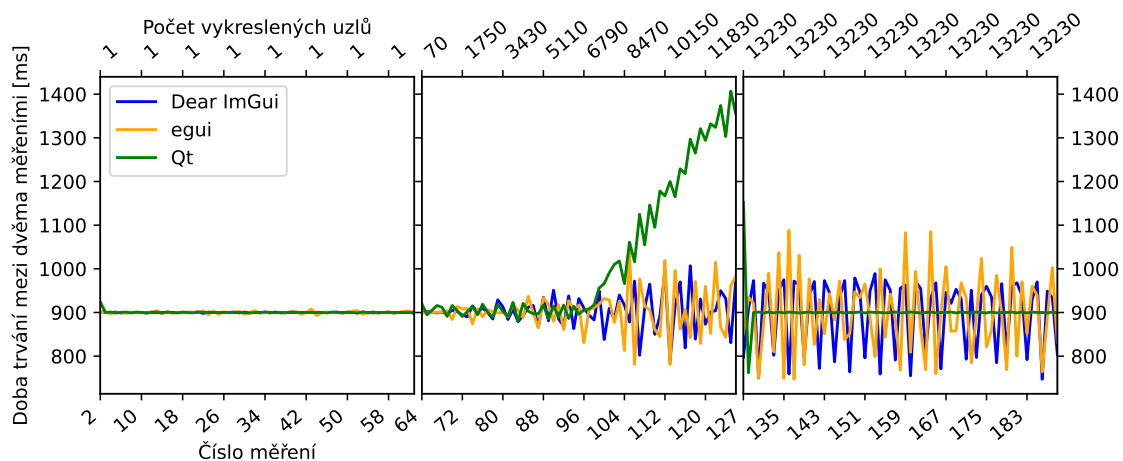
Obrázek B.1: Benchmark vykreslení diagramu: celkové využití procesoru systémem



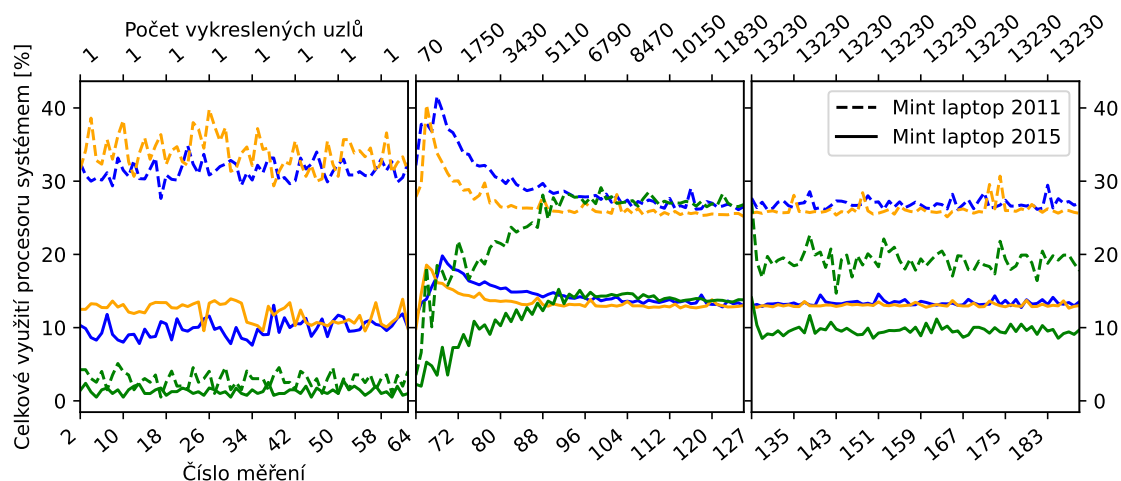
Obrázek B.2: Benchmark vykreslení diagramu: využití operační paměti procesem



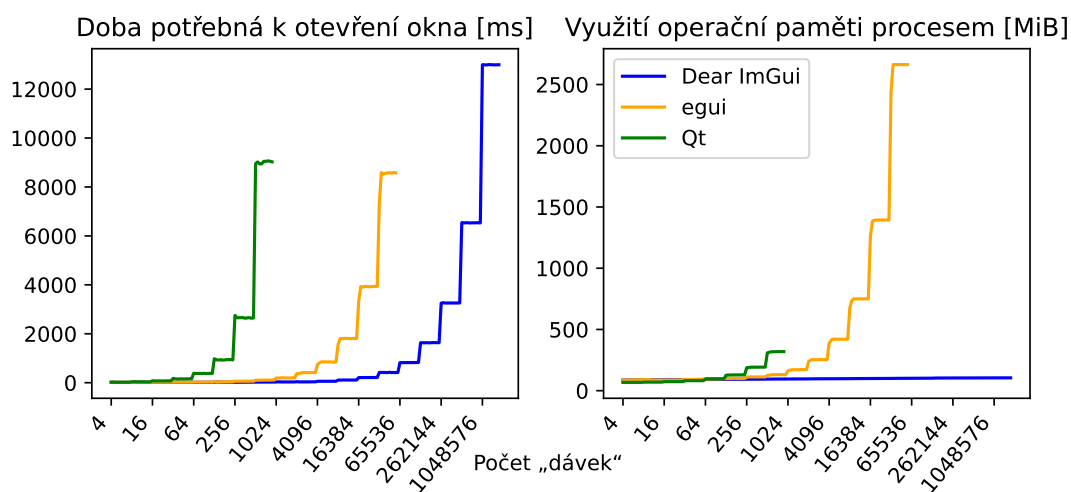
Obrázek B.3: Benchmark vykreslení diagramu: doba trvání mezi dvěma měřeními



Obrázek B.4: Benchmark vykreslení diagramu: doba trvání mezi dvěma měřeními bez textového pole



Obrázek B.5: Benchmark vykreslení diagramu: stejný trend využití procesoru na dvou různých zařízeních



Obrázek B.6: Benchmark ovládacích prvků